Digital logic Design

* Numbering Systems :-

① Decimal (Base 10) → [0-9]

② Binary (Base 2) → [0-1]

③ Octal (Base 8) → [0-7]

④ Hexadecimal (Base 16) → [0-9] + [A(10) B(11) C(12) D(13) E(14) F(15)]

* Decimal representation :-

$*(7230)_{10}$

$7000 \quad 200 \quad 30 \quad 0$

$7\times10^3 + 2\times10^2 + 3\times10^1 + 0\times10^0$

* يمين الفاصلة ($*10^{-1}$)

يسار الفاصلة ($*10^{x}$)

$*(146.53)_{10}$

$1\times10^2 + 4\times10^1 + 6\times10^0 + 5\times10^{-1} + 3\times10^{-2}$

* to convert from any System (Base(r)) to decimal we multiply the digit by the corrosponding weight and final the summation.

Ex :- Convert the following to decimal :-

① $(10\,11.11)_2$

$= 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-2}$

$= 8 + 2 + 1 + \frac{1}{2} + \frac{1}{4}$

$= \boxed{(11.75)_{10}}$

Scanned with CamScanner

② $(127.4)_8$

$= 1\times8^2 + 2\times8^1 + 7\times8^0 + 4\times8^{-1}$

$= 64 + 16 + 7 + \frac{1}{2}$

$= (87.5)_{10}$

③ $(10B.A)_{16}$

$= 1\times16^2 + 0\times16^1 + 11\times16^0 + 10\times16^{-1}$

$= (267.625)_{10}$

④ $(142)_5$

$= 1\times5^2 + 4\times5^1 + 2\times5^0$

$= 25 + 20 + 2$

$= (47)_{10}$

⑤ $(11010.11)_2$ (16 8 4 2 1 ½ ¼) $= (26.75)_{10}$

$= 1\times2^4 + 1\times2^3 + 1\times2^1 + 1\times2^{-1} + 1\times2^{-2}$

$= 16 + 8 + 2 + \frac{1}{2} + \frac{1}{4}$

$= (26.75)_{10}$

⑥ $(4021.2)_5$

$= 4\times5^3 + 0\times5^2 + 1\times5^1 + 2\times5^{-1} + 2\times5^1$

$= (511.4)_{10}$

⑦ $(127.4)_8$

$= 1\times8^2 + 2\times8^1 + 7\times8^0 + 4\times8^{-1}$

$= 64 + 16 + 7 + \frac{1}{2}$

$= (87.5)_{10}$

⑧ $(B65F)_{16}$

$= 12\times16^3 + 6\times16^2 + 5\times16^1 + 15\times16^0$

$= (46687)_{10}$

⑨ $(10010)_2$ (16 8 4 2 1) $= 16+2 = (18)_{10}$

$= 1\times2^4 + 1\times2^1$

$= (18)_{10}$

* Notes :- (Binary System) :-

→ A Binary digit is called a bit

8 bits (b) → Byte (B)

2^{10} bit (b) = Kilo (K)

2^{20} bit (b) = Mega (M)

2^{30} bit (b) = Giga (G)

2^{40} bit (b) = Tera (T)

Scanned with CamScanner

* Faster Way to Convert Binary → Decimal :-

① $(101)_2 = (5)_{10}$

② $(1101)_2 = (13)_{10}$

③ $(10010)_2 = (18)_{10}$

④ $(11010)_2 = (26)_{10}$

⑤ $(101011)_2 = (43)_{10}$

* To Convert from Decimal to any system with base (r) we divide the decimal by the system I want to convert then the remainder is the answer.

→ For the whole part divide by base (r) and Keep remainder.

→ For the fraction multiply by base (r).

Ex :- Convert to Bainary :-

① $(41)_{10} = (101001)_2$

41	/2	
20	1	least S.B
10	0	
5	0	
2	1	
1	0	
0	1	Most S.B

② $(0.625)_{10} = (0.101)_2$

Most S.B

$0.625 \times 2 = 1.25$

$0.25 \times 2 = 0.5$

$0.5 \times 2 = 1.0$

$0 \times 2 = 0$ least S.B

Scanned with CamScanner

64 32 16 8 4 2 1

65 → 1 0 0 0 0 0 1

64 32 16 8 4 2 1

67 → 1 0 0 0 0 1 1

16 8 4 2 1

17 → 1 0 0 0 1

32 16 8 4 2 1

60 → 1 1 1 1 0 0

Scanned with CamScanner

③ $(12.125)_{10} = (1100.001)_2$

12	÷2
6	0
3	0
1	1
0	1

$0.125 \times 2 = 0.25$

$0.25 \times 2 = 0.5$

$0.5 \times 2 = 1.0$

$0 \times 2 = 0$

$(0.001)_2 = 0.125$

$(1100)_2 = 12$

Ex :- Decimal to octal :-

① $(41)_{10} = (51)_8$

41	÷8
5	1
0	5

② $(12.125)_{10} = (14.1)_8$

12	÷8
1	4
0	1

$0.125 \times 8 = 1.0$

$0 \times 8 = 0$

$(0.1)_8$

$(14)_8 = 12$ ✓

Ex :- Decimal to Hex :-

① $(41)_{10} = (29)_{16}$

41	÷16
2	9
0	2

$(29)_{10} = 41$

② $(12.125)_{10} = (\quad)_{16}$

12	÷16

Scanned with CamScanner

Notes :- * Each Octal digit = 3 bits ($2^3 = 8$)

* Each Hex digit = 4 bits ($2^4 = 16$)

* Convert from octal to binary → replace each digit by 3 bits.

Hex to binary → replace each digit by 4 bits.

* Convert from binary to Octal → group 3 bits → replace by octal digit

binary to Hex → group 4 bits → replace by Hex digit

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Hex	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10 A	1010
11 B	1011
12 C	1100
13 D	1101
14 E	1110
15 F	1111

8 4 2 1

0 = 0000

A 10 = 1010

2 = 0010

Scanned with CamScanner

Ex :- Convert :-

① $(01001110.)_2 \rightarrow (4E)_H$ (افتراض)

② $(01001110.11001100)_2 \rightarrow (2A6.C4)_H$

② $(D5.5)_{16} \rightarrow (11010101.0101)_2$

③ $(73.4)_8 \rightarrow (111011.100)_2$

④ $(001001110)_2 \rightarrow (116)_8$

⑤

⑥ $(001010100110.110001)_2 \rightarrow (1246.61)_8$

* To convert from Hex / octa to octa / Hex, we must convert to Binary first.

Ex :- ① $(317)_8 \rightarrow (0CF)_{16}$ ② $(45.12)_8 \rightarrow (25.28)_{16}$

③ $(136)_{16} \rightarrow (0466)_8$ ④ $(72.5A)_{16} \rightarrow (162.264)_8$

Scanned with CamScanner

* Mathematical operations :-

• Binary :-

① Addition :-

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 (Carry = 1)

32 16 8 4 2 1

45 1 0 1 1 0 1

+ 39 1 0 0 1 1 1 +

= 84 1 0 1 0 1 0 0 ✓

② Subtraction :-

0 - 0 = 0

0 - 1 = 1 (Borrow = 1)

1 - 0 = 1

1 - 1 = 0

32 16 8 4 2 1

45 1 0 1 1 0 1

- 39 1 0 0 1 1 1 -

= 6 0 0 0 1 1 0 ✓

③ Multiplication :-

1 0 1 1 #1

x 1 0 1 x5

① 1 0 1 1

0 0 0 0 +

1 0 1 1 +

1 1 0 1 1 1 55 ✓

32 16 8 4 2 1

Scanned with CamScanner

* Binary Addition :-

x	y	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ex :-

$(0101)_2 + (0100)_2 = (1001)_2$

$$\begin{array}{r} {\scriptstyle 1}\ 0\ 1\ 0\ 1 \\ +\ 0\ 1\ 0\ 0 \\ \hline 1\ 0\ 0\ 1 \end{array}$$

* Hexadecimal Addition :-

Ex :-

① $(3C + 12)_{16} = (4E)_{16}$

$$\begin{array}{r} 3\ C \\ +\ 1\ 2 \\ \hline 4\ E \end{array}$$

② $(99 + 37)_{16} = (D0)_{16}$

$$\begin{array}{r} {\scriptstyle 1}\ 9\ 9 \\ +\ 3\ 7 \\ \hline D\ 0 \end{array} \qquad \begin{array}{r} {\scriptstyle 9}\ {\scriptstyle 7} \\ +\ \ \ \\ \hline 16 \\ -\ 16 \\ \hline 0 \end{array}$$

③ $(101 + 011)_{16} = (1000)_{16}$

$$\begin{array}{r} {\scriptstyle 1}\ 0\ 1 \\ +\ 0\ 1\ 1 \\ \hline 1\ 0\ 0\ 0 \end{array}$$

④ $BFC + 8AB = (14A7)_{16}$

$$\begin{array}{r} {\scriptstyle 1}\ {\scriptstyle 1}\ {\scriptstyle 1} \\ B\ F\ C \\ +\ 8\ A\ B \\ \hline 1\ 4\ A\ 7 \end{array} \qquad \begin{array}{r} 12 \\ +\ 11 \\ \hline 23 \\ -\ 16 \\ \hline 7 \end{array} \qquad \begin{array}{r} {\scriptstyle 1}\ 15 \\ +\ \ 10 \\ \hline 26 \\ -\ 16 \\ \hline 10 \end{array} \qquad \begin{array}{r} {\scriptstyle 1}\ 11 \\ +\ \ 8 \\ \hline 20 \\ -\ 16 \\ \hline 4 \end{array}$$

* Octal Addition :-

Ex :-

① $\begin{array}{r} {\scriptstyle 1}\ {\scriptstyle 1}\ {\scriptstyle 1} \\ 3\ 7 \\ +\ 5\ 4 \\ \hline (1\ 1\ 3)_8 \end{array}$ $\begin{array}{r} 7 \\ +\ 4 \\ \hline 11 \\ -\ 8 \\ \hline 3 \end{array}$ $\begin{array}{r} {\scriptstyle 1} \\ 3 \\ +\ 5 \\ \hline 9 \\ -\ 8 \\ \hline 1 \end{array}$

② $\begin{array}{r} {\scriptstyle 1}\ {\scriptstyle 1}\ {\scriptstyle 1} \\ 1\ 2\ 3 \\ +\ 7\ 7\ 6 \\ \hline (1\ 1\ 2\ 1)_8 \end{array}$ $\begin{array}{r} 3 \\ +\ 6 \\ \hline 9 \\ -\ 8 \\ \hline 1 \end{array}$ $\begin{array}{r} {\scriptstyle 1} \\ 2 \\ +\ 7 \\ \hline 10 \\ -\ 8 \\ \hline 2 \end{array}$ $\begin{array}{r} {\scriptstyle 1} \\ 1 \\ +\ 7 \\ \hline 9 \\ -\ 8 \\ \hline 1 \end{array}$

Scanned with CamScanner

* Binary Multiplication :-

Ex :-

① $9 \times 2 = 18$

$$\begin{array}{r} 1001 \quad 9 \\ \times \\ \hline 10 \quad 2 \\ 0000 \\ 1001 \quad + \\ \hline 10010 \quad 18 \end{array}$$

② $3 \times 3 = 9$

$$\begin{array}{r} 11 \quad 3 \\ \times \\ \hline 11 \quad 3 \\ 11 \\ 11 \quad + \\ \hline 1001 \quad 9 \end{array}$$

③ $11 \times 3 = 33$

$$\begin{array}{r} 1011 \quad 11 \\ \times \\ \hline 11 \quad 3 \\ 1011 \\ 1011 \\ \hline 100001 \quad 33 \end{array}$$

Scanned with CamScanner

* Binary Codes :-

* Binary Coded Decimal (BCD) :

- A number with "n" decimal digits will require "4n" bits in BCD.
- The number in Binary, BCD have the same representation if the decimal number has one digit only.

Ex :- (BCD)

① 452 → 0100 0101 0010 (4, 5, 2)

② 9 → Binary 1001, BCD 1001

③ 18 → Binary 10010, BCD 0001 1000 (1, 8)

* BCD Addition :-

- We can perform arithmatic operation directly when they are stored as BCD
- When the binary sum is equal to or less than (9 → 1001) [without carry]
  ⇒ BCD digit correct ✓
- Otherwise the BCD must be corrected by adding (6 → 0110)

Ex :- (BCD Addition)

① 6 + 2 = 8

6 → 0110
2 → 0010 +
8 → 1000

② 5 + 5 = 10

5 → 0101
5 → 0101 +
1010 > 9 (not correct)
0110 +
10000
1 0

③ 7 + 9 = 16

7 → 0111
9 → 1001 +
10000 > 9 (not correct)
0110 +
10110
1 6

④ 134 + 149 = 283

0001 0011 0100 (1, 3, 4)
0001 0100 1001 +
0010 0111 1101 > 9 (not correct)
0110 +
0010 1000 0011
2 8 3

Scanned with CamScanner

⑤ 184 + 576 = 760

```
  1     8     4
 0001  1000  0100
  5     7     6   +
 0101  0111  0110
 0110  1111  1010   +1 correct
                0110  +
 ------------------
 0111  0000  0000   +
       0110
 ------------------
 0111  0110  0000
  7     6     0
```

⑥ 73 + 62 = 135

```
  1     7     3
 0111  0011
  6     2   +
 0110  0010
 0101  0101   +
 ------------
 1011  0101
 1     3     5
```

* Other Decimal Codes :-

Decimal	2,4,2,1	Excess-3	8,4,-2,-1
0	0000	0011	0000
1	0001	0100	0111
2	1000	0101	0110
3	1001	0110	0101
4	1010	0111	0100
5	1011	1000	1011
6	1100	1001	1010
7	1101	1010	1001
8	1110	1011	1000
9	1111	1100	1111

Scanned with CamScanner

* Gray Code :-

- Only one bit changes between consective numbers.

Binary	Gray
0 0 0	0 0 0
0 0 1	0 0 1
0 1 0	0 1 1
0 1 1	0 1 0
1 0 0	1 1 0
1 0 1	1 1 1
1 1 0	1 0 1
1 1 1	1 0 0

* How to convert from binary to Gray code :-

① 0 0 0 → 0 0 0

② 0 0 1 → 0 0 1

③ 0 1 0 → 0 1 1

④ 0 1 1 → 0 1 0

⑤ 1 0 0 → 1 1 0

⑥ 1 0 1 → 1 1 1

⑦ 1 1 0 → 1 0 1

⑧ 1 1 1 → 1 0 0

* When we change from binary to Gray code we put the first number the same then we compare with the next number.

Same = 0

Diff = 1

Scanned with CamScanner

Ex :- Convert to Gray Code :-

① 1 0 1 0 1 1 → 1 1 1 1 1 0

② 1 0 0 0 1 1 → 1 1 0 0 1 0

* How to convert From Gray to Binary :-

① 0 0 0 (same same) → 0 0 0 Decimal 0

② 1 1 0 (Diff same) → 1 0 0 Decimal 4

③ 1 0 0 (Diff Same) → 1 1 1 Decimal 7

④ G = 1 1 0 0 1 0
B = 1 0 0 0 1 1

⑤ G = 1 1 1 1 1 0
B = 1 0 1 0 1 1

* ASCII Code :-

- 7 bits are used to represent different characters.
(numbers, symbols, letters, special characters).

* Binary numbers operations :-

* Complements :-

- Complements are used to preform the subtraction operation.

A - B = (A) + (-B)

Scanned with CamScanner

[1] one's complement (First complement) :-

- Change all 0-bits to 1-bits
  1-bits to 0-bits

Ex :- Find the First complement :-

① $(110101001)_2 \xrightarrow{1's} 001010110$

② $(100010101)_2 \xrightarrow{1's} 011101010$

[2] Two's complement (second complement) :-

- To Find the 2nd complement We have two methods:
  ① Find the 1st complements and then add "1".
  ② Scan binary number From right to left, when finding the First "1" bit, Keep it and then all the next bits.

Ex :- Find the second complement :-

① $(11010100)_2$

* 1st comp: 00101011

$+1$

00101100

* $110\,1\,00$
  $001\,0\,11$

② $(10101101)_2$

* 1st comp: 01010010

$+1$

01010011

* $1\,0\,1\,0\,11\,0\,1$
  $0\,1\,0\,1\,00\,1\,1$

Scanned with CamScanner

* Subtraction with Complements :-

[1] 2's Complement :

$M - N = M + (-N)$

- add M to the 2's complement of N.
- $M \geq N \rightarrow$ Sum will produce carry (discard carry)
- $M < N \rightarrow$ No carry $\rightarrow M - N = -(2\text{'s complement})$ (sum)

Ex:- Preform the following subtraction using 2's complement :-

① $(1011)_2 - (0110)_2$ (11 − 6 = 5) $= (1011)_2 + 2\text{'s}(0110)$

$= 1011 + 1010$

$= 0101$ 5

1011
1010 +
[1] 0101
discard

② $(1101)_2 - (0100)_2$ (13 − 4 = 9) $= (1101)_2 + 2\text{'s}(0100)_2$

$= 1101 + 1100$

$= 1001$ 9

1101
1100 +
[1] 1001
discard

③ $(0100)_2 - (1101)_2$ (4 − 13 = −9) $= (0100)_2 + 2\text{'s}(1101)$

$= 0100 + 0011$

$= -2\text{'s}(0111)$

$= -(1001)$ −9

0100
0011 +
0111

A − B
- A > B (carry) ✗
- A < B −2's (sum)

Scanned with CamScanner

1) 1's complement :-

$M - N = M + (-N)$

- add M to the 1's complement of N
- $M \geq N \rightarrow$ Carry (add carry to sum)
- $M < N \rightarrow$ No carry (Answer = - 1's comp (sum))

Ex :- Perform the following subtraction using 1's complement :-

① $1011 - 0110 = 1011 + 1's(0110)$ (11 − 6 = 5)

$= 1011 + 1001$

$= 0101$ 5

1 0 1 1
1 0 0 1 +
1 0 1 0 0
1 +
0 1 0 1 5

② $1101 - 0100 = 1101 + 1's(0100)$ (13 − 4 = 9)

$= 1101 + 1011$

$= 1001$ 9

1 1 0 1
1 0 1 1 +
1 1 0 0 0
1 +
1 0 0 1 9

③ $0100 - 1101 = 0100 + 1's(1101)$ (4 − 13 = −9)

$= 0100 + 0010$

$= -1's(0110)$

$= -(1001)$ −9

0 1 0 0
0 0 1 0 +
0 1 1 0

Scanned with CamScanner

Ex :- $X = 1010100$ (84), $y = 1000011$ (67)

① 2's Complement :-

$X - y = 1010100 - 1000011$ (84 − 67 = 17)

$= 1010100 + 2's(1000011)$

$= 1010100 + 0111101$

$= 0010001$ 17

```
 1 1 1 1
 1010100
 0111101 +
1 0010001
```
discard

$y - X = 1000011 - 1010100$ (67 − 84 = −17)

$= 1000011 + 2's(1010100)$

$= 1000011 + 0101100$

$= -2's(1101111)$

$= -(0010001)$ −17

```
 1 0 0 0 0 1 1
 0 1 0 1 1 0 0 +
 1 1 0 1 1 1 1
```
No carry

② 1's complement :-

$X - y = 1010100 - 1000011$ (84 − 67 = 17)

$= 1010100 + 1's(1000011)$

$= 1010100 + 0111100$

$= 0010001$ 17

```
 1 1 1
 1 0 1 0 1 0 0
 0 1 1 1 1 0 0 +
1 0 0 1 0 0 0 0
  ↳ 1 +
  0 0 1 0 0 0 1
```

$y - X = 1000011 - 1010100$ (67 − 84 = −17)

$= 1000011 + 1's(1010100)$

$= 1000011 + 0101011$

$= -1's(1101110)$

$= -(0010001)$ −17

```
   1
 1 0 0 0 0 1 1
 0 1 0 1 0 1 1 +
 1 1 0 1 1 1 0
```
No carry

Scanned with CamScanner

* Binary logic :-

- Basic (primitive) logic operation :-

1. AND : this operation is represented by a dot or the absence of an operator.

$Z = X \text{ AND } y$

$Z = X \cdot y$

$Z = Xy$

Def :- Truth Table (TT) :-

A table of all the possible combinations of the input variables, the output of these combinations

** entries of the truth table = $2^{\#\text{inputs}}$

2 inputs → #entries = $2^2 = 4$

3 inputs → #entries = $2^3 = 8$

Truth Table (AND) :- $Z = Xy$

X . y	Z	
0 0	0	0 → False
0 1	0	1 → True
1 0	0	
1 1	1	

inputs = 2 → #entries = $2^2 = 4$

- AND. For the output to be true all inputs must be true.

Scanned with CamScanner

[2] OR: The operator is represented by (+)

$Z = X \text{ OR } y$

$Z = X + y$

- For the output to be true one of the inputs must be true.

X + y	Z
0 0	0
0 1	1
1 0	1
1 1	1

[3] NOT: This operation is represented by a prime or over bar

$Z = NOT(x)$

$Z = X'$ #entries (IT) = $2^1 = 2$

$Z = \bar{X}$

X	X'
0	1
1	0

Scanned with CamScanner

* primitive logic Gates :-

- logic gates are electronic circuits that operate one or more inputs signals to produce an output signals.
- Symbols for primitive digit alogic gates.

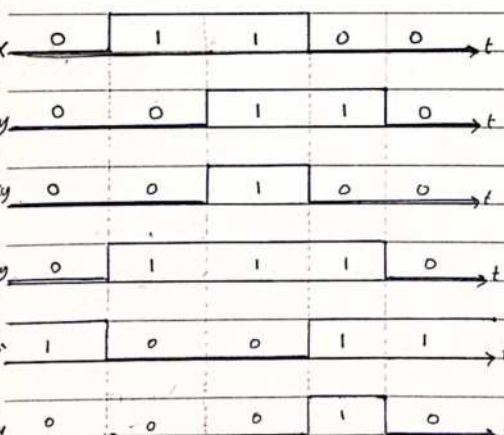


2-inputs AND gate 2-inputs OR gate input-NOT gate
Inverter

* Gates with multipleinputs :-



3-inputs AND gate 4-input OR gate



For the inputs signals x, y
Draw the output signal :-

① x.y
② x+y
③ x`
④ x`.y

Scanned with CamScanner

* Signed Numbers :-

① 1's complement

② 2's complement

③ Signed Magnitude → +3, -3 ; + → 0, - → 1 ; sing bit (MSB)

* Represent (-9) with 8-bits :- (0000 1001)

① 1's comp → -9 = 1's (9)

= 1's (0000 1001)

= 1111 0110 $\xrightarrow{1's}$ 0000 1001 -9 ✓

② 2's comp → 2's = 1111 0111 $\xrightarrow{2's}$ 0000 1001 -9 ✓

③ Singed mag → 1000 1001 -9 ✓ (9) ; recall: for signed mag + → 0, - → 1

Ex :- ① 2's 1 0 1 0 0 1 → (64 32 16 8 4 2 1) 0 0 1 0 1 1 1 = -23

② 1's 1 1 0 1 0 0 1 → 0 0 1 0 1 1 0 = -22

③ Singd mag 1 1 0 1 0 0 1 → 1 1 0 1 0 0 1 = -41 (1 = -, 1 0 0 1 = 41)

Scanned with CamScanner

* Boolean Algebra (simplification)

- Inputs (Boolean Variables $\leftarrow$ 0, 1), (Basic operations (+, ·, ')

* Boolean Algebra Basics :-

① Identity Element :-

- Identity Element for OR (+) → "0" $\Longrightarrow$ $X+0=X$

  $0+0=0$

  $1+0=1$

- Identity Element for AND (·) → "1"

  $\Downarrow$

  $X \cdot 1 = X$

  $0 \cdot 1 = 0$

  $1 \cdot 1 = 1$

② Commutative Law :-

- $X+y = y+X$

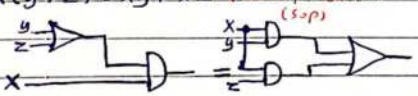


- $X \cdot y = y \cdot X$



③ Distributive Law :-

- $X(y+z) = Xy + Xz$ (sum of products) (SOP)



- $X+yz = (X+y) \cdot (X+z)$ (product of sums) (POS)

Scanned with CamScanner

④ Complement :-

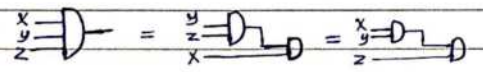
$(X')' = X$

X	X'	(X')'
0	1	0
1	0	1

⑤ Associative Law :-

$X+y+z = X+(y+z) = (X+y)+z$

$Xyz = X(yz) = (Xy)z$



Notes :- $X+1 = 1$ $X+X = X$ $X.X = X$

$X.0 = 0$ $0+0 = 0$ $0.0 = 0$

$X+X = X$ $1+1 = 1$ $1.1 = 1$

$X.X = X$

- $Xy + Xyz + Xy$ ($Xy + Xy = Xy$)
- $Xy + Xyz$
- $Xy(1+z)'$
- $Xy.1$
- Xy

Scanned with CamScanner

Prove :-

① $Xy + Xy' = X$

$X(y+y')^1$

$X \cdot 1$

X ✓

② $X + Xy = X$

$X(1+y)^1$

$X \cdot 1$

X ✓

③ $(X+y')y = Xy$

$Xy + yy'^0$

$Xy + 0$

Xy ✓

Simplify :-

① $(X+y) \cdot (X+y')$

$XX + Xy' + yX + yy'^0$

$X + Xy' + Xy$

$X(1+y'+y)^1$

$X \cdot 1$

X

② $X(X+y)$

$X \cdot X + Xy$

$X + Xy$

$X(1+y)^1$

$X \cdot 1$

X

③ $X \cdot y' + y$

$(X+y) \cdot (y'+y)^1$

$(X+y) \cdot 1$

$X+y$

Scanned with CamScanner

* Demorgan's Theory :-

- used to find the complement of a Boolean Function
- $F \Rightarrow F'$ (Apply Demorgan's)

Replace $\cdot \rightarrow +$

$+ \rightarrow \cdot$

$x \rightarrow x'$

$x' \rightarrow x$

* Apply Demorgan :-

Prove by T.T

$F = x.y \Rightarrow F' = x' + y'$

x y	x · y	(x.y)'	x'	y'	x'+y'
0 0	0	1	1	1	1
0 1	0	1	1	0	1
1 0	0	1	0	1	1
1 1	1	0	0	0	0

↳ $x'+y'$ ✓

* Boolean Function :-

- Boolean Function can be represented as :-

① Algebric expression :-

$F_1 = x + y'z$

$F_2 = x'z + xy'$

② Truth Table.

③ Gate Implementation (Schematic diagram)

Scanned with CamScanner

① $F_1 = X + y'z$ 3 inputs ∴ combination $= 2^3 = 8$ [000 → 111]

$F_1 = 1$, when $X = 1$ OR $y'z = 1$
(01)

X	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

② $F_2 = X'z + Xy'$ 3 inputs ∴ $2^3 = 8$ [000 → 111]

$F_2 = 1$ when, $X'z = 1$ OR $Xy' = 1$
(01) (10)

X	y	z	F_2
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Scanned with CamScanner

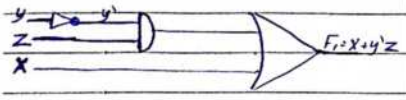
③ $F_3 = wx'z + w'x'yz + xy'z$ 4 inputs Combin = $2^4 = 16$

$F_3 = 1$ when, $wx'z = 1$ OR $w'x'yz = 1$ OR $xy'z = 1$

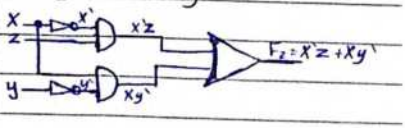
W	X	y	z	F_3
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

* Gate Implementation :-

① $F_1 = X + y'z$



② $F_2 = X'z + Xy'$



Scanned with CamScanner

Ex :- Simplify the following :-

① $F = AB + A'C + BC$

$= AB + A'C + BC(A + A')$

$= AB + A'C + ABC + A'BC$

$= AB(1 + C) + A'C(1 + B)$

$= AB + A'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

② $F = C + (BC)'$

$= C + B' + C'$

$= 1 + B'$

$= 1$

Recall: (Demorgan)

$+ \rightarrow \cdot$

$\cdot \rightarrow +$

$X \rightarrow X'$

$X' \rightarrow X$

③ $F = xyz' + x'yz' + xy'z' + x'y'z' + z$

$= yz'(x + x') + y'z'(x + x') + z$

$= yz' + y'z' + z$

$= z'(y + y') + z$

$= z' + z$

$= 1$

Ex :- Find the complement ← Demorgan

① $F_1 = x'yz' + x'y'z$

$= (x'yz' + x'y'z)'$

$= (x'yz')' \cdot (x'y'z)'$

$= (x + y' + z) \cdot (x + y + z')$

② $F_2 = x(y'z' + yz)$

$= [x(y'z' + yz)]'$

$= x' + (y'z' + yz)'$

$= x' + (y'z')' \cdot (yz)'$

$= x' + (y + z) \cdot (y' + z')$

Scanned with CamScanner

* Dual of a Function :-

$+ \rightarrow \cdot$

$\cdot \rightarrow +$

Ex:- Find the Dual :-

① $F_1 = x'yz' + x'y'z$

$F_{1D} = (x'yz')_D \cdot (x'y'z)_D$

$= (x'+y+z')_D \cdot (x'+y'+z)_D \xrightarrow[\substack{x \rightarrow x' \\ x' \rightarrow x}]{F_1'} F_1' = (x+y'+z) \cdot (x+y+z')$

② $F_2 = x(y'z' + yz)$

$F_{2D} = x + (y'z' + yz)_D$

$= x + (y'z')_D \cdot (yz)_D$

$= x + (y'+z') \cdot (y+z) \xrightarrow{F_2'} F_2' = x' + (y'+z) \cdot (y+z')$

* Canonical, Standard forms :-

* Minterms, Maxterms :-

- A binary variable in a boolean function in the original form (x) or complemented form (x').

- Consider two binary variables x, y $\Rightarrow$ 4 possible combinations.

(x: x, x' ; y: y, y')

$x'y'$
$x'y$
xy'
xy

} these combinations are called minterms

Scanned with CamScanner

x y z	and Minterms	Minterms designation	OR Maxterms	Maxterms designation
0 0 0	$x'y'z'$	m_0 000	$x+y+z$	M_0
0 0 1	$x'y'z$	m_1 001	$x+y+z'$	M_1
0 1 0	$x'yz'$	m_2 010	$x+y'+z$	M_2
0 1 1	$x'yz$	m_3 011	$x+y'+z'$	M_3
1 0 0	$xy'z'$	m_4 100	$x'+y+z$	M_4
1 0 1	$xy'z$	m_5 101	$x'+y+z'$	M_5
1 1 0	xyz'	m_6 110	$x'+y'+z$	M_6
1 1 1	xyz	m_7 111	$x'+y'+z'$	M_7

Note :-

each maxterm in the complement of the corresponding minterm.

$000 \rightarrow \underset{x'y'z'}{m_0} \rightarrow \underset{x+y+z}{M_0}$

$110 \rightarrow \underset{xyz'}{m_6} \rightarrow \underset{x'+y'+z}{M_6}$

Ex :- Represent the functions in canonical forms :-

x y z	F_1	F_2
0 0 0	0	0
0 0 1	(1)	0
0 1 0	0	0
0 1 1	0	1
1 0 0	(1)	0
1 0 1	0	1
1 1 0	0	1
1 1 1	(1)	1

* Sum of minterms (1's)

$F_1 = m_1 + m_4 + m_7$

$= \Sigma(1,4,7) = \Pi(0,2,3,5,6)$

* Product of Maxterms (0's)

$F_1 = M_0 + M_2 + M_3 + M_5 + M_6$

$= \Pi(0,2,3,5,6)$

Scanned with CamScanner

* A Boolean function can be expressed as :-

1. Sum of minterms
2. product of maxterms

] canonical forms

← From last example :-

$F_2 = \Sigma(3,5,6,7) = \Pi(0,1,2,4)$

Ex :- Represent $F = ABC + A'B'C + BC'$ in canonical forms :-

$A=1, B=1, C=1$ $A=0, B=0, C=1$ $B=1, C=0$

	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

$F = \Sigma(1,2,6,7)$

$= \Pi(0,3,4,5)$

Scanned with CamScanner

* Standard , Canonical Forms :-

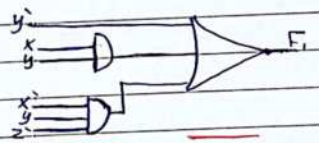
[1] Canonical Forms :-
- Sum of minterms (1's) and-OR
- product of maxterms (0's) OR-and

[2] Standard Forms :-
- Sum of products (sop) (1's) and-OR
- productof sum (pos) (0's) OR-and

2- level gate Implementation

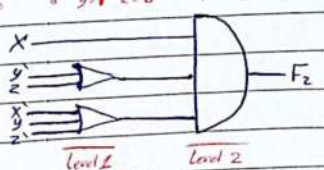
Ex :-

① $F_1 = y' + xy + x'yz'$ (sop) (1's)



x	y	z	F_1
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

② $F_2 = x(y'+z).(x'+y+z')$ (pos) (0's)



x	y	z	F_2	F_2'	F_2
0	0	0	0	1	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	1	0
1	0	0	1	0	1
1	0	1	0	1	0
1	1	0	0	1	0
1	1	1	1	0	1

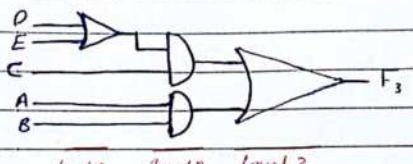
0 → x

1 → x'

Scanned with CamScanner

③ $F_3 = AB + C(D+E)$ [non standard form]

$= AB + CD + CE$



Ex:- Convert $F = A + B'C$ to sum of minterms :-

A B C

① Truth table :-

A	B	C	F	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	2
0	1	1	0	3
1	0	0	1	4
1	0	1	1	5
1	1	0	1	6
1	1	1	1	7

$F = \Sigma(1,4,5,6,7)$

② Boolean Algebra :- $(x+x')$

$F = A + B'C$

$= A + B'C(A+A')$

$= A + AB'C + A'B'C$

$= A(B+B') + AB'C + A'B'C$

$= AB + AB' + m_5 + m_1$

$= AB(C+C') + AB'(C+C') + m_5 + m_1$

$= ABC + ABC' + AB'C + AB'C' + m_5 + m_1$

$= m_7 + m_6 + m_5 + m_4 + m_5 + m_1$

$= \Sigma(1,4,5,6,7)$

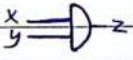
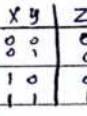
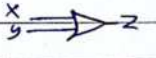
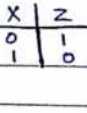
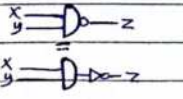
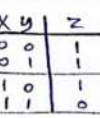
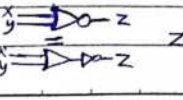
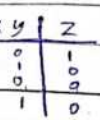
Scanned with CamScanner

Ex: Express $F = xy + x'z$ as pos ← 0's

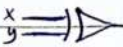
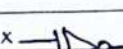
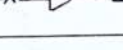
x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$F = \Pi(0,2,4,5)$

* Digital Logic Gates :-

Name	Symbol	Function	T.T
① AND		$z = x.y$ $z = xy$	
② OR		$z = x+y$	
③ Inverter (Not)		$z = x'$ $z = \bar{x}$	
④ NAND Not AND		$z = (x.y)'$	
⑤ NOR Not OR		$z = (x+y)'$	

Scanned with CamScanner

Name	Symbol	Function	T.T
⑥ XOR (Exclusive OR)		$Z = X \oplus Y$ $Z = X'y + Xy'$	X y | Z 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
⑦ XNOR (Exclusive NOR)		$Z = (X \oplus y)'$ $Z = X'y' + Xy$	X y | Z 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1
⑧ Buffer		$Z = X$	X | Z 0 | 0 1 | 1

* multiple input XOR

1's odd → 1

1's even → 0

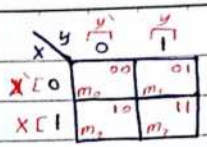
* Karnaugh map (K-map) :-

- The map method is used to find the simplest form of Boolean function.
- The K-map is a map of squares each square represents a minterm
- # Squares = # minterms = 2^n [n = # variables]

Scanned with CamScanner

[1] 2-Variable K-map :-

- # Var = 2 → # squares = $2^2 = 4$
- each square minterm



Steps :-

① Draw Map (x-section, y-section, number squares).

② Fill in the 1's from T.T.

③ Circle largest number of 1's.

→ Bubble size = 4, 2, 1

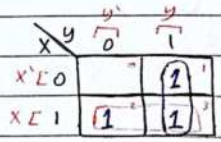
④ Write the function from the map.

Ex :- Simplify :-

$F(x,y) = x`y + xy` + xy$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

$F = \sum(1,2,3)$



$F = x + y$

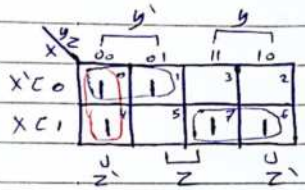
Scanned with CamScanner

[2] 3- Variable K- map :-

- # Squares $= 2^3 = 8$
- Bubble Size $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$
  $\quad\quad\quad

④ $F = \Pi(2,3,5)$

$= \Sigma(0,1,4,6,7)$



$F = x'y' + xy + y'z'$

$= x'y' + xy + xz'$

⑤ $F = A'C + A'B + AB'C + BC$ ← a. SOP, b. POS

A B C	F
0 0 0	0
0 0 1	1
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	1

$F = \Sigma(1,2,3,5,7)$

$= \Pi(0,4,6)$

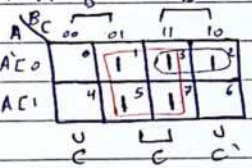
Notes:

Bubble Size	Term
8 →	0
4 →	1
2 →	2
1 →	3

F' (SOP)

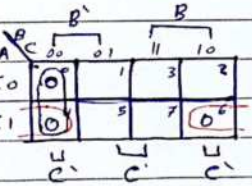
$F_{pos} = (F'_{sop})'$

ⓐ SOP :-



$F = C + A'B$

ⓑ POS :-



① $F'_{sop} = B'C' + AC'$

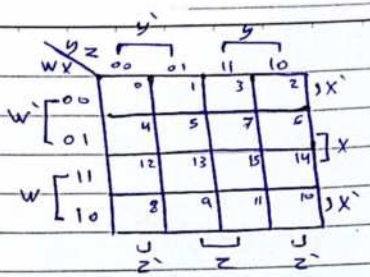
② $F_{pos} = (F'_{sop})' = (B'C' + AC')'$

③ $F_{pos} = (B+C) \cdot (A'+C)$

Scanned with CamScanner

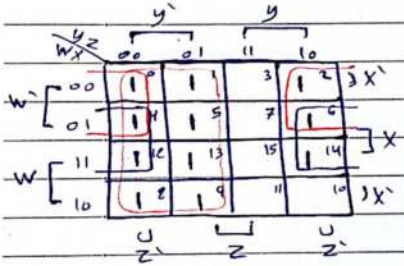
[3] 4-Variable K-map :-

- * Squares = $2^4 = 16$
- Bubble size $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$
  * terms 0 1 2 3 4



Ex :- Simplify :-

① $F(w,x,y,z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$

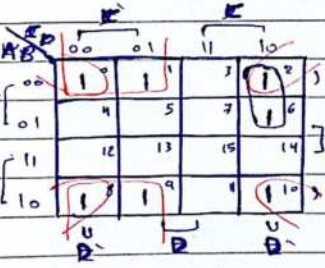


$F = y' + w'z' + xz'$

② $F = A'B'C' + B'CD' + A'BCD' + AB'C'$

A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

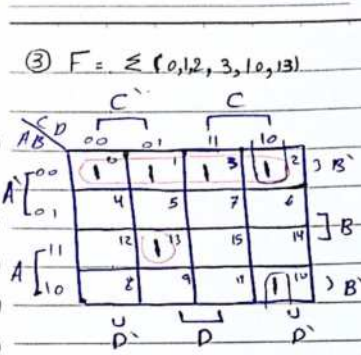
$F = \Sigma(0,1,2,6,8,9,10)$



$F = B'D' + B'C' + A'CD'$

Scanned with CamScanner

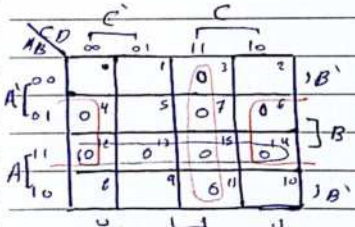
③ $F = \Sigma(0,1,2,3,10,13)$



$F = A`B` + B`CD` + ABC`D$

④ $F(A,B,C,D) = \Sigma(0,1,2,5,6,9,10)$ in pos

$= \Pi(3,4,6,7,11,12,13,14,15)$



$F`_{sop} = CD + BD` + AB$

$F_{pos} = (F`_{sop})` = (CD + BD` + AB)`$

$F_{pos} = (C`+D`).(B`+D).(A`+B`)$

Scanned with CamScanner

* Don't care conditions :-

- In some Boolean function we don't care what is the value of the function for some minterms.

Ex : Simplify :-

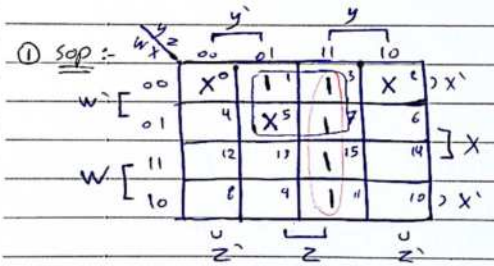
$F(w,x,y,z) = \Sigma(1,3,7,11,15) = \Pi(4,6,8,9,10,12,13,14)$

$d(w,x,y,z) = \Sigma(0,2,5)$

① SOP

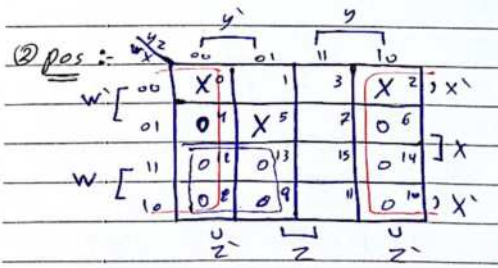
② POS

① SOP :-



$F_{sop} = yz + w'z$

② POS :-



① $F_{sop}' = z' + wy'$

② $F_{pos} = (F_{sop}')' = (z' + wy')'$

③ $F_{pos} = z \cdot (w' + y)$

Scanned with CamScanner

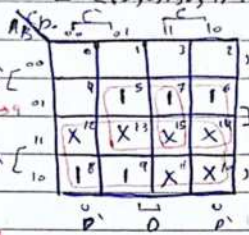
Ex: Design a Circuit that convert from BCD to Excess-3 :

BCD A B C D	Ex-3 E F G H
0 0 0 0	0 0 1 1
0 0 0 1	0 1 0 0
0 0 1 0	0 1 0 1
0 0 1 1	0 1 1 0
0 1 0 0	0 1 1 1
0 1 0 1	1 0 0 0
0 1 1 0	1 0 0 1
0 1 1 1	1 0 1 0
1 0 0 0	1 0 1 1
1 0 0 1	1 1 0 0
1 0 1 0	X X X X
1 0 1 1	X X X X
1 1 0 0	X X X X
1 1 0 1	X X X X
1 1 1 0	X X X X
1 1 1 1	X X X X

Not BCD

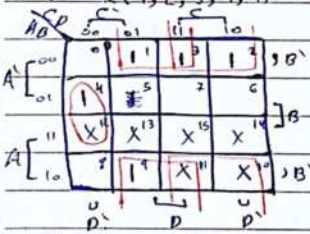
10 → 15

$E = \Sigma(5,6,7,8,9)$. $E(x) = \Sigma(10,11,12,13,14,15)$



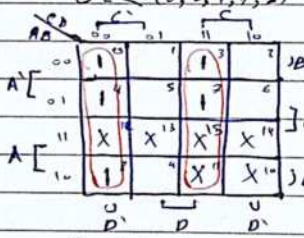
$E = A + BD + BC$

$F = \Sigma(1,2,3,4,9)$



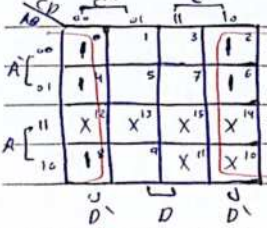
$F = B'C + B'D + BC'D'$

$G = \Sigma(0,3,4,7,8)$

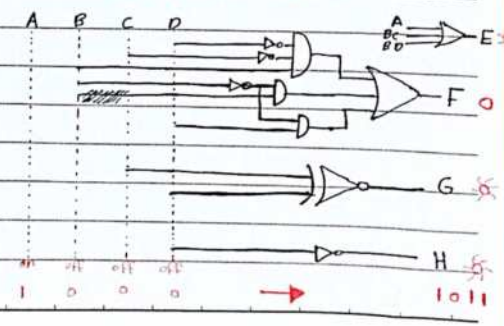


$G = C'D' + CD = (C \oplus D)'$

$H = \Sigma(0,2,4,6,8)$



$H = D'$



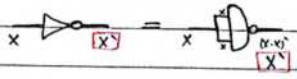
Scanned with CamScanner

* Universal Gates :-

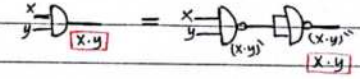
- primitive gates (and, or, not)
- From universal gates (nand / nor) I can implement primitive gates
- any circuit can be implemented using universal gates

* Nand gate implementation :-

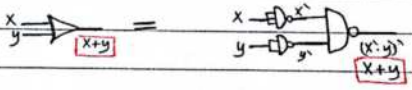
1 not



2 and

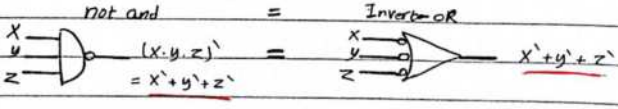


3 or



Note :-





Scanned with CamScanner

* How to Implement a function using nand gates only :-

① write the function as sop.

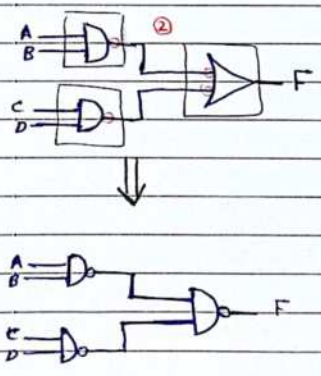
② Draw as and - OR

③ Replace and → nand , OR → invert OR

④ Return all nands to

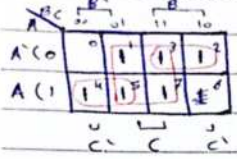
Ex :- Implement the following function using nand gates only :-

① F = AB + CD sop ✓

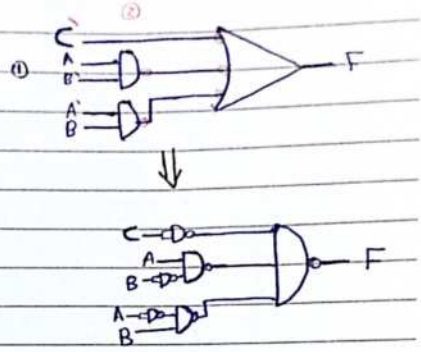


Scanned with CamScanner

② $F = \Sigma(1,2,3,4,5,7)$



$F = AB` + C + A`B$ sop ✓



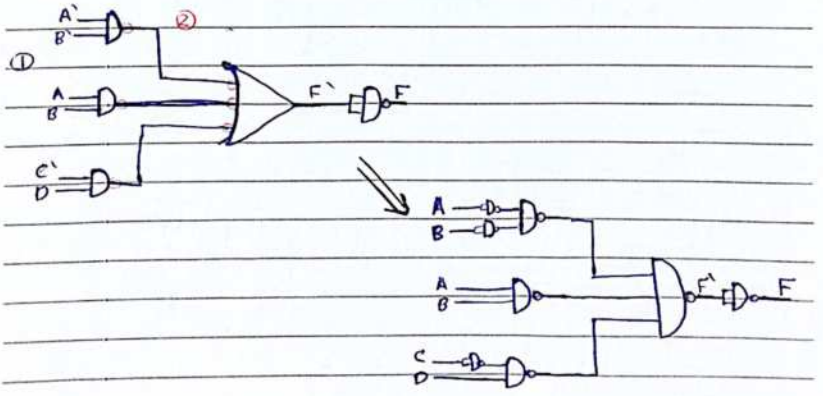
③ $F = (AB` + A`B).(C+D`)$ pos $\xrightarrow[\times]{\text{mult}}$ sop

$F` = (AB` + A`B)` + (C+D`)`$

$= (A`+B).(A+B`) + C`D$

$= A`A + A`B` + AB + BB` + C`D$

$F`_{sop} = \boxed{A`B` + AB + C`D}$ sop ✓



Scanned with CamScanner

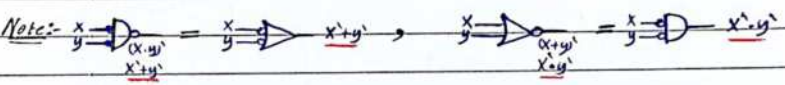
* NOR Implementation :-

① F as pos

② Draw OR/AND

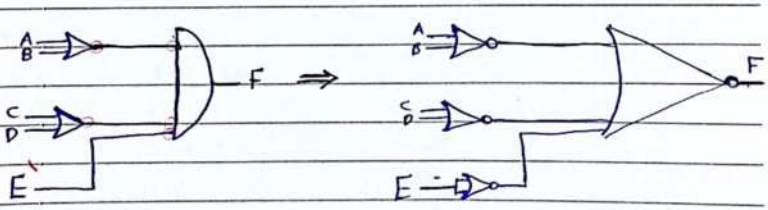
③ Replace 

④ 

Note:- 

Ex:- Implement the following functions using NOR gates only:- pos

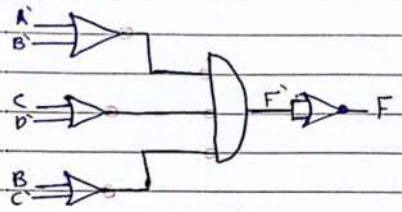
① F = (A+B)(C+D)E pos

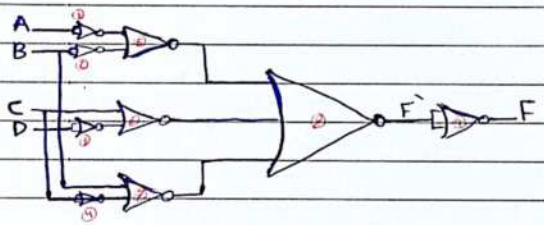


Scanned with CamScanner

② $F = AB + C'D + B'C$ sop → pos

$F' = (A'+B').(C+D).(B+C')$ pos



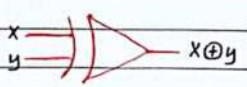


* Exclusive OR Function :-

① XOR. Function

$X \oplus y = X'y + Xy'$

x	y	x⊕y
0	0	0
0	1	1
1	0	1
1	1	0



- XOR is an odd Function → # 1's @ input odd → output = 1

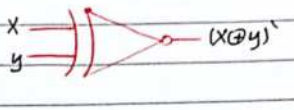
even → output = 0

Scanned with CamScanner

[2] XNOR Function :-

$(X \oplus y)' = X'y' + Xy$

X	y	$(X \oplus y)'$
0	0	1
0	1	0
1	0	0
1	1	1



- XNOR is an even function → # 1's even → output = 1
  odd → output = 0

Ex :

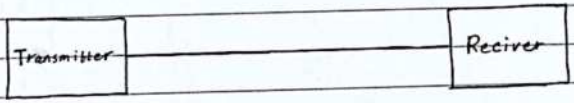


,



* Parity check :-

- used for error detection
- Two types of parity ← odd parity, even parity



Ex : what is the transmitted msg for the following using odd parity :-

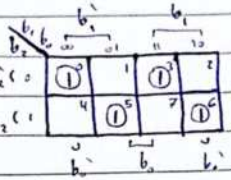
① 1101101 → 11011010 = 7
6 ✗ msg P

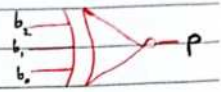
② 1111101 → 11011010
7 ✓ msg P

Scanned with CamScanner

Ex:- Design an odd parity bit generator for a 3-bit word:-

	b_2	b_1	b_0	P
#1's even	0	0	0	1
odd=1	0	0	1	0
odd=1	0	1	0	0
even=0	0	1	1	1
odd=1	1	0	0	0
even=0	1	0	1	1
even=0	1	1	0	1
odd=1	1	1	1	0





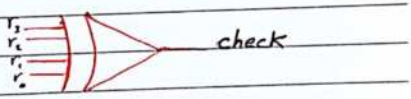
↳ XNOR Function $P = b_2'b_1'b_0' + b_2'b_1b_0 + b_2b_1'b_0 + b_2b_1b_0'$

Ex:- Design an odd parity check circuit for received 4-bit word:-

	r_3	r_2	r_1	r_0	check
#1's even	0	0	0	0	0
odd=1	0	0	0	1	1
odd=1	0	0	1	0	1
even=0	0	0	1	1	0
odd=1	0	1	0	0	1
even=0	0	1	0	1	0
even=0	0	1	1	0	0
odd=1	0	1	1	1	1
odd=1	1	0	0	0	1
even=0	1	0	0	1	0
even=0	1	0	1	0	0
odd=1	1	0	1	1	1
even=0	1	1	0	0	0
odd=1	1	1	0	1	1
odd=1	1	1	1	0	1
even=0	1	1	1	1	0

#1's odd → correct → check=1

#1's even → Incorrect → check=0



↳ XOR Function

even → 0

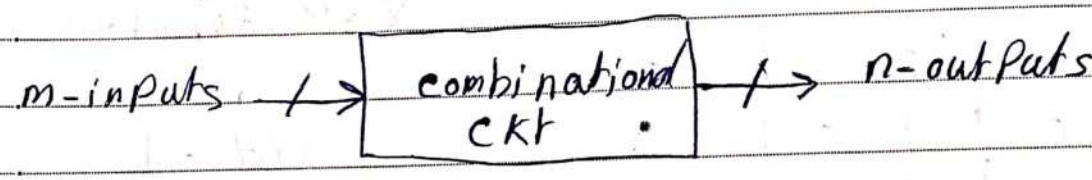
odd → 1

Scanned with CamScanner

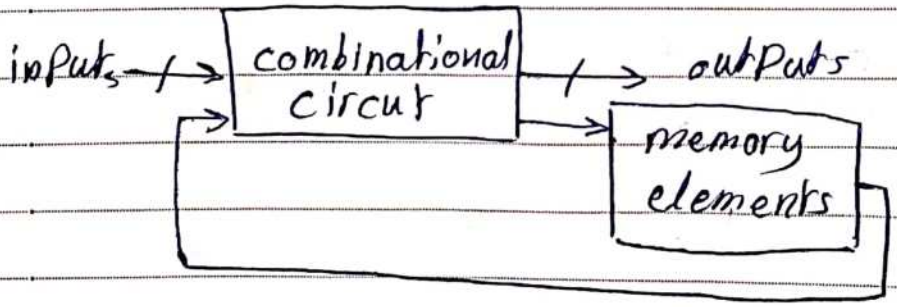
* Combinational circuits:-

→ Two types of logic circuits:-

[1] combinational circuits:-
consists of logic gates whose outputs at any time are determined from only the present inputs



[2] Sequential circuits:-
- employ memory elements in addition to logic gates
- Their outputs are a function of the inputs, the state of the storage elements.

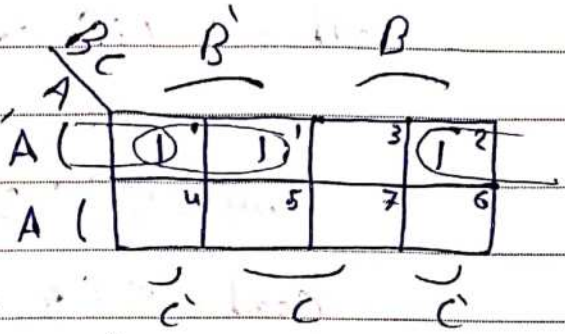


* Design Procedure:-

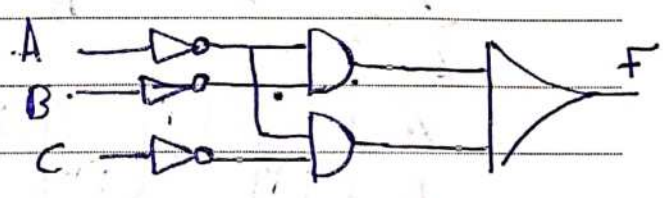
① circuit specification
② Truth table
③ K-maps
④ Boolean function
⑤ logic design (gates)

Ex: Design a combinational circuit which has a 3-bit input, a single output (F) where: F = 1, when input number is less than 3
F = 0, other wise

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0



$F = A'B' + A'C'$



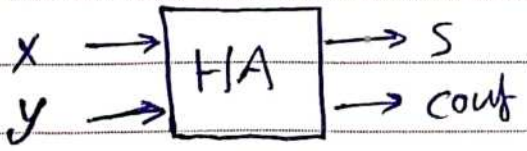
* Binary adders, subtractors :-

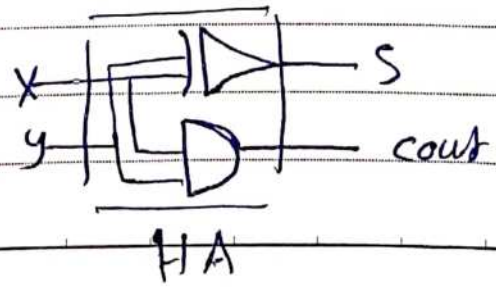
1) Half Adder (HA):
- 2 inputs (x, y)
- 2 outputs (sum(s), carry out (cout))
- Sum = x + y (binary addition)
- cout = 1 (if carry occurs)

x y	S	cout
0 0	0	0
0 1	1	0
1 0	1	0
1 1	0	1

$S = X \oplus Y$

$cout = XY$





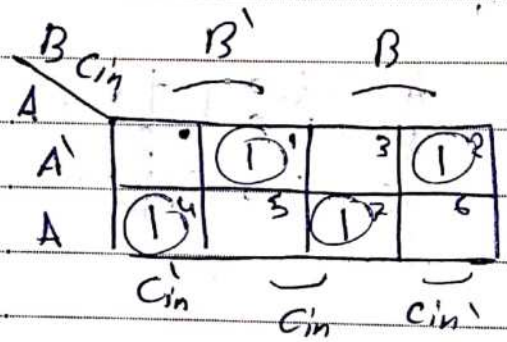
[2] Full Adder (FA) :

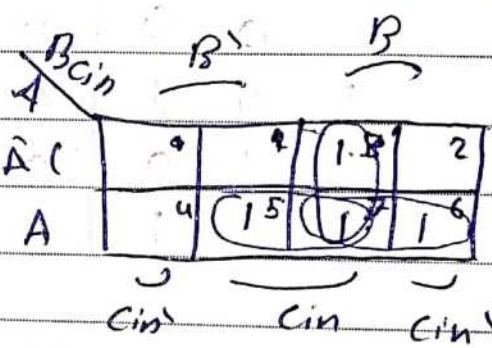
- 3 - inputs (A, B, Cin)
- 2 - outputs (sum(S), carry out (cout))
- sum = A + B + Cin
- cout = 1 (if cout occurs)

A	B	Cin	S	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$sum = \Sigma(1,2,4,7)$ ←

$cout = \Sigma(3,5,6,7)$





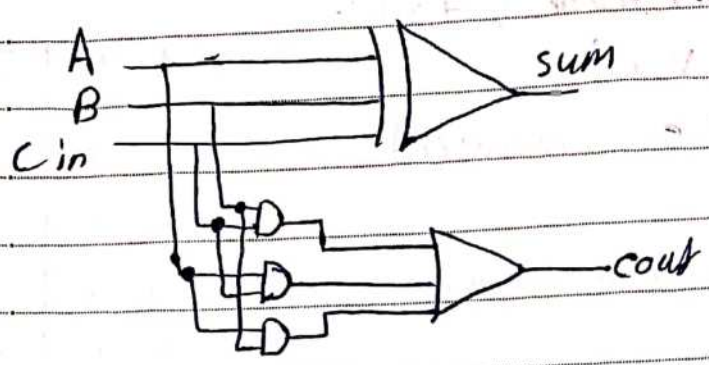
$sum = A\bar{B}\bar{C}in + \bar{A}\bar{B}\bar{C}in + A\bar{B}\bar{C}in + ABCin$

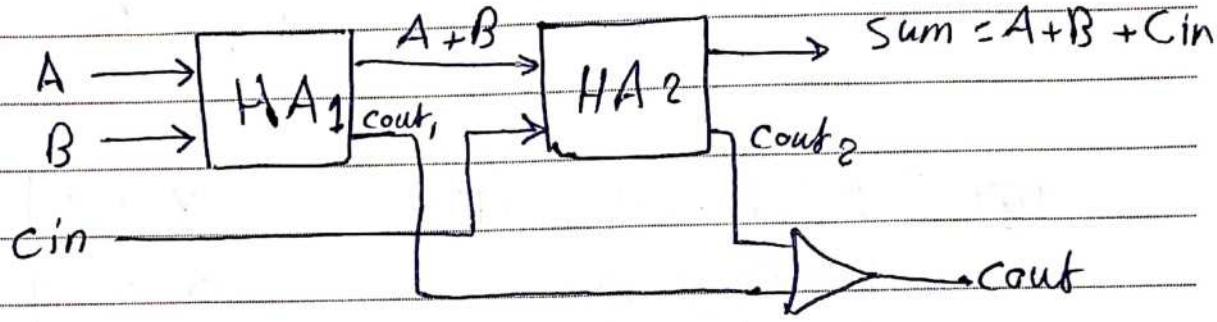
$cout = BCin + ACin + AB$

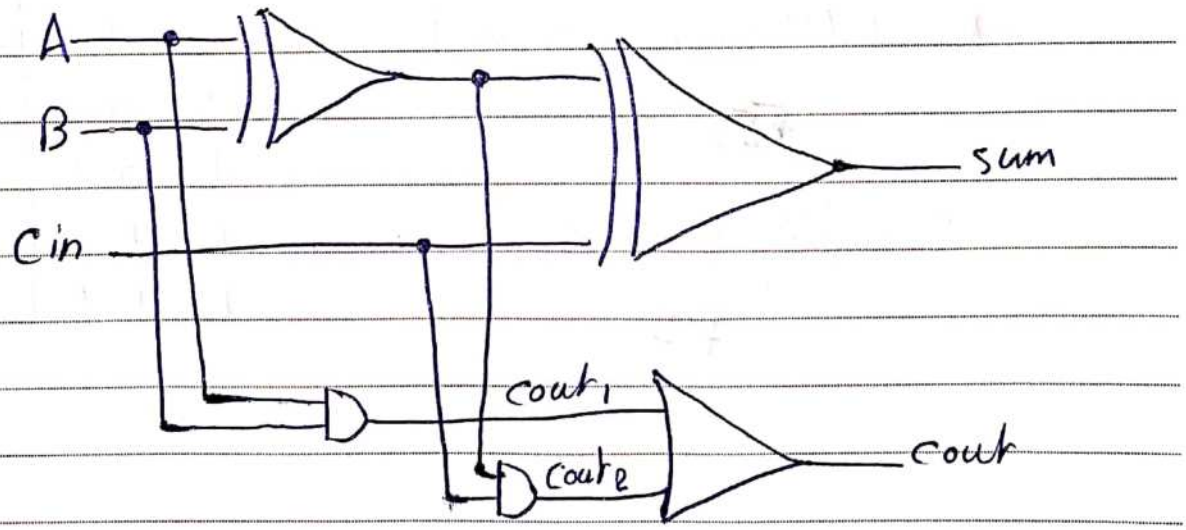
$S = (A + B) + Cin$

HA

HA_2


A
B
Cin
sum
cout

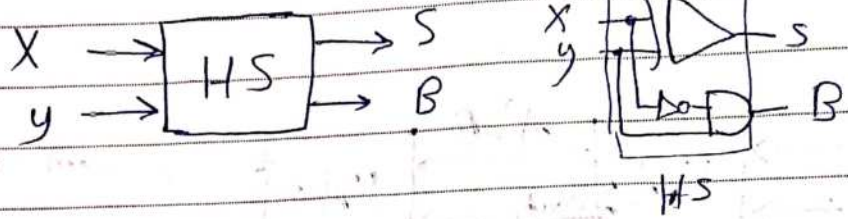

A
B
Cin
HA1
HA2
A+B
cout1
cout2
Sum = A+B+Cin
cout


A
B
Cin
sum
cout1
cout2
cout

[3] Half Subtractor (HS):-

- 2-inputs (X, y)
- 2-outputs (S, Borrow (B))
- S = X - y
- B = 1 (If borrow occurs)

X	y	S	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



[4] Full Subtractor (FS):-

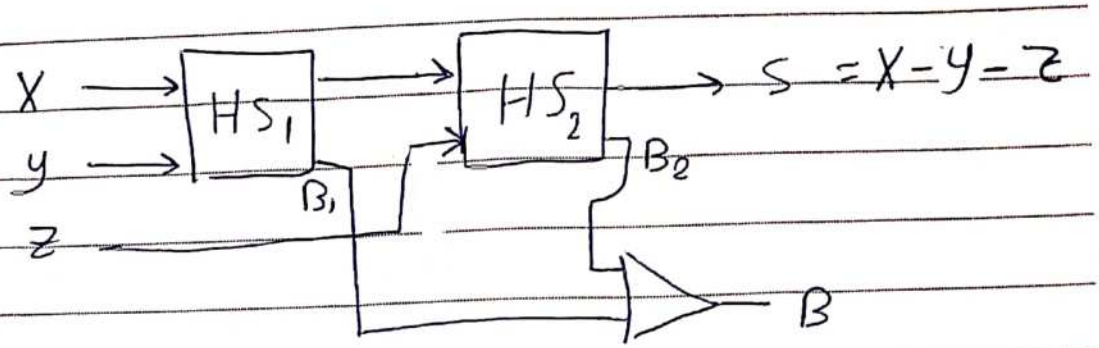
- 3-inputs (X, y, Z)
- 2-outputs (S, B)
- S = X - y - Z
- B = 1 (If borrow occurs)

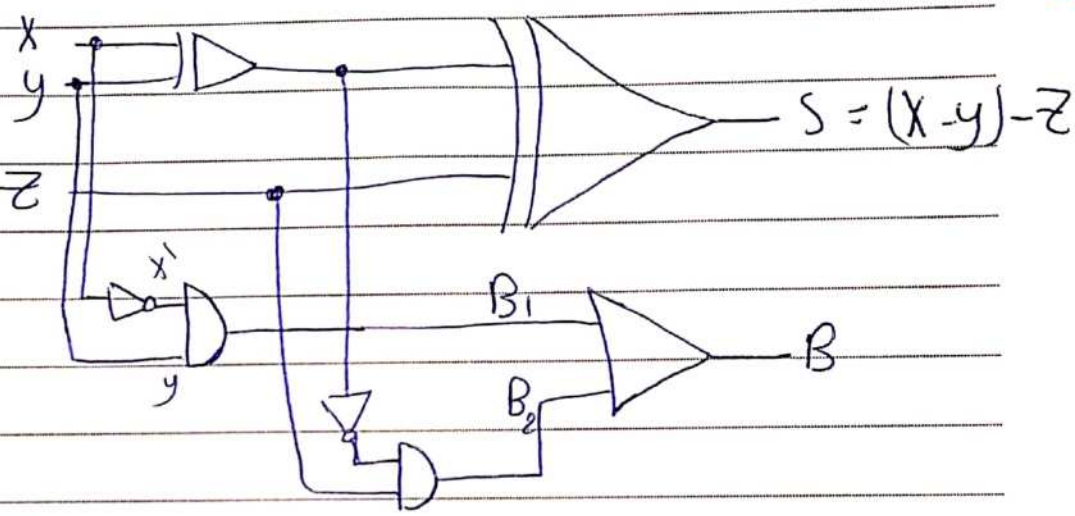
HS_1

$S = \underbrace{(X - y)}_{HS_1} - Z$

HS_2

X	y	Z	S	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1


X
y
z
HS1
HS2
B1
B2
S = X - y - z
B


X
y
z
x'
y
B1
B2
B
S = (X - y) - z

* 4-bit adder (4bit ripple adder)

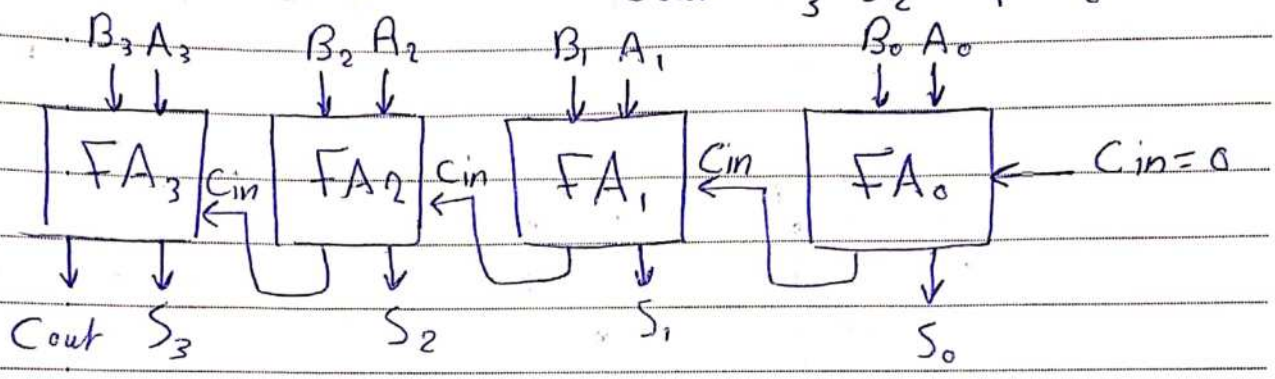
$A = A_3 A_2 A_1 A_0$

$B = B_3 B_2 B_1 B_0$

$S = A + B$

FA FA FA Cout HA

	A_3	A_2	A_1	A_0
	B_3	B_2	B_1	B_0 +
Cout	S_3	S_2	S_1	S_0



* Binary subtractor

$D = A - B$

$= A + (-B)$

$= A + 2's(B)$

$= A + 1's(B) + 1$

$A = A_3 A_2 A_1 A_0$

$B = B_3 B_2 B_1 B_0$

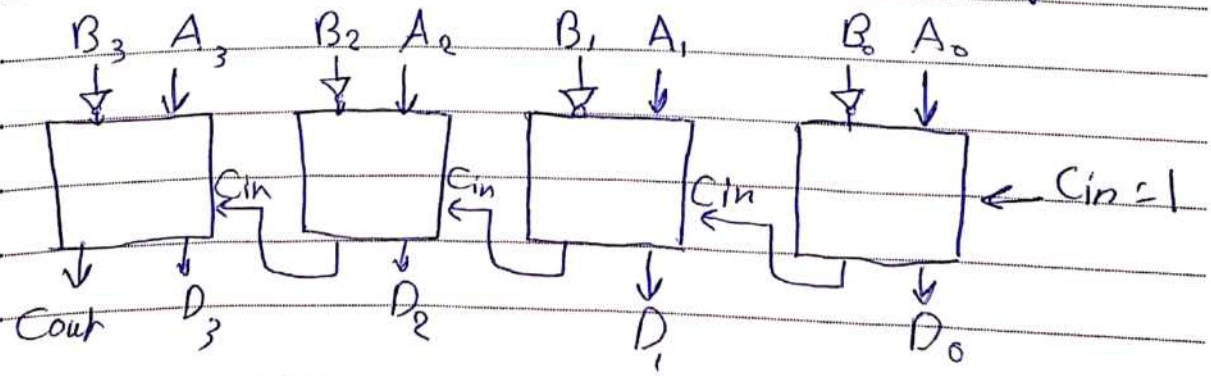
$D = A - B$

$= A_3 A_2 A_1 A_0 + \bar{B_3}\,\bar{B_2}\,\bar{B_1}\,\bar{B_0} + 1$

$A_3\ A_2\ A_1\ A_0$

$\bar{B_3}\ \bar{B_2}\ \bar{B_1}\ \bar{B_0}$ +

1 +



* 4 bit Adder saptractor :

- we will use xor

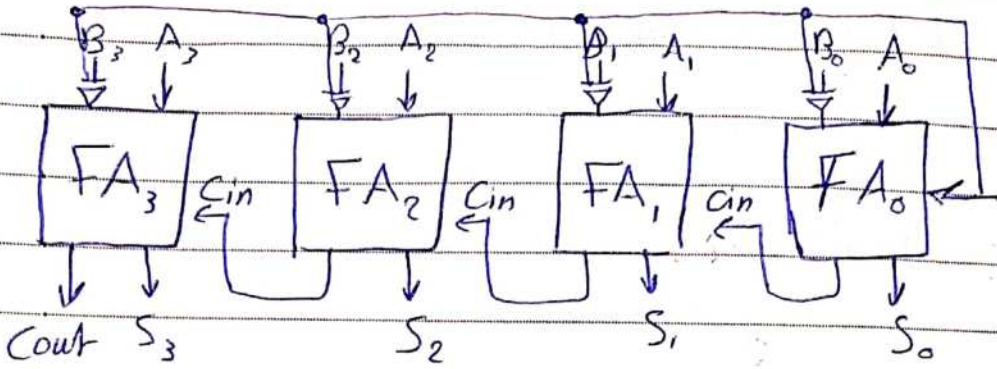
$0 \oplus X = X$

$1 \oplus X = X'$

* $A + B = A_3 A_2 A_1 A_0$
$B_3 B_2 B_1 B_0$ +

* $A - B = A_3 A_2 A_1 A_0$
$B_3 B_2 B_1 B_0$ +
1 +

$C_{in} \rightarrow = 0$ adder
$\rightarrow = 1$ subtractor



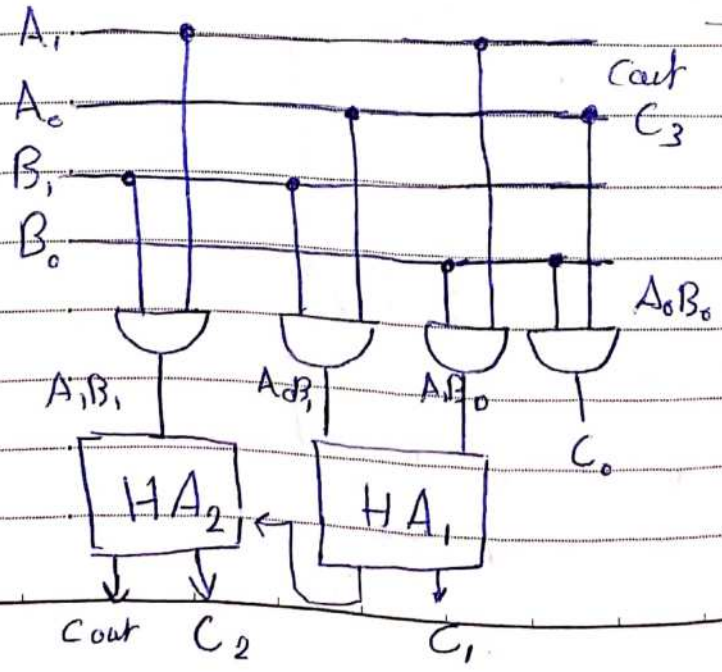
* Binary Multiplier

Ex: Design a 2-bit binary multiplier

$C = A \times B$

$A_1 A_0 \times B_1 B_0$

$A_1 A_0$
$B_1 B_0$ x
$A_1B_0 \quad A_0B_0$
$A_1B_1 \quad A_0B_1$ +
Cout C_2 C_1 C_0



* BCD Adder

- A BCD adder adds two BCD digits and produces sum in BCD

→ Carry → correct digit [+6]

→ not BCD → correct digit [+6]

↓

>9

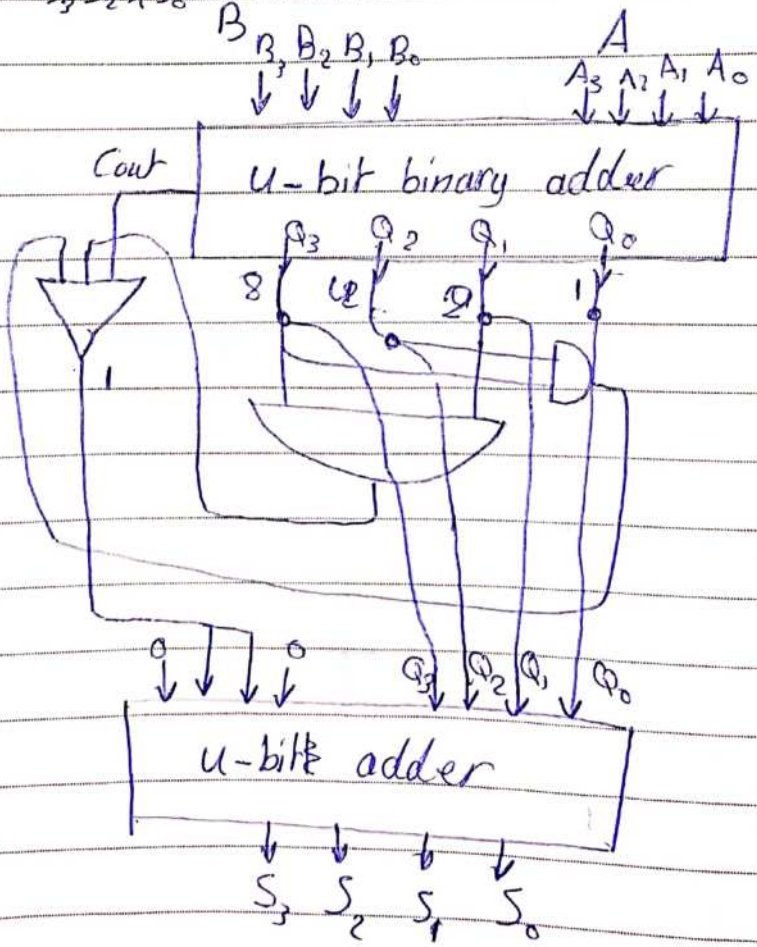
0000 ↓ 1000] BCD ≤ 9

1001

8+2 [1010, 1011]

8+4 [1100, 1101, 1110, 1111] Not BCD > 9

$S_3 S_2 S_1 S_0$



* Magnitude Comparator:

- combinational circuit that compares two numbers, A, B and determines their relative magnitude.
- Inputs: 2 numbers A, B
- outputs: 3, $A>B=1$ if $A>B$
  $A<B=1$ if $A<B$
  $A=B=1$ if $A=B$

Ex: Design a 2-bit comparator:

$A = A_1 A_0$

$B = B_1 B_0$

$(A \odot B)' = A_1 B_1 + A_0 B_0$

* If $A=B \rightarrow$ =)D- $A=B$

* If $A>B \rightarrow A_1 \bar{B_1} + A_1 B_1 A_0 \bar{B_0}$

$+ \bar{A_1} \bar{B_1} A_0 \bar{B_0}$

$A_1 = 1$, $B_1 = 0$

$A_1 = 0$, $B_1 = 0$, $A_0 = 1$, $B_0 = 0$

* If $A<B \rightarrow \bar{A_1} B_1 + A_1 B_1 \bar{A_0} B_0$

$+ \bar{A_1} \bar{B_1} \bar{A_0} B_0$

$A_1 = 0$, $B_1 = 1$

$A_1 = 0$, $B_1 = 0$, $A_0 = 0$, $B_0 = 1$

$A_1 = 1$, $B_1 = 1$, $A_0 = 0$, $B_0 = 1$

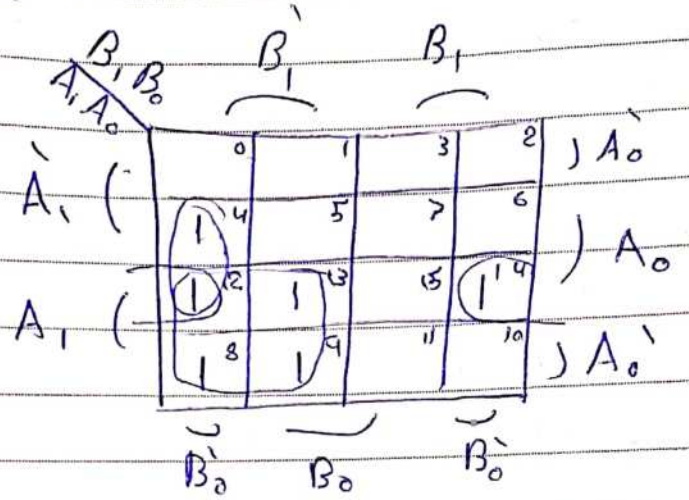
A_1	A_0	B_1	B_0	$A>B$	$A<B$	$A=B$
0	0	0	0	0	0	1
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	1
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	1	0
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	0	1

$(A>B) = \Sigma(4, 8, 9, 12, 13, 14)$

$(A<B) = \Sigma(1, 2, 3, 6, 7, 11)$

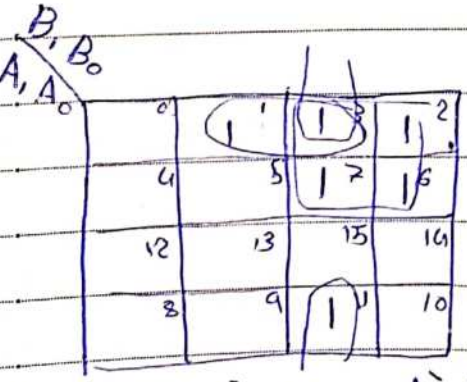
$(A=B) = \Sigma(0, 5, 10, 15)$

$(A>B) = \Sigma(4,8,9,12,13,14)$



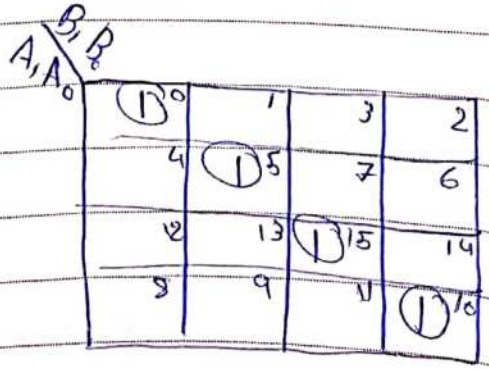
$(A>B) = A_1B_1' + A_1A_0B_0' + A_0B_1'B_0$

$(A<B)$



$(A<B) = A_1'B_1 + A_1'A_0'B_0 + A_0'B_1B_0$

$(A=B)$



$(A=B) = A_1'B_1'A_0'B_0' + A_1'B_1A_0B_0$

$+ A_1B_1A_0B_0 + A_1B_1A_0'B_0'$

* Decoders:-

- A decoders is a combinational circuit that converts binary information from (n) inputs to (2^n) out put lines

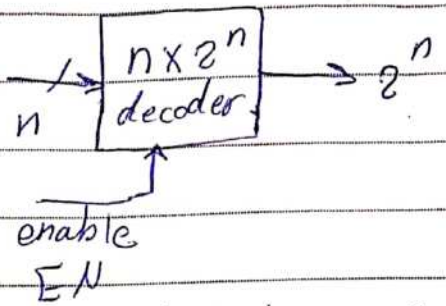
- n to 2^n decoder

2x4

3x8

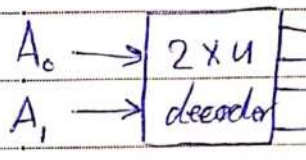
1x2

4x16



- each input of the decoder represents a minterm

Ex: Design a 2x4 decoder



A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

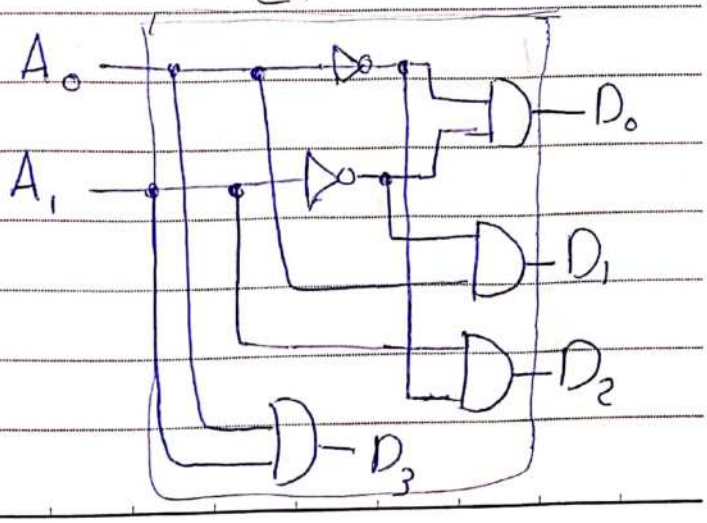
$D_0 = A_1' A_0' = m_0$ ←

$D_1 = A_1' A_0 = m_1$

$D_2 = A_1 A_0' = m_2$

$D_3 = A_1 A_0 = m_3$

2x4 Decoder



Ex: Design a 3x8 decoder Inputs $A_2 A_1 A_0$

$D_0 = m_0 \rightarrow \bar{A}_2 \bar{A}_1 \bar{A}_0$

$D_1 = m_1 \rightarrow \bar{A}_2 \bar{A}_1 A_0$

$D_2 = m_2 \rightarrow \bar{A}_2 A_1 \bar{A}_0$

$D_3 = m_3 \rightarrow \bar{A}_2 A_1 A_0$

$D_4 = m_4 \rightarrow A_2 \bar{A}_1 \bar{A}_0$

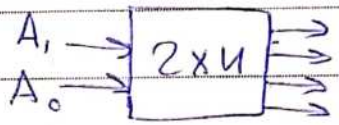
$D_5 = m_5$

$D_6 = m_6$

$D_7 = m_7$

Ex: Design a 2-to-4 Active low decoder

$A_1 A_0$	$D_0 D_1 D_2 D_3$
0 0	0 1 1 1
0 1	1 0 1 1
1 0	1 1 0 1
1 1	1 1 1 0



→ In active low decoder we used nand

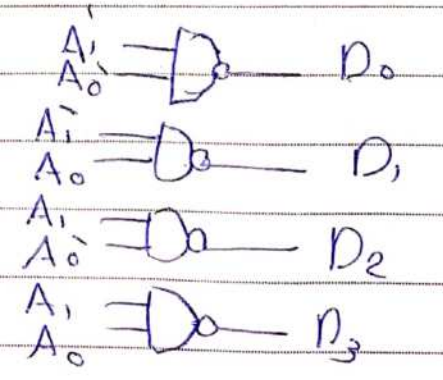
m_0

$D_0 \rightarrow \overline{m_0} \quad (\bar{A}_1 \bar{A}_0)'$

$D_1 \rightarrow \overline{m_1} \quad (\bar{A}_1 A_0)'$

$D_2 \rightarrow \overline{m_2} \quad (A_1 \bar{A}_0)'$

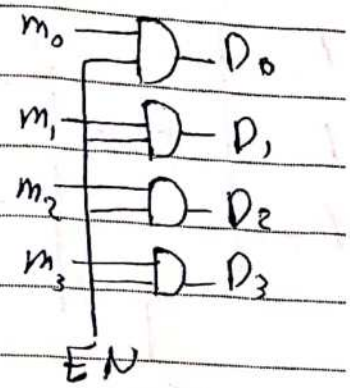
$D_3 \rightarrow \overline{m_3} \quad (A_1 A_0)'$



Ex: Design 2-to-4 Active high decoder with enable → EN = 1 decoder on
EN = 0 decoder off

$X \cdot 0 = 0$
$X \cdot 1 = X$

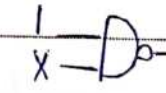
EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1



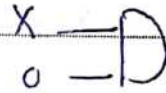
$m_0 = \grave{A}_1 \grave{A}_0 \quad m_1 = \grave{A}_1 A_0 \quad m_2 = A_1 \grave{A}_0 \quad m_3 = A_1 A_0$

Ex: Design a 2x4 Active low decoder with enable EN = 0 → on
EN = 1 → off

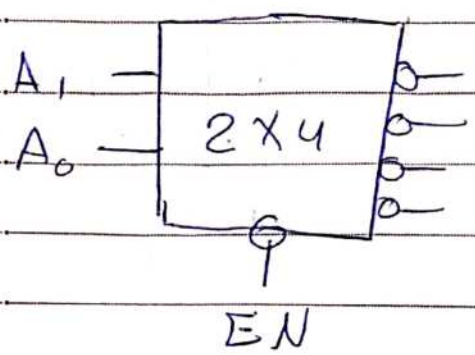
$(1 \cdot X)\grave{} = X'$

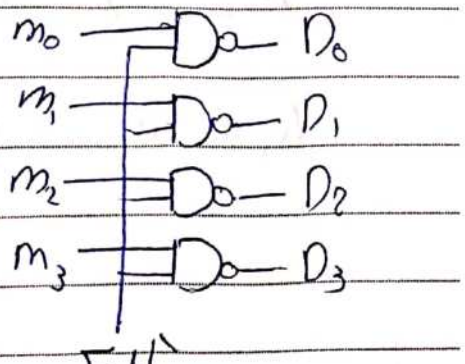


$(X \cdot 0)\grave{} = 0\grave{} = 1$

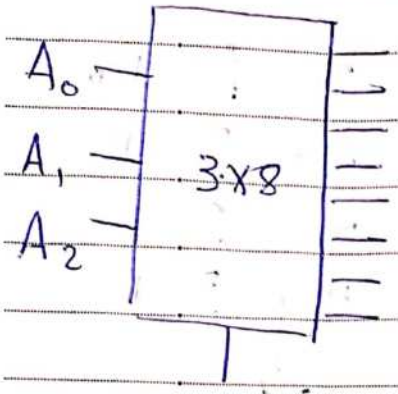


EN	A_1	A_0	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0





Ex: Active High 3 x 8 decoder with enable



EN	A_2	A_1	A_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	1

* Decoder Expansion.
- Building larger decoders using two or more smaller decoders

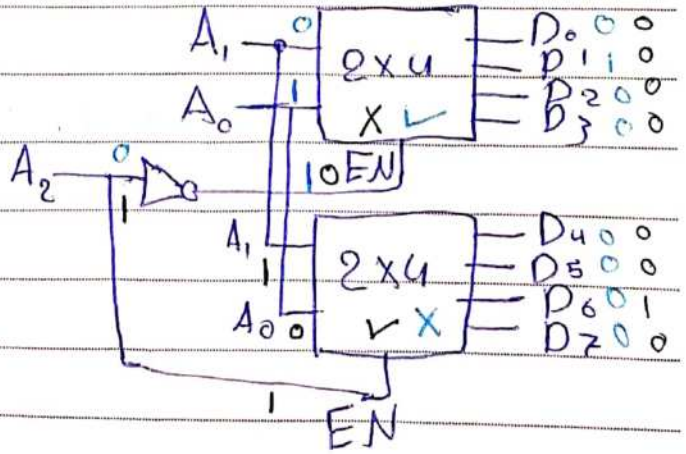
Ex: Constract a 3x8 decoder using 2x4 decoders with enable inputs.

3 x 8 I have 2 x 4

(1)

$A_2 A_1 A_0 \rightarrow 001$

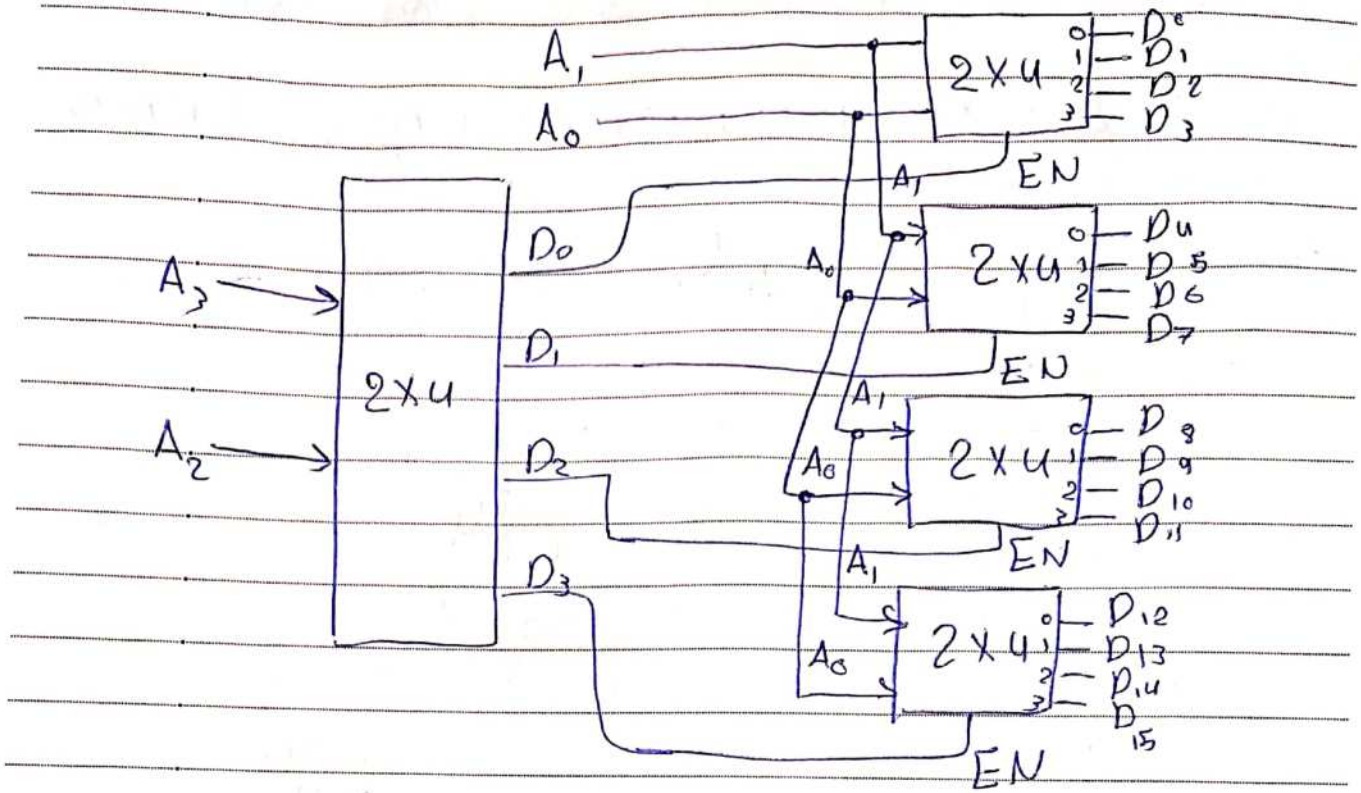
$A_2 A_1 A_0 \rightarrow 110$



Ex: Constrack a 4*16 decoder using 2x4 decoders

A_3	A_2	A_1	A_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8	D_9	D_{10}	D_{11}	D_{12}	D_{13}	D_{14}	D_{15}
EN	0	0	0	0	1	0	0												
	0	0	0	1															
	0	0	1	0															
	0	0	1	1															
	0	1	0	0				1	0	0	0								
	0	1	0	1															
	0	1	1	0															
	0	1	1	1															
	1	0	0	0								1	0	0	0				
	1	0	0	1															
	1	0	1	0															
	1	0	1	1															
	1	1	0	0												0	0	0	0
	1	1	0	1															
	1	1	1	0															
	1	1	1	1												0	0	0	1

11



$A_3A_2 = 00 \rightarrow D_0 = 1$ & on ① ② ③ off

$D_1, D_2, D_3 = 0$

$A_3A_2 = 11 \rightarrow D_3 = 1$

$D_0, D_1, D_0 = 0$

* Using Decoder to implement a Boolean function

→ Decoder output are minterms.

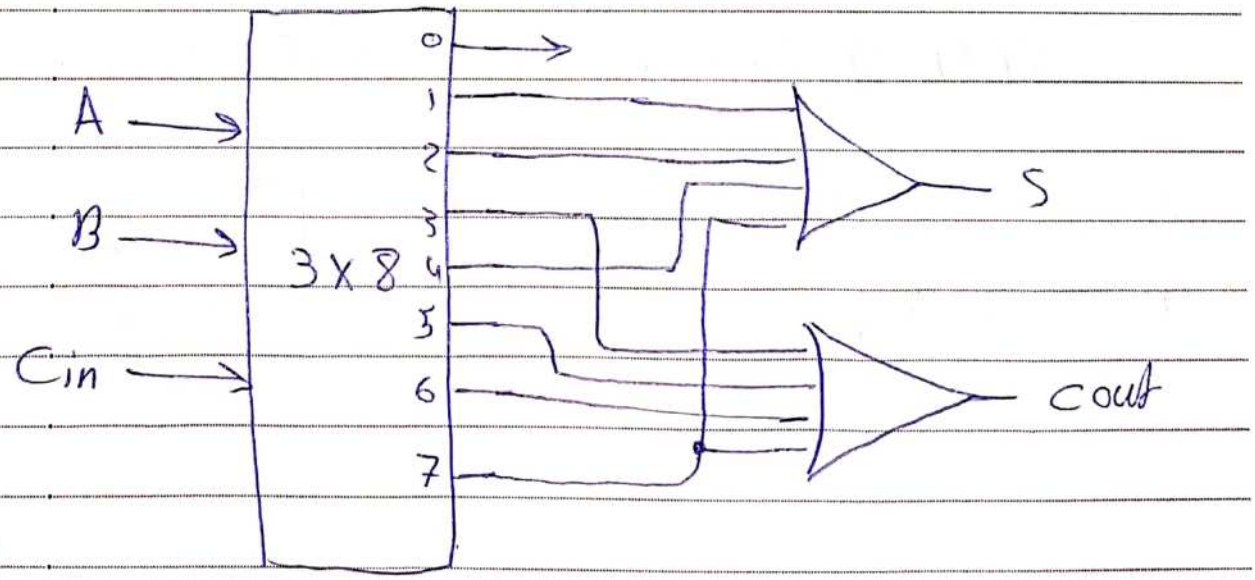
→ Any Boolean function in a sum of minterms

Ex: Implement a full adder using decoder

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$S = \Sigma(1,2,4,7)$

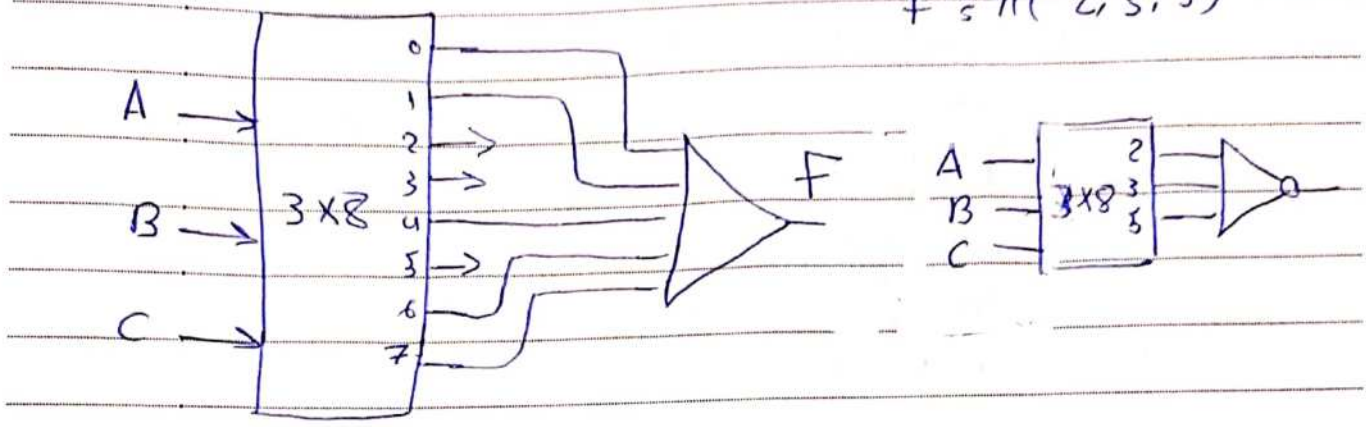
$Cout = \Sigma(3,5,6,7)$



Ex: Implement the Boolean function
$F(ABCD) = \Sigma(0,1,4,6,7)$
using 3 x 8 Decoder

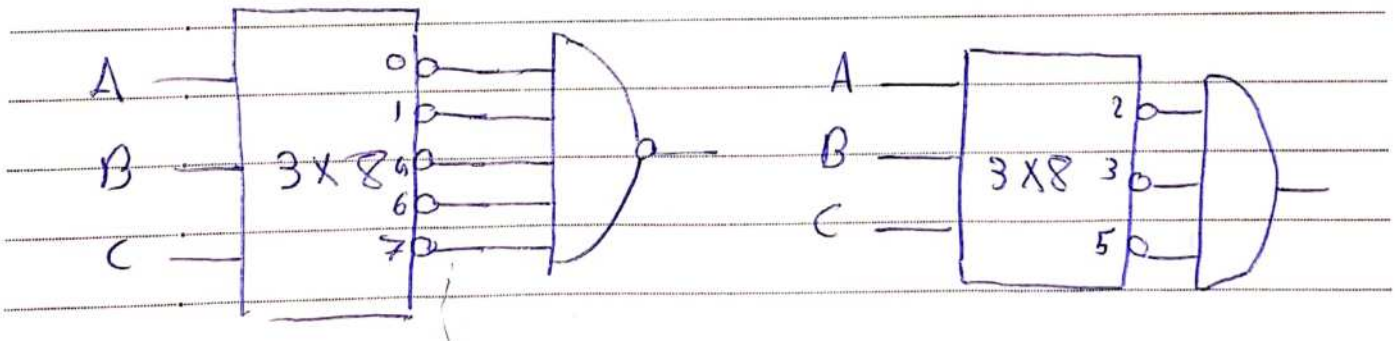
① Active high

$F = \pi(2,3,5)$



② Active low

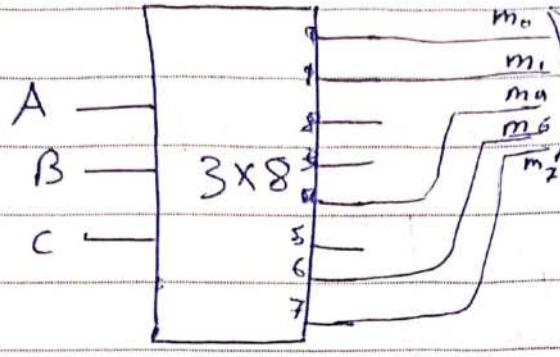
$F(ABC) = (0,1,4,6,7)$ $F = \pi(2,3,5)$



Ex: Implement $F(A,B,C) = \Sigma(0,1,4,6,7)$ using decoder

* Σ + Active high → OR
* Π + Active high → NOR

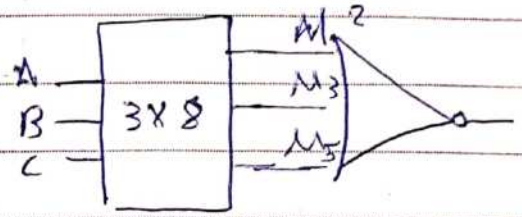
Active high decoder



$m_0 + m_1 + m_4 + m_6 + m_7$

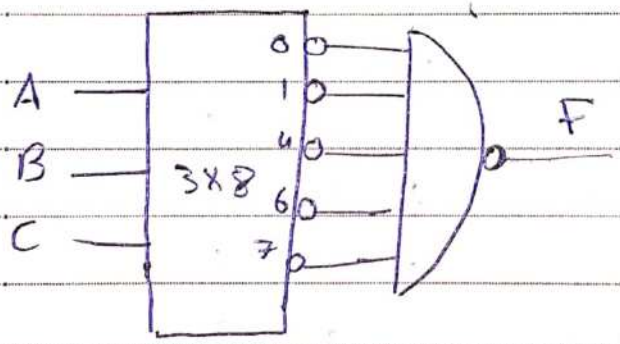
$= \Sigma(0,1,4,6,7)$

$F = \Pi(2,3,5)$ →



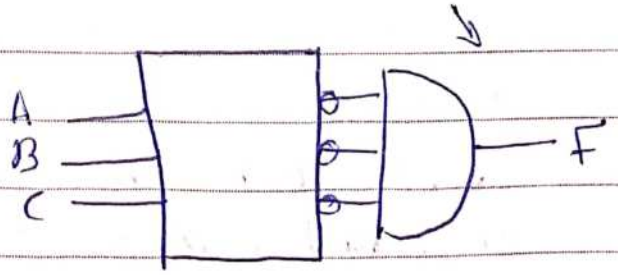
Active low decoder

* Σ + Active low → Nand
* Π + Active low → And



$F = \Sigma(0,1,4,6,7)$

$F = \Pi(2,3,5)$



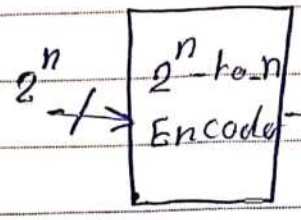
* Encoders:-

→ Performs the inverse operation of a decoder

→ # inputs = 2^n , # outputs = n

→ only one input is allowed to be active

→ output lines generate the binary code corresponding to the active input



Ex: octal to binary encoder 8X3

Invalid input	Inputs								outputs			
	E_7	E_6	E_5	E_4	E_3	E_2	E_1	E_0	A_2	A_1	A_0	V
←	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	1	0	0	0	1
	0	0	0	0	0	0	1	0	0	0	1	1
	0	0	0	0	0	1	0	0	0	1	0	1
	0	0	0	0	1	0	0	0	0	1	1	1
	0	0	0	1	0	0	0	0	1	0	0	1
	0	0	1	0	0	0	0	0	1	0	1	1
	0	1	0	0	0	0	0	0	1	1	0	1
	1	0	0	0	0	0	0	0	1	1	1	1

* two Problems

→ All outputs are 0's in 2 cases:

① all inputs = 0 ② $E_0 = 1$

solution: add one more output (V) if inputs is valid

→ If two inputs are active at the same time

$A_2A_1A_0$

$E_5 = 1$ 101 → 111 → 7

$E_2 = 1$ 010 ✗

Solution:-

use Priority Encoder → Produces output corresponding to the input with higher priority

$E_5 = 1$ $A_2A_1A_0$ * نأخذ الرقم الأكبر

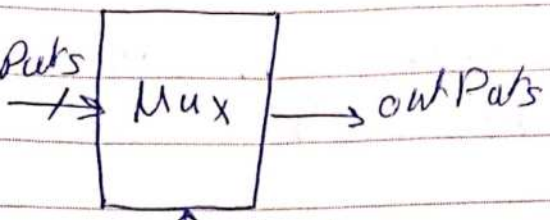
$E_2 = 1$ x 101

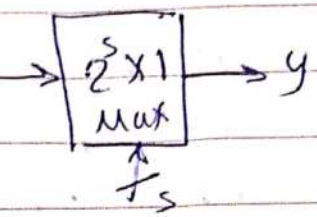
Ex: 4-to-2 Priority Encoder

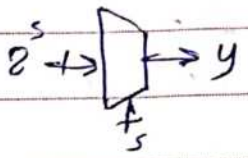
E_3	E_2	E_1	E_0	A_1	A_0	V
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	0	1	1	0	1	1
0	1	0	0	1	0	1
0	1	0	1	1	0	1
0	1	1	0	1	0	1
0	1	1	1	1	0	1
1	0	0	0	1	1	1
1	0	0	1	1	1	1
1	0	1	0	1	1	1
1	0	1	1	1	1	1
1	1	0	0	1	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

* Multiplexers (data selector)

- A multiplexer is a combinational circuit that selects on of the 2^s input lines to be passed to a single output line



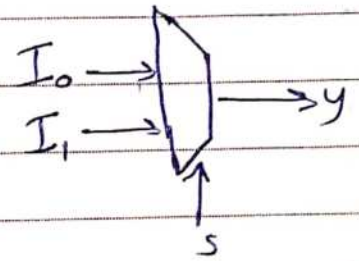




Ex: 2 x 1 Mux

① → # select s 1

2 x 1

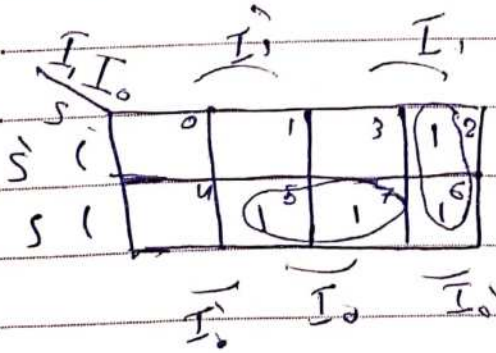


$S=0 \rightarrow y = I_0$

$S=1 \rightarrow y = I_1$

OL: I_s Li *

I_1	I_0	S	y	
0	0	0	0	0
0	0	1	0	1
0	1	0	1	2
0	1	1	0	3
1	0	0	0	4
1	0	1	1	5
1	1	0	1	6
1	1	1	1	7

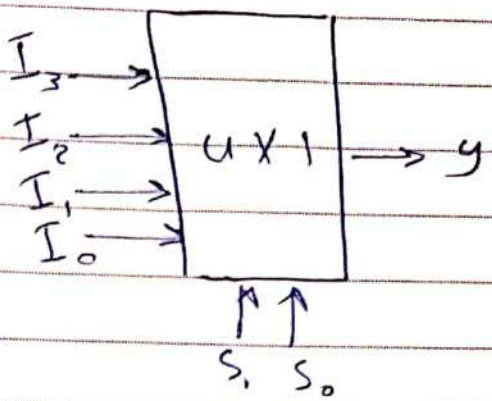


$y = S\bar{I}_0 + I_1 I_0$

Ex: 4x1 Mux

$4 \times 1 = 2^{(2)} \times 1$ → select

S_1 S_0	y
0 0	I_0
0 1	I_1
1 0	I_2
1 1	I_3



* Implementation of logic circuits using Mux

- If we have an n-variable logic function we can implement using Mux.

① using a mux with n-select lines
② using a mux with less than n-select lines

Ex: Implement

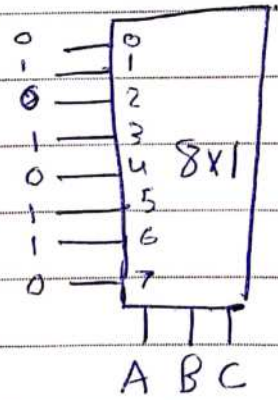
$$F(A,B,C) = \Sigma(1,3,5,6)$$

using

① 8x1 Mux

②

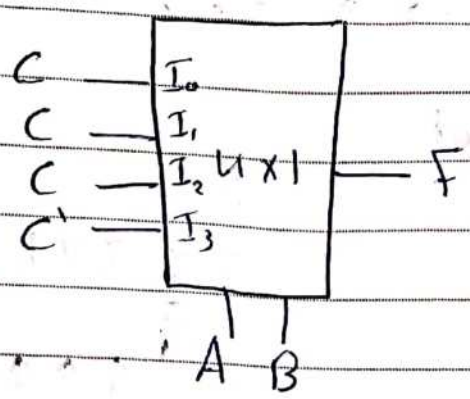
2 x 1 → #select ≤ 3 ≤ #inputs



A B C	F
0 0 0	0
0 0 1	1
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	0

② use a 4x1 Mux $F(ABC) = \Sigma(1,3,5,6)$

$4 \times 1 = 2^2 \times 1 \rightarrow$ # select = 2

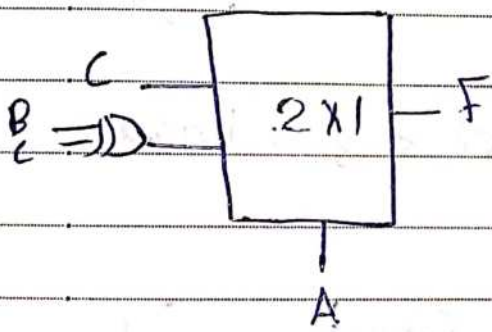


select msB select

A	B	C	F	
0	0	0	0	I_0
0	0	1	1	
0	1	0	0	I_1
0	1	1	1	
1	0	0	0	I_2
1	0	1	1	
1	1	0	1	I_3
1	1	1	0	

③ use 2x1 Mux $F(ABC) = \Sigma(1,3,5,6)$

2 x 1 → # select = 1



select

A	B	C	F	
0	0	0	0	I_0
0	0	1	1	
0	1	0	0	
0	1	1	1	
1	0	0	0	I_1
1	0	1	1	
1	1	0	1	
1	1	1	0	

$I_0 = B'C + BC$

$= C(B' + B)$

$= C$

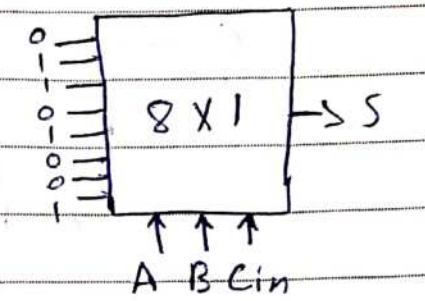
$I_1 = B \oplus C$

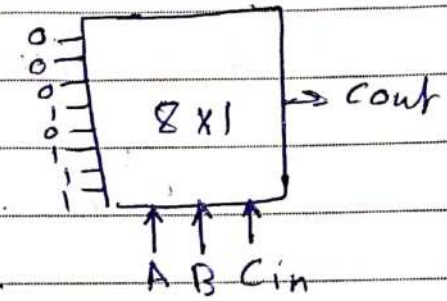
Ex: Implement FA using

① 8 x 1 Mux

$8 \times 1 \rightarrow 2^3 \times 1 \rightarrow$ # select = 3 = inputs

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

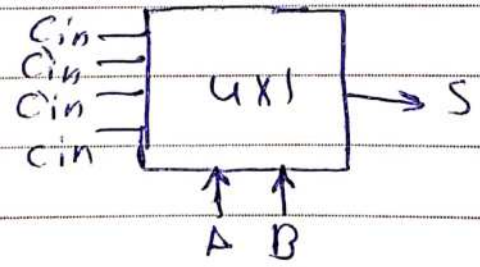


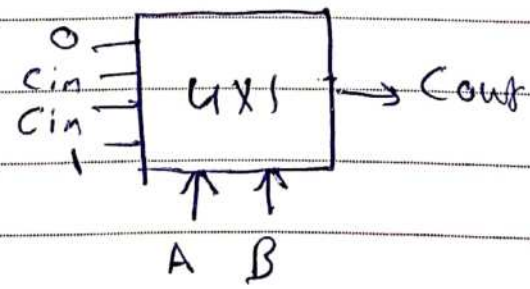


② 4 x 1 Mux

$4 \times 1 \rightarrow 2^2 \times 1 \rightarrow$ select ≤ 3 = inputs

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	Cin	0
0	1	0	Cin	0
0	1	1	0	1
1	0	0	Cin	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





③ 2 X 1 Mux

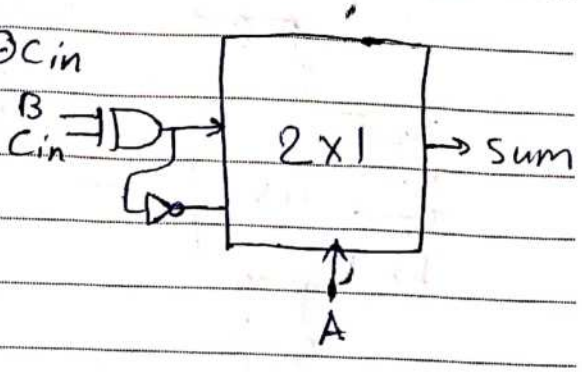
2 X 1 → 2' X 1 → #select = 1

select

msB

A	B	Cin	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

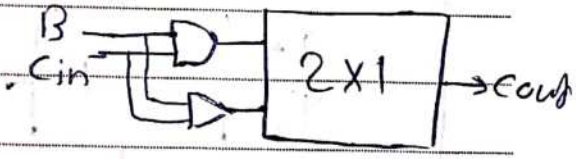
→ B ⊕ Cin



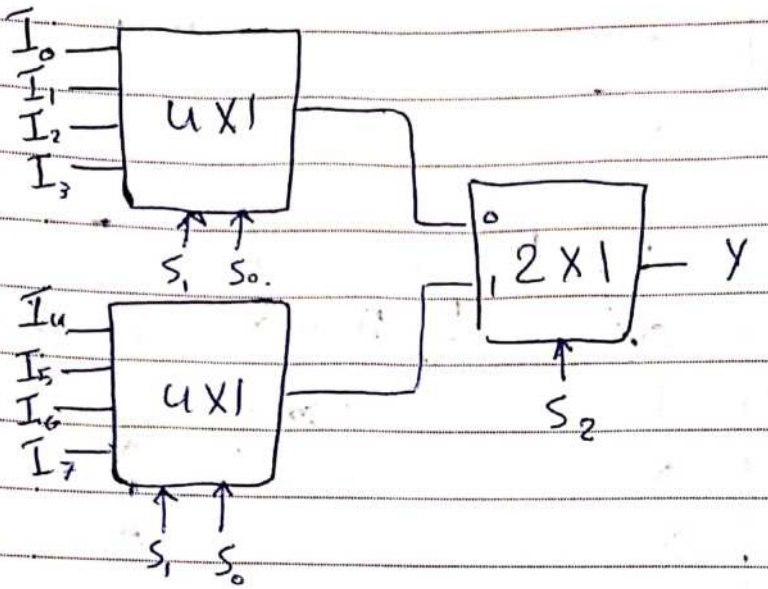
A	B	Cin	Cout
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Cout = B · Cin

Cout = B + Cin

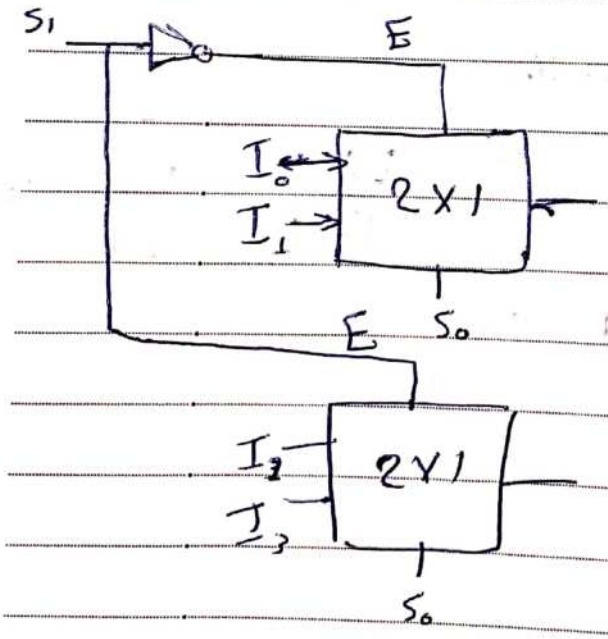


Ex: Construct an 8x1 Mux using 4x1 and 2x1 Muxs.



S_2 S_1 S_0	Y
0 0 0	I_0
0 0 1	I_1
0 1 0	I_2
0 1 1	I_3
1 0 0	I_4
1 0 1	I_5
1 1 0	I_6
1 1 1	I_7

Ex: Construct an 4x1 Mux using 2x1 Mux with enable.

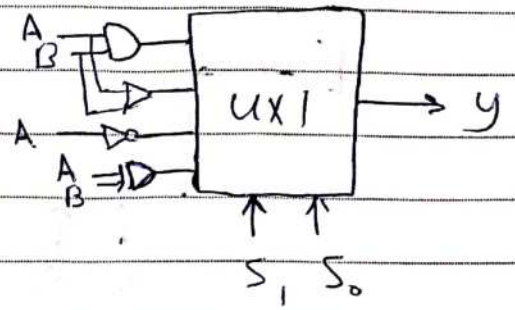


enable

S_1 S_0	Y
0 0	I_0
0 1	I_1
1 0	I_2
1 1	I_3

Ex: Design a circuit to represent the following function using 4x1 Mux and external gates

S_1 S_0	output y
0 0	A OR B
0 1	A and B
1 0	A`
1 1	A XOR B.



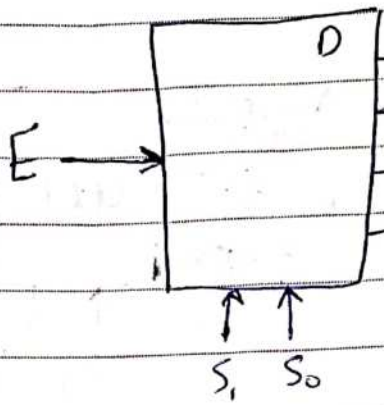
* Demutiplexer

→ Performes inverse of multiplexing operation
→ It has one input line (E) at transmits it to one of 2^n possible output lines
→ The selection of the specific output is controlled by the input on n-select lines

1x2	Mux →	# s = 1
1x4		# s = 2
1x8		# s = 3
1x16		# s = 4

Ex: 1x4 Demux

select lines = 2



$D_0 = E \cdot \bar{S_1} \bar{S_0}$ 0 0

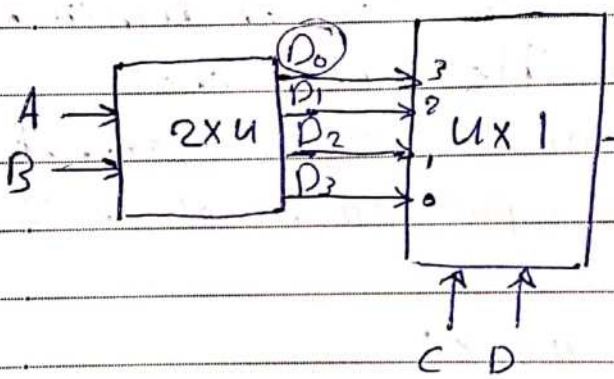
$D_1 = E \cdot \bar{S_1} S_0$ 0 1

$D_2 = E \cdot S_1 \bar{S_0}$ 1 0

$D_3 = E \cdot S_1 S_0$ 1 1

Examples

[1] Fill in the T.T



$D_0 D_1 D_2 D_3$

AB → 00 → 1 0 0 0

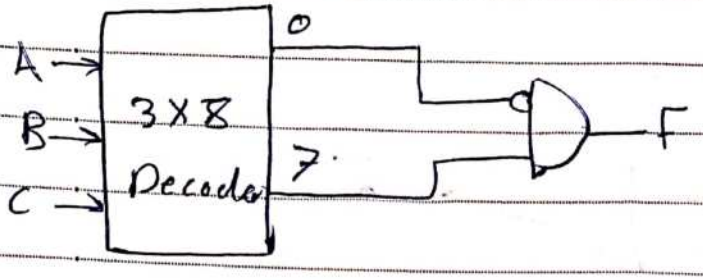
AB → 01 → 0 1 0 0

AB → 10 → 0 0 1 0

AB → 11 → 0 0 0 1

A B C D	y
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	1
0 1 0 0	0
0 1 0 1	0
0 1 1 0	1
0 1 1 1	0
1 0 0 0	0
1 0 0 1	1
1 0 1 0	0
1 0 1 1	0
1 1 0 0	1
1 1 0 1	0
1 1 1 0	0
1 1 1 1	0

Ex.[2] Find F(A,B,C) in SoP



* each output of decoder is minterm

$F = m_0 \cdot m_7$

$= (A'B'C')' \cdot (ABC)$

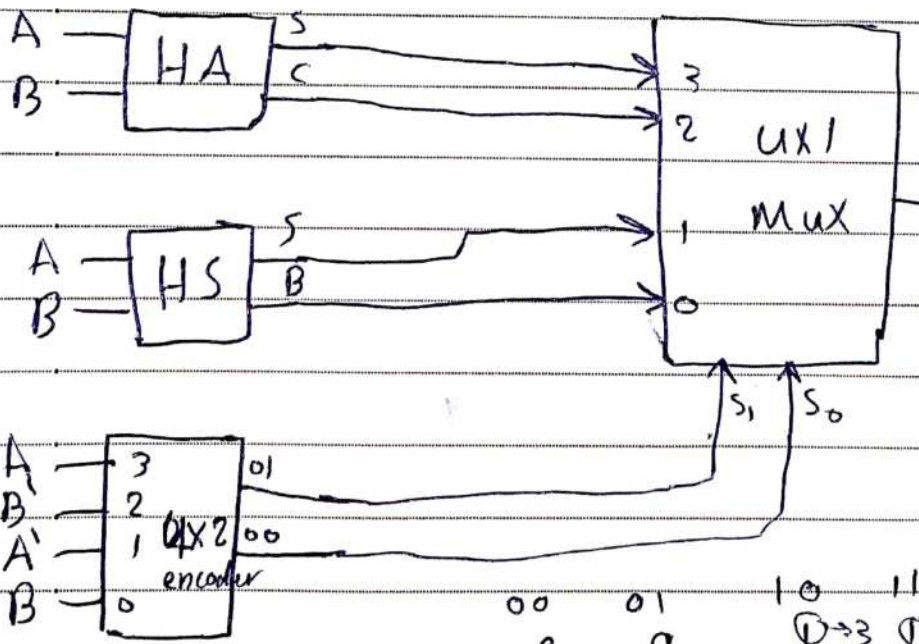
$= (A+B+C) \cdot (ABC)$

$= AABC + ABBC + ABCC$

$= ABC + ABC + ABC$

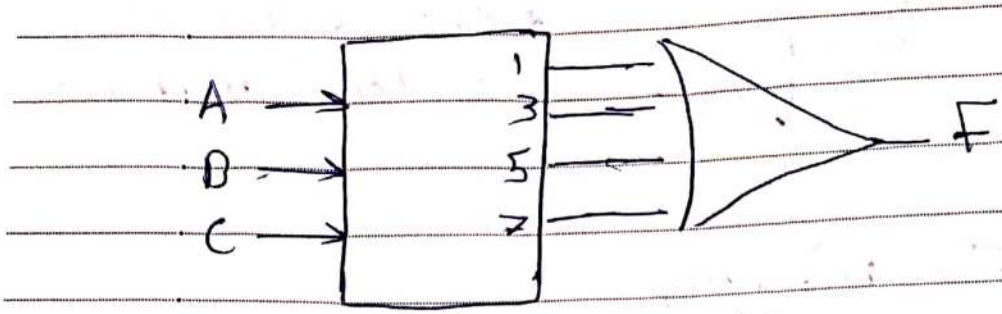
$= ABC$

Ex: fill the table



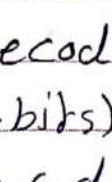
A	B	y
0	0	0
0	1	1
1	0	1
1	1	0

Ex: Build $F(A,B,C) = \Sigma(1,3,5,7)$ using decoder

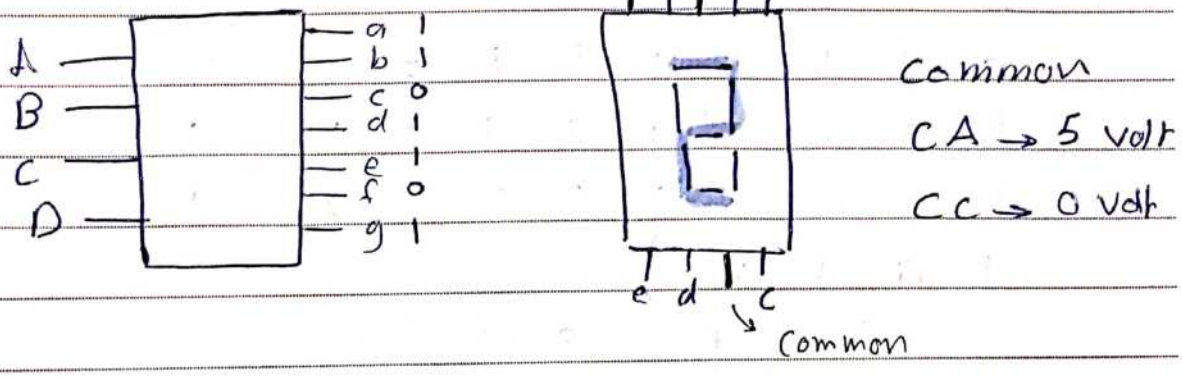


* BCD to 7 segment Display Decoder :-

- The BCD-7 segment decoder
  input : BCD code (4-bits)
  output : 7-outputs (a, b, c, d, e, f, g)

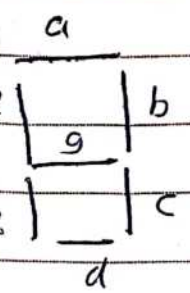


outputs drive the 7-segment display devices to produce decimal read out



- Two types of 7-segment.
  ① Common Anode (Active low)
  ② Common Cathode (Active high)

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	dont care						
:										
1	1	1	1							



$a = \Sigma(0, 2, 3, 5, 6, 7, 8, a)$

$b = \Sigma(0, 1, 2, 3, 4, 7, 8, a)$

$c = \Sigma(0, 1, 3, 4, 5, 6, 7, 8, a)$

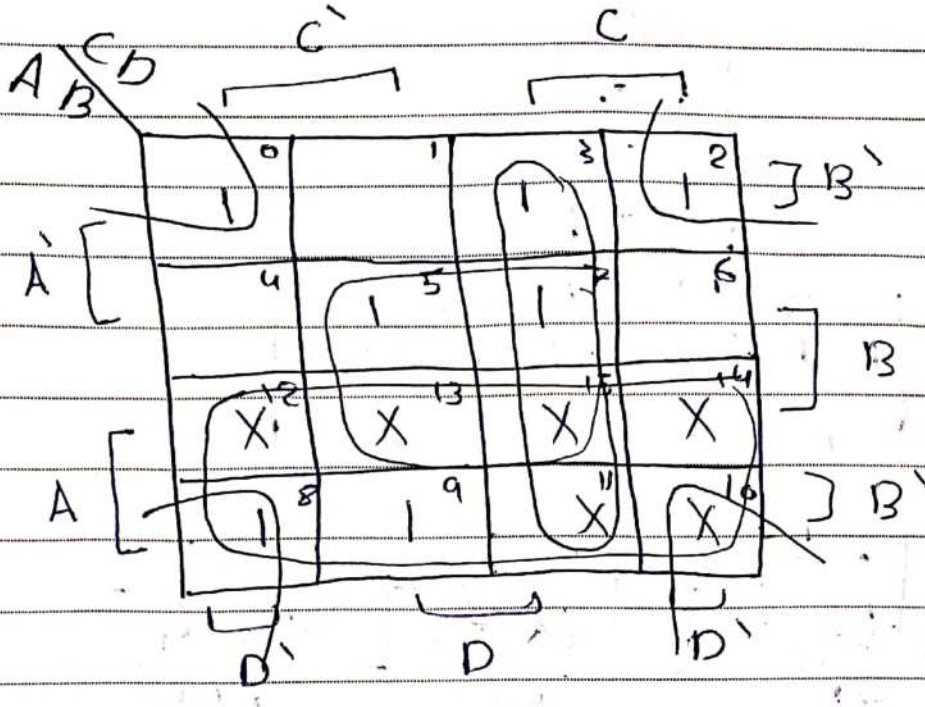
$d = \Sigma(0, 2, 3, 5, 6, 8, a)$

$e = \Sigma(0, 2, 6, 8)$

$f = \Sigma(0, 4, 5, 6, 8, a)$

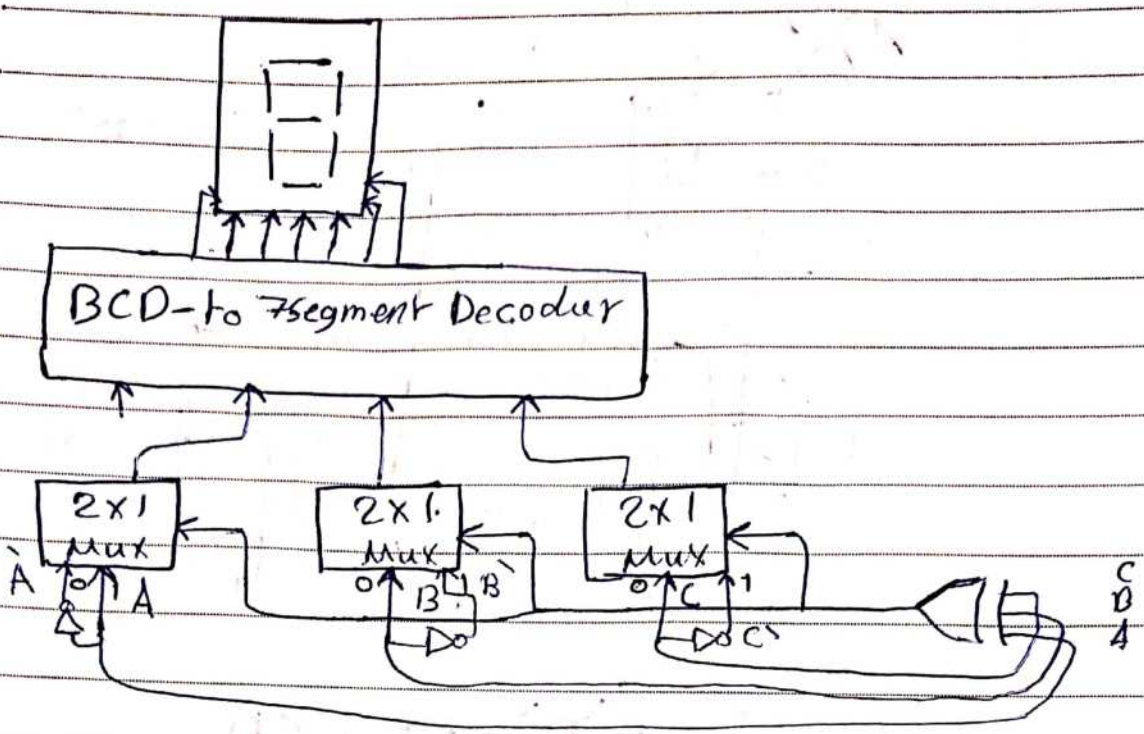
$g = \Sigma(2, 3, 4, 5, 6, 8, a)$

$a = \Sigma(0,2,3,5,7,8,9) + \Sigma_d(10,11,12,13,14,15)$



$a = A + B'D' + BD + CD$

Ex: what is the output shown on the 7-segment display

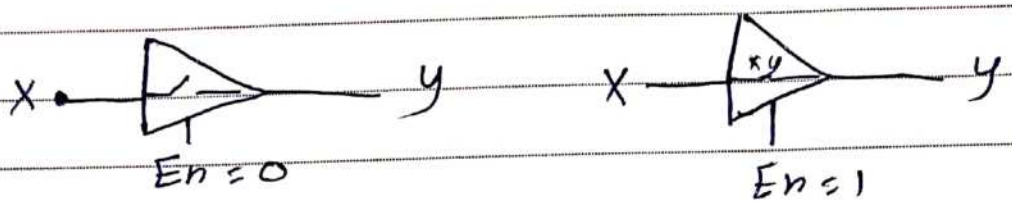


A B C	S	BCD	Display
0 0 0	0	0 0 0 0	8
0 0 1	1	0 1 1 0	6
0 1 0	1	0 1 0 1	5
0 1 1	0	0 0 1 1	3
1 0 0	1	0 0 1 1	3
1 0 1	0	0 1 0 1	5
1 1 0	0	0 1 1 0	6
1 1 1	1	0 0 0 0	0

* Buffer

- A basic logic gate that passes input unchanged to it output
- A Buffer has one input and one output. its output is always equal to its input
- Buffers are used to increas Propogation delay of circuits.

* Tri-state buffer



open circuit
y = high Impedence
open circuit (Z)

y = X

X	y
0	0
1	1

tri-state

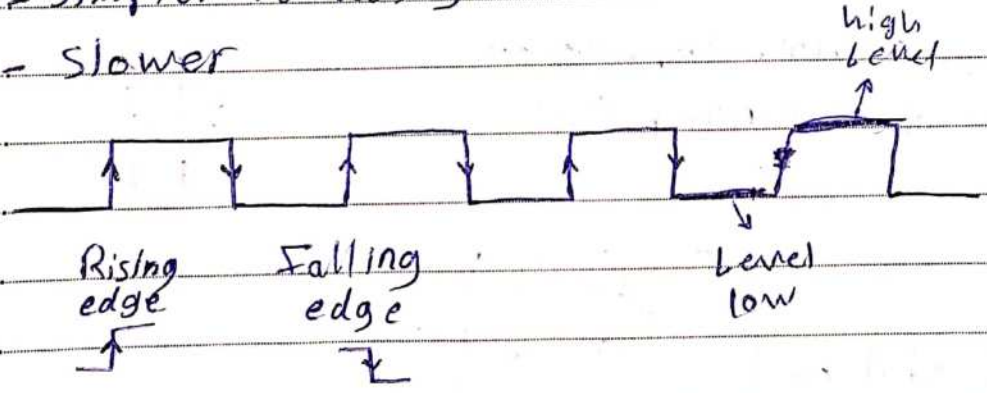
En	X	y
0	X	Z (open circuit)
1	0	0
1	1	1

* Sequential logic circuits

- sequential circuits include memory elements that store binary information.
- outputs in sequential circuits.
  1. Present state (strong element)
  2. Inputs.

* Two types of sequential circuits

[1] Synchronous sequential circuit
- The state is allowed to change at specifice time
- There is a clock (CLK) input
- simpler to design
- slower



[2] Asynchrounous
- The stored value is allowed to change at any time
- There is no clock
- complex to design
- faster

* Basic Memory Elements

[1] Latches:

- operate with signal Level (level sensitive)
- output can change many times in clock cycle

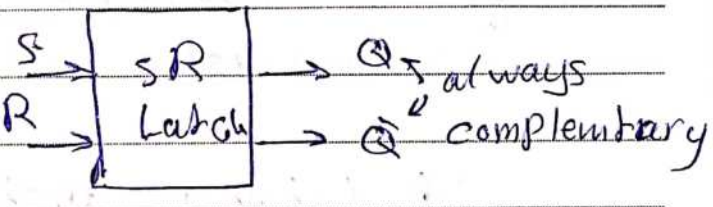
[2] Flip Flops (FFs)

- controled by clock transition (edge sensitive)
- output changes only once in clock cycle

* Latches

[1] S-R Latch (set/Rest latch)

→ has two inputs
① set input
② Rest input



→ has three modes

① set mode → lunch output is set

$Q^+ = 1$
$R = 0 \quad S = 1$

② Rest mode → lunch output is Reset

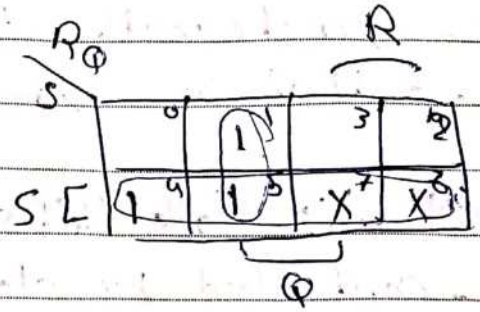
$S = 0 \quad R = 1$
$Q^+ = 0$

③ store mode → lunch output doesn't change

$Q^+ = Q$
$R = 0 \quad S = 0$

State table

	S	R	Q	Q^+
$Q^+ = Q$ store	0	0	0	0
	0	0	1	1
reset $Q^+ = 0$	0	1	0	0
	0	1	1	0
set $Q^+ = 1$	1	0	0	1
	1	0	1	1
un	1	1	0	X
	1	1	1	X

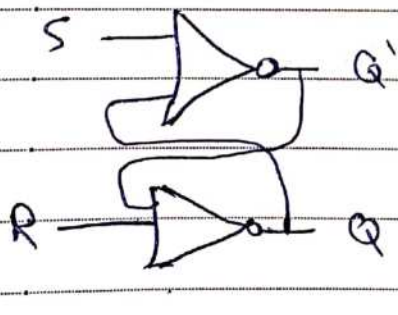


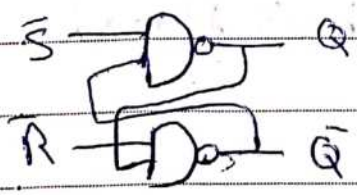
$Q^+ = S + R'Q$

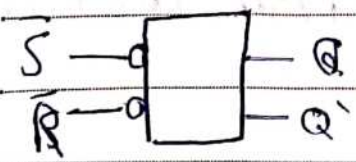
$Q^+ = \Sigma(1,4,5) + \Sigma d(6,7)$

Charecterstic equation

- SR latch is constructed from two cross coupled nand/noR gates

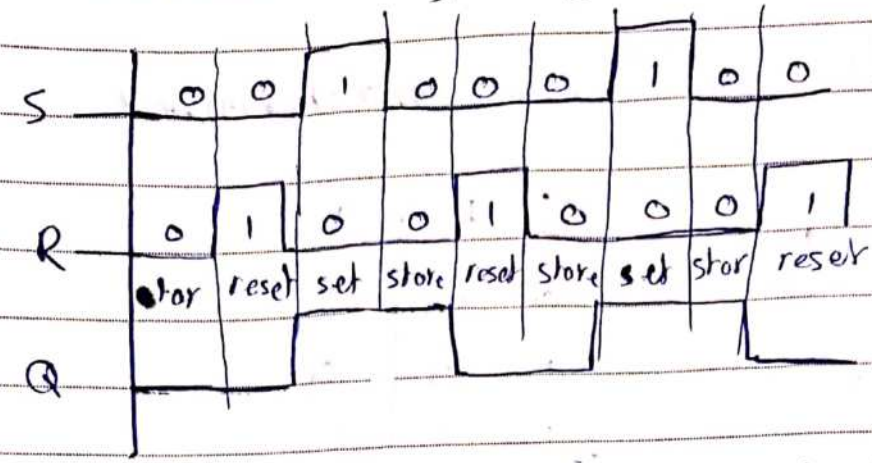






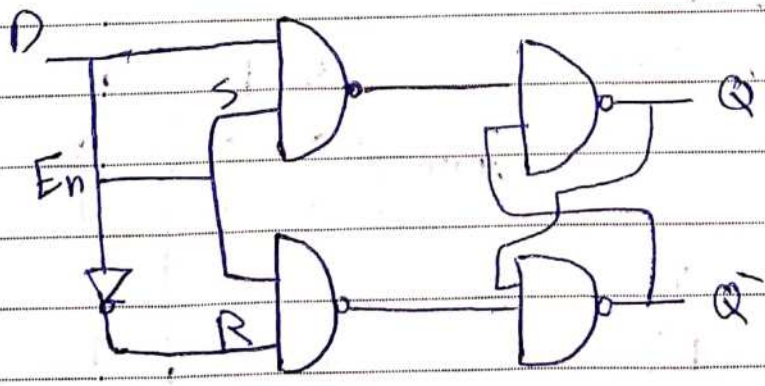
	$\bar{S}$	$\bar{R}$	Q	Q^+
SR || un	0	0	0	X
	0	0	1	X
SR 10 set	0	1	0	1
	0	1	1	1
SR 01 reset	1	0	0	0
	1	0	1	0
SR 00 state	1	1	0	0
	1	1	1	1

Ex: (timing diagram) SR Latch



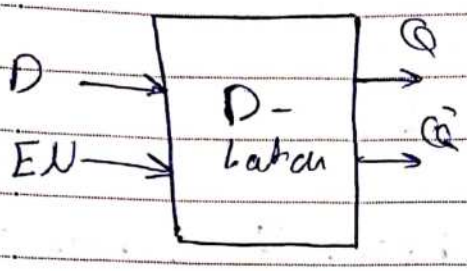
- The operation of the basic SR-latch can be modified by providing an additional input signal that controls when the latch operates

	En	S	R	Q^+
	0	X	X	Q
store ←	1	0	0	Q^+
reset ←	1	0	1	0
set ←	1	1	0	1
X ←	1	1	1	X



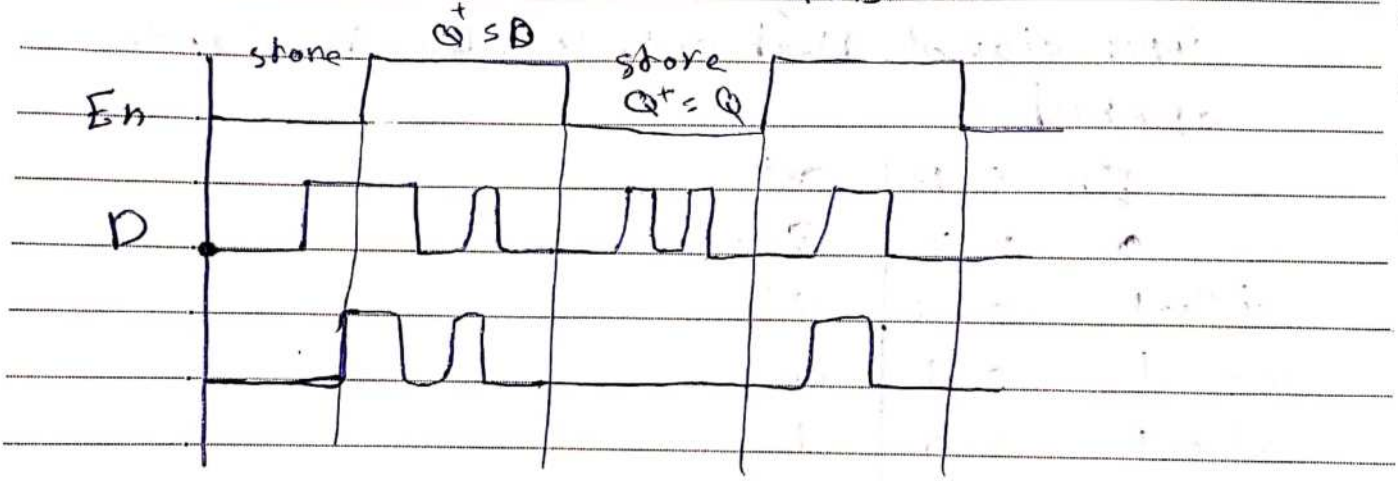
[2] D-Latch (Data Latch)

- To solve the condition of undetermined state



En	D	Q^+
0	x	Q
1	0	0
1	1	1

$Q^+ = D$



[3] Gated D-Latch (Transparent Latch)

- has two inputs

① Gate input (G)

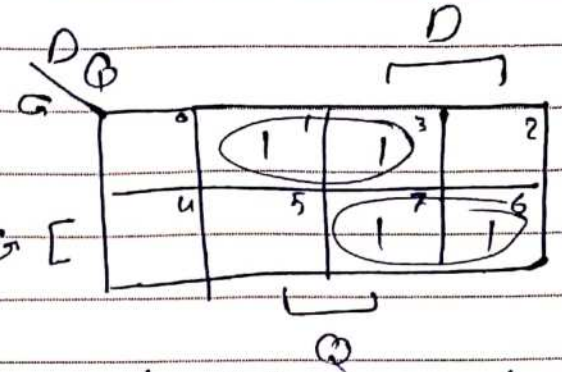
② Data input

$G = 0$ $(Q^+ = Q)$

$G = 1$ $(Q^+ = D)$

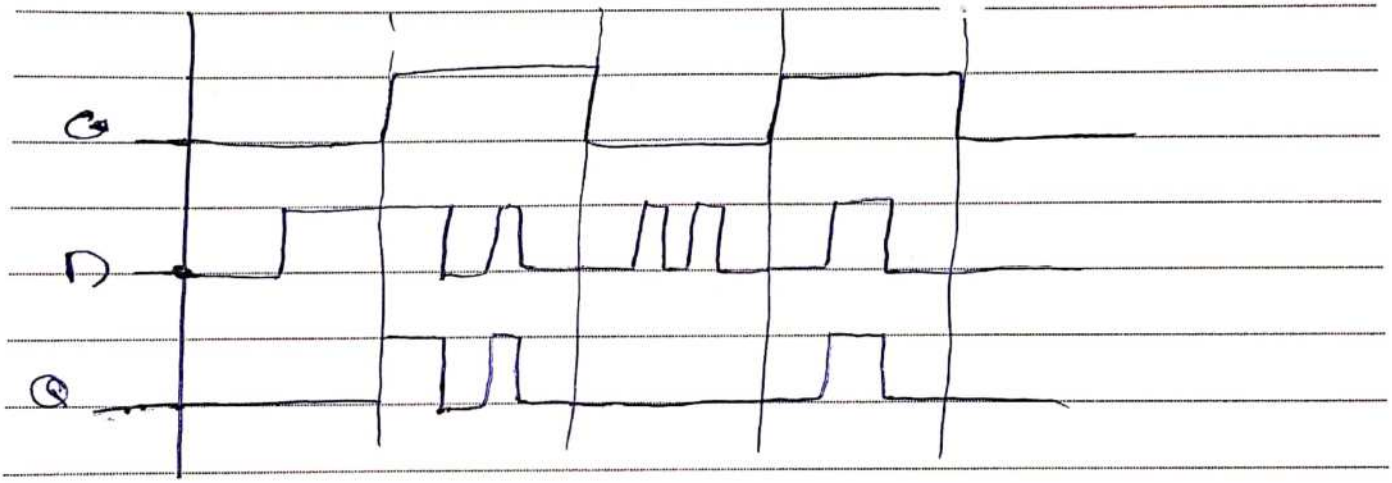
G	D	Q^+	
0	0	0	store
0	1	1	store
1	0	0	
1	1	1	

G	D	Q	Q^+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



$Q^+ = \Sigma(1,3,6,7)$

$= G'Q + GD$



22/8/

Flip Flops

- Edge sensitive memory elements
- Responce may be on either ① Rising edge ↑
  ② Falling edge ↓

- Operations Performed by ① Set ($Q^+=1$) ③ Store ($Q^+=Q$)
  ② Reset ($Q^+=0$) ④ Toggle ($Q^+=\bar{Q}$)

* Types of Flip Flops

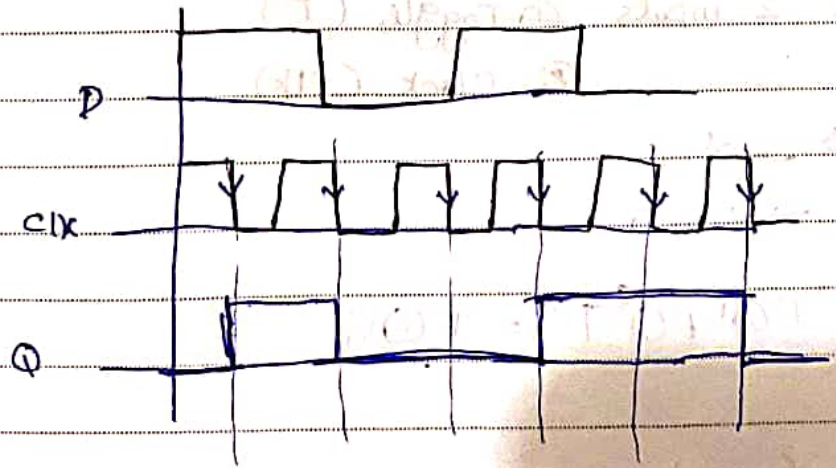
1 - D - Flip Flop Has two inputs ① Data (D)
- The D-Flip Flop is edge triggred ② clk denoted by arrow
- positive edge triggred (responce rising edge)
- Negative " " (" falling ")





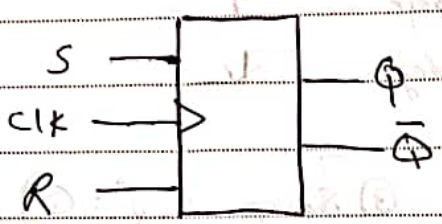
D	Q^+
0	0
1	1

$Q^+ = D$



Types of F.Fs

② SR - Flip Flop - Has 3 inputs ① Set (S - i/p)
② Reset (R - i/p)
③ Clock (Clk)



S	R	Q	Q^+	
0	0	0	0	$Q^+ = Q$
0	0	1	1	
0	1	0	0	$Q^+ = 0$
0	1	1	0	
1	0	0	1	$Q^+ = 1$
1	0	1	1	
1	1	0	x	undet.
1	1	1	x	

SR \ Q	0	1
00		
01		1
11	1	x
10	1	x

$Q^+ = S + R'Q$

Has 3 modes
① Set (S=1, R=0)
② Reset (S=0, R=1)
③ Store (S=0, R=0)

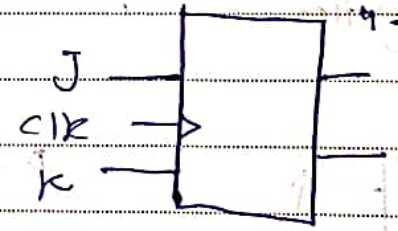
③ J-K-F.Fs - Has 3 inputs ① J → Set
② K → Reset
③ Clk

has 4 modes
1 - Set JK = 10 → $Q^+ = 1$
2 - Reset JK = 01 → $Q^+ = 0$
3 - Store JK = 00 → $Q^+ = Q$
4 - Toggle JK = 11 → $Q^+ = \bar{Q}$

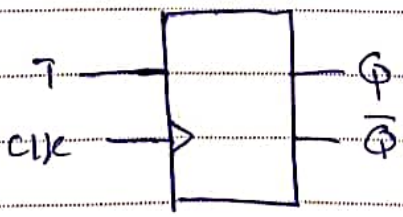
J	K	Q	Q^+	
0	0	0	0	$Q^+ = Q$
0	0	1	1	
0	1	0	0	$Q^+ = 0$
0	1	1	0	
1	0	0	1	$Q^+ = 1$
1	0	1	1	
1	1	0	1	$Q^+ = \bar{Q}$
1	1	1	0	

JK \ Q	0	1
00		
01		1
11	1	
10	1	1

$Q^+ = JQ' + K'Q$



④ T-F.F. (Toggle F.F) Has 2 inputs ① Toggle (T)
② Clock (Clk)



T = 0 → $Q^+ = Q$
T = 1 → $Q^+ = \bar{Q}$

$Q^+ = TQ' + Q^2T' = T \oplus Q$

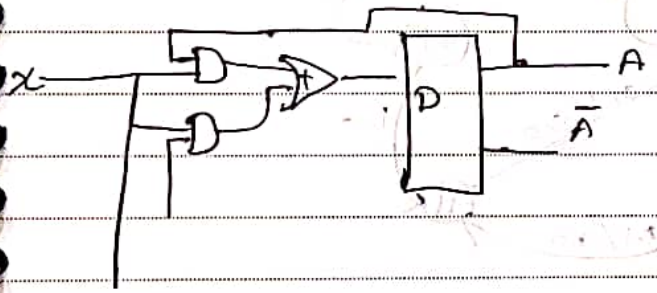
Has 2 modes
① store $Q^+ = Q$ (T=0)
② Toggle $Q^+ = \bar{Q}$ (T=1)

Analysis of sequential circuits

- Obtaining a table or diagram for the time sequence of i/ps & o/ps

- The behaviour of the seq. circuit can be described
  ① algebraically (transition equations)
  ② State table ③ State diagram.

Ex Analyze the sequantial circuit



① ccs Eqn (D) → $Q^+ = D$

$Q_A^+ = D_A \Leftrightarrow A(t+1) = D_A$

$B(t+1) = B_A$

② $D_A = Ax + Bx$

$A(t+1) = A(t)x + B(t)x$ ——— ①

③ $D_B = A'x$

$B(t+1) = A'(t)x$ ——— ②

④ $y(t) = [A(t) + B(t)] \cdot x'(t)$

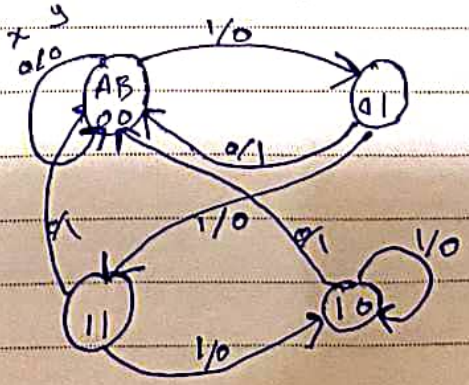
$= A(t)x'(t) + B(t)x'(t)$

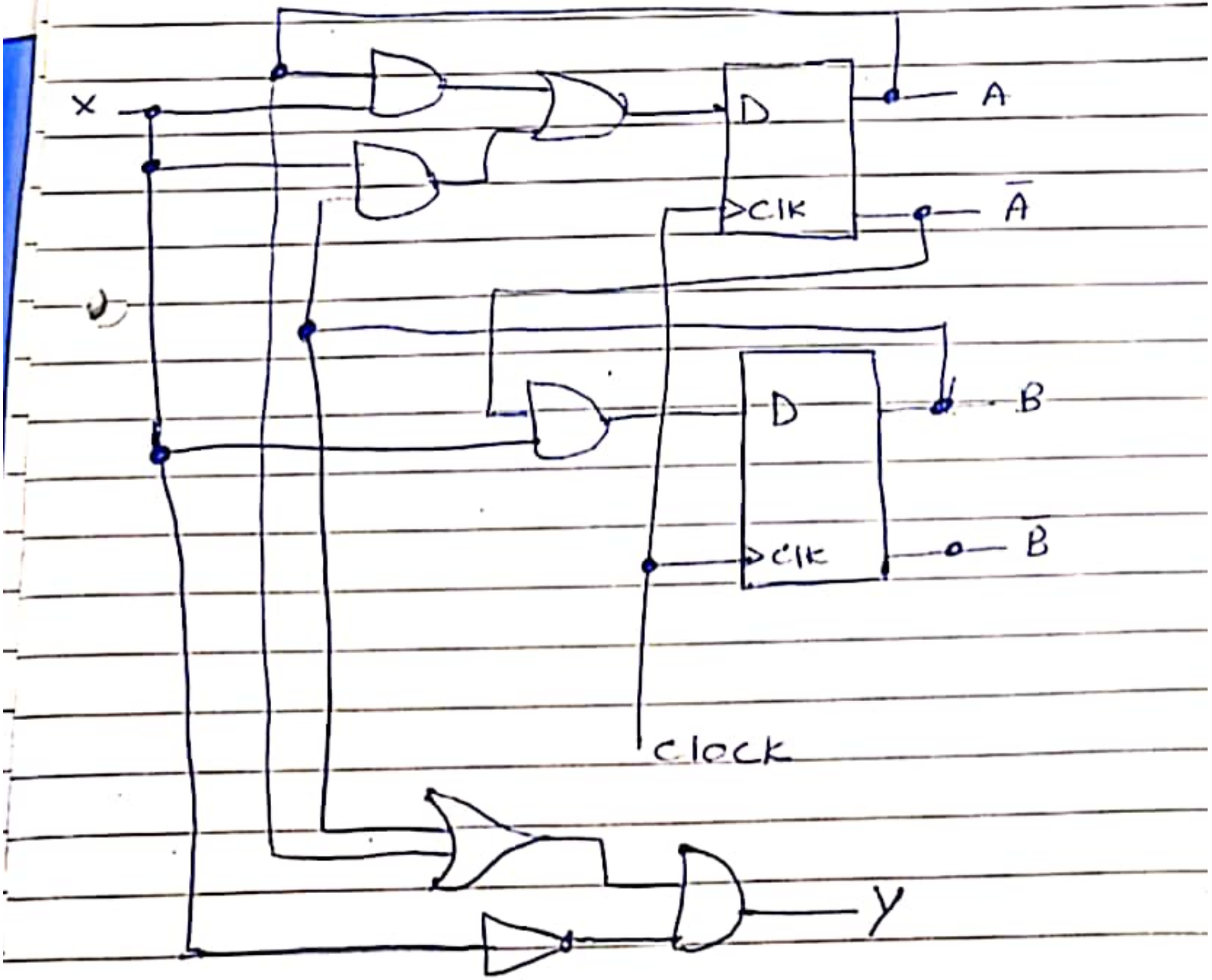
② State table

Present		i/p	Next		o/p
A(t)	B(t)	x	A(t+1)	B(t+1)	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

machines ① mealy y α x [o/p dep on i/p]
② moor

——— state diagram




X
D
A
>CIK
$\bar{A}$
J
D
B
>CIK
$\bar{B}$
clock
Y

EX2 Derive the state table & State diagram

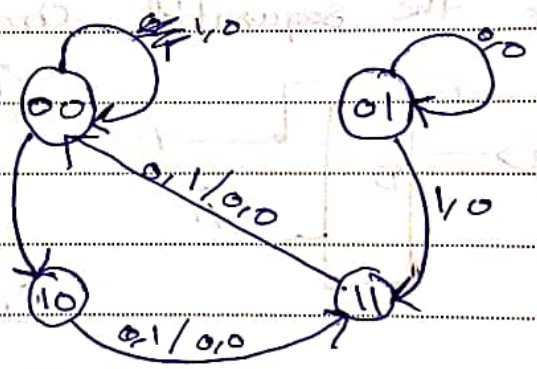
D- F.F → $Q^+ = D$

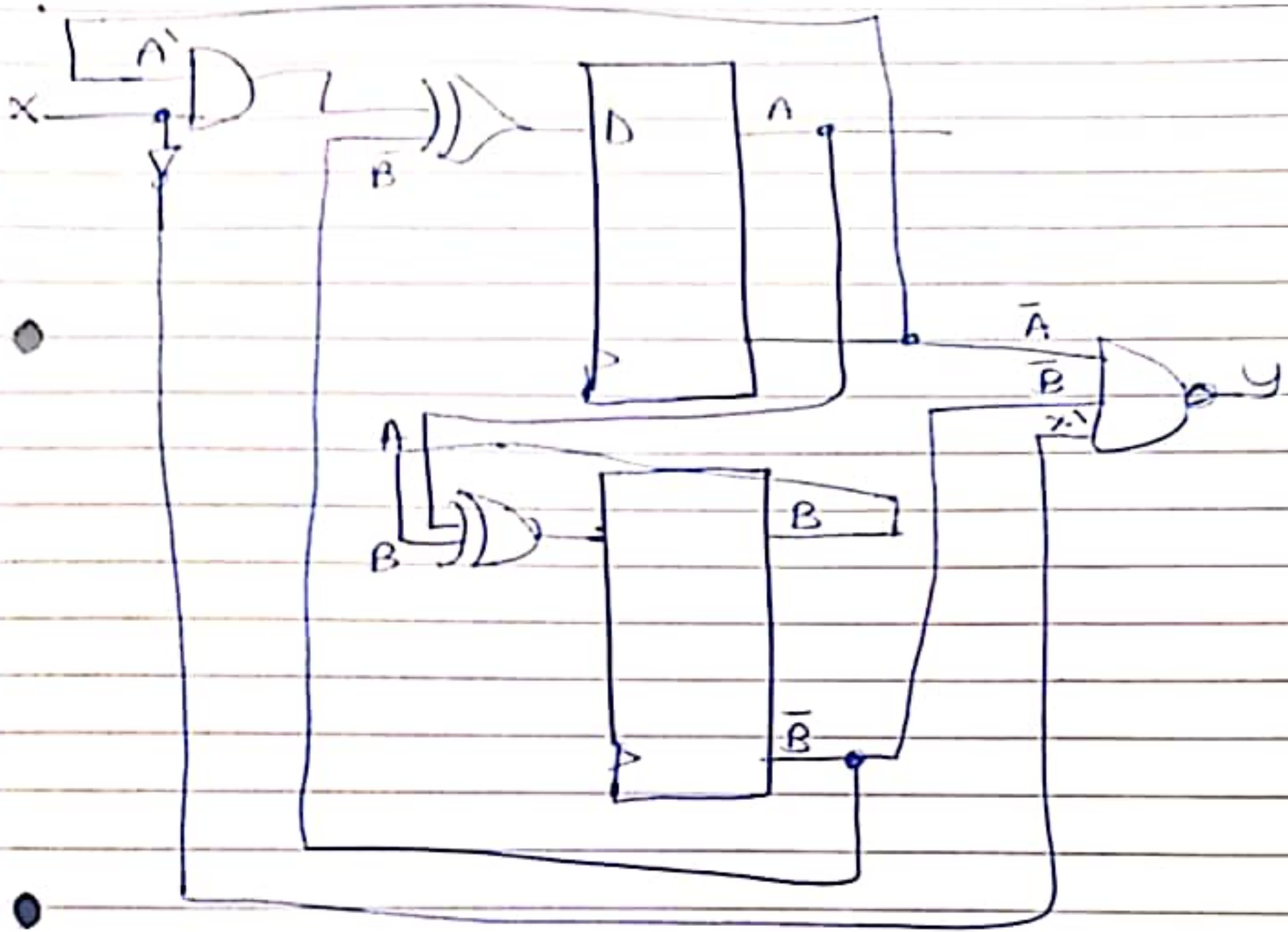
$A^+ = D_A = A'x \oplus B'$

$B^+ = D_B = A \oplus B$

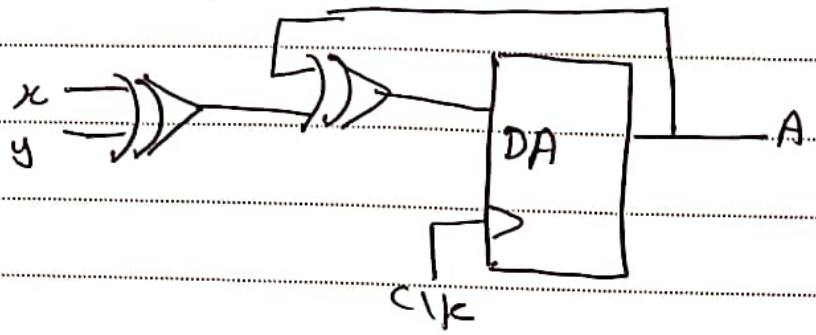
$y = A'B'x'$

A	B	x	A^+	B^+	y
0	0	0	0	0	1
0	0	1	0	0	0
0	1	0	0	1	0
0	1	1	1	1	0
1	0	0	1	1	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	0	0	0




A
X
Y
B
D
A
$\bar{A}$
$\bar{B}$
Y
B
B
B
$\bar{B}$

Ex3 Analyze

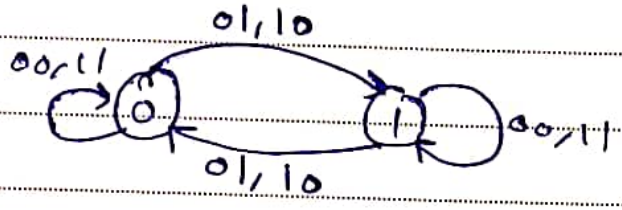


$Q^+ = D$

$A^+ = DA$

$A^+ = x \oplus y \oplus A$

A	x	y	A+
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Ex: Analysis with JK - F.F

① ccs equation

$JK \rightarrow Q^+ = JQ' + K'Q$

$A^+ = J_A A' + K_A' A$, $B^+ = J_B B' + K_B' B'$

$J_A = B$, $K_A = x'B$ $J_B = \bar{x}$, $K_B = A \oplus x$

Sub. J_A, K_A in A^+ Sub J_B, K_B in B^+

$\rightarrow A^+ = BA' + (x + B')A$ $\rightarrow B^+ = x'B' + (A + x')(A' + x)B$

$= A'B + Ax + AB'$ $= B'x' + ABx + A'Bx'$

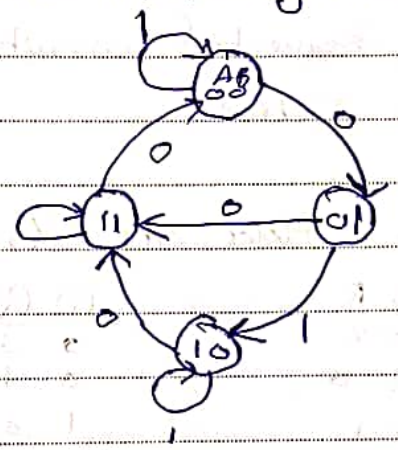
② State table

A	B	x	A^+	B^+
0	0	0	0	1
0	0	1	0	0
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	1
1	1	1	1	1

states → 4

$AKB = 2^2 =$

③ State diagram



Ex T - F.F

① CCS → $Q^+ = T \oplus Q$ $B^+ = T_B \oplus B$

wealy machin $A^+ = T_A \oplus A$ $T_B = x$

y ∝ FFs only $T_A = Bx$ $B^+ = x \oplus B$

$\leftarrow A^+ = Bx \oplus A$ $y = AB$

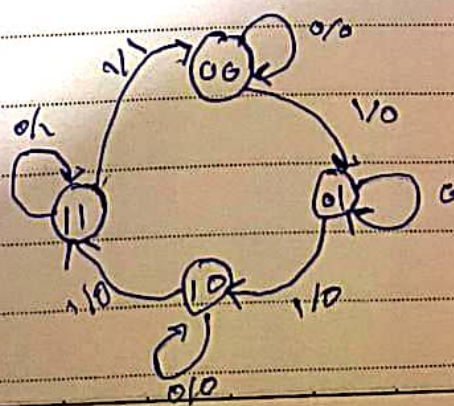
Bx

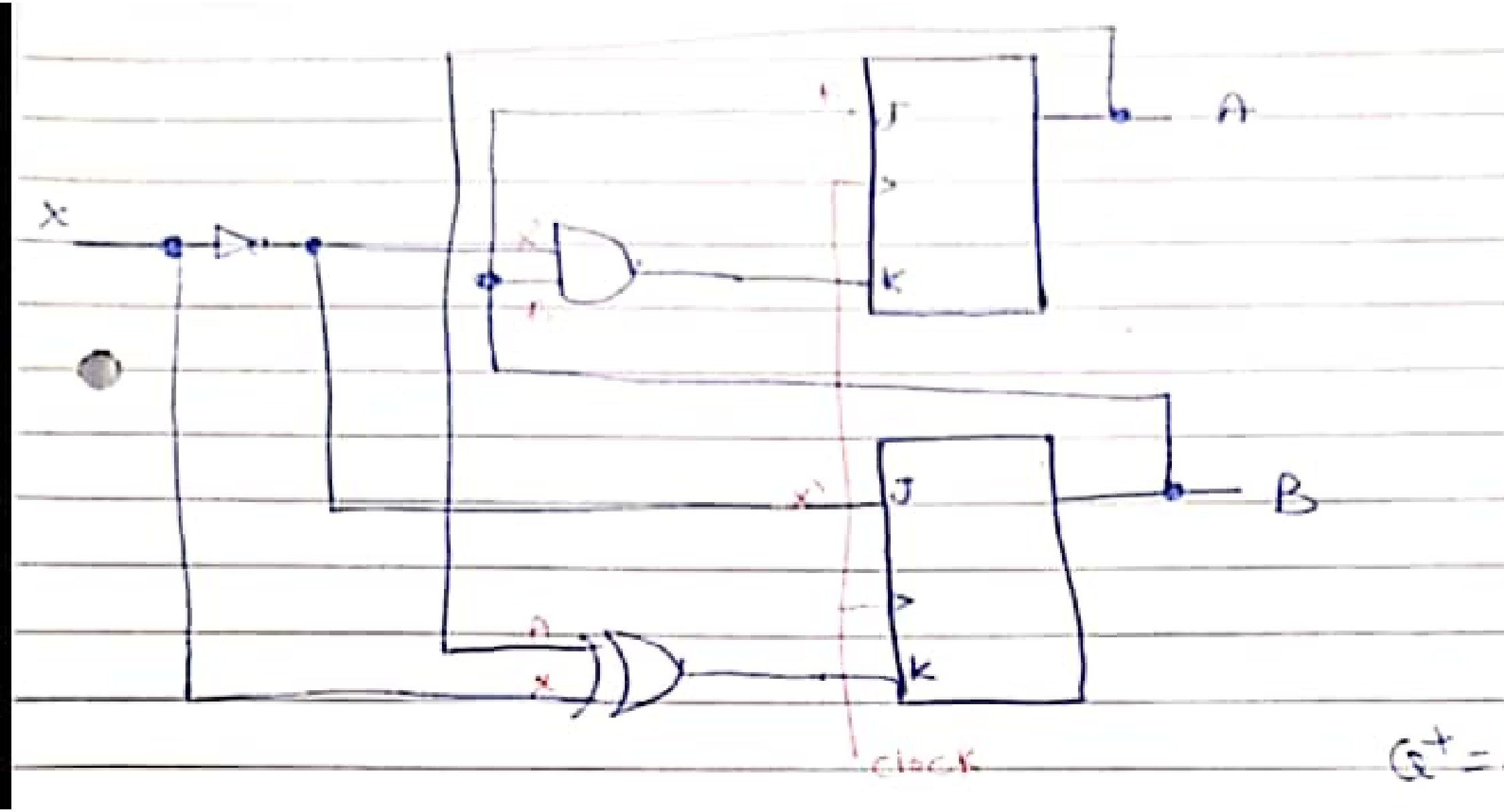
① = A مكتوبة

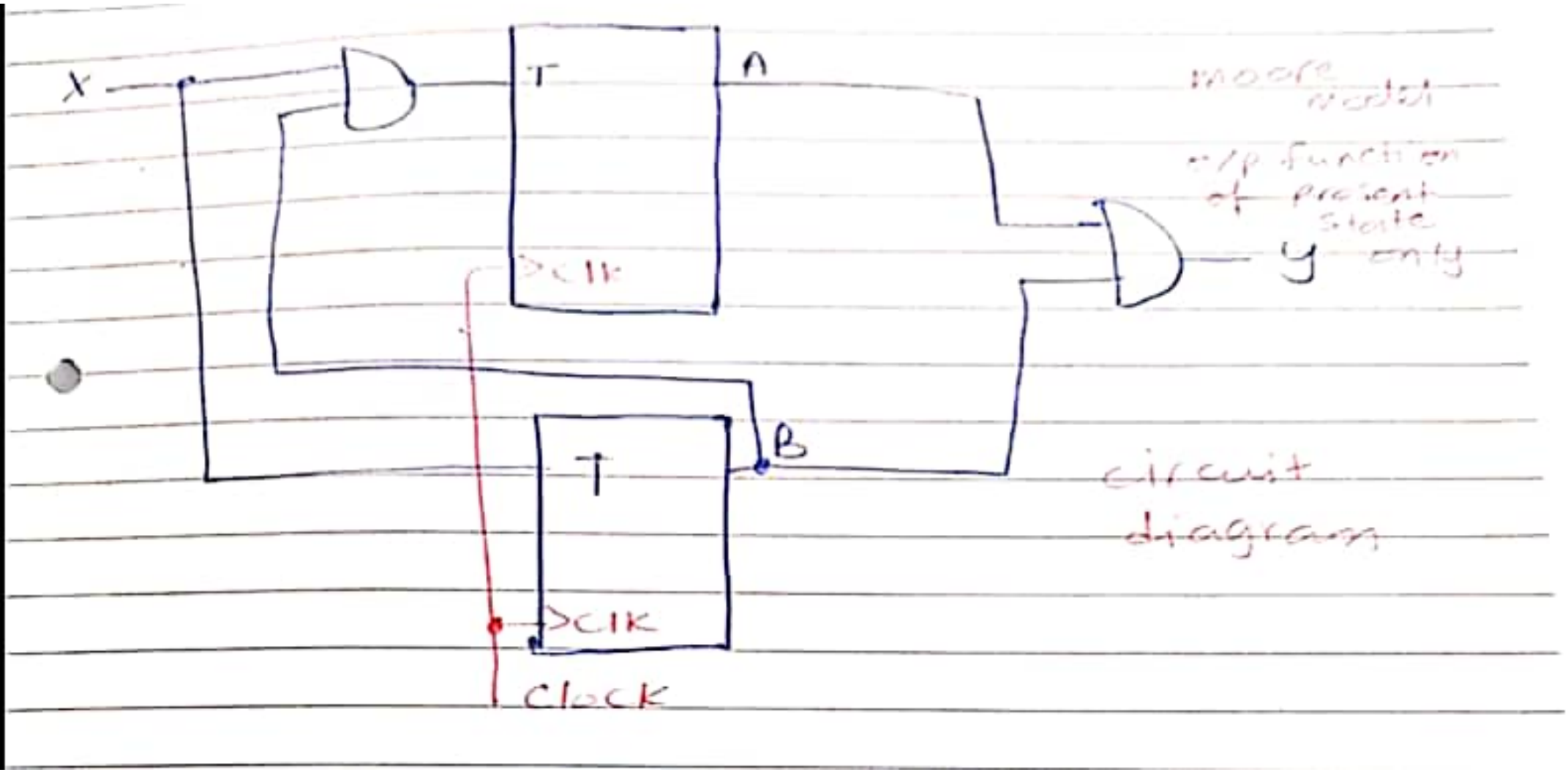
② State table

A	B	x	A^+	B^+	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

③ State diagram



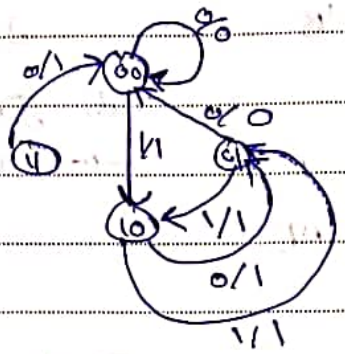

x
J
A
K
J
B
K
clock


X
T
A
>CLK
B
T
>CLK
Clock
moore model
o/p function of present state only
circuit diagram

Ex SR

① CCS equation ⇒ Q^+

A	B	x	A^+	B^+	y
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	1	0	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	0	0	0
1	1	1	0	0	1



$S_A = A'x$ $S_B = (A'+B)'$

$R_A = (A'x)'$ $R_B = A'+B$

$S_R \rightarrow Q^+ = S + R'Q$

$A^+ = S_A + R_A'Q$

$= A'x + A'x\,A$

$A^+ = A'x$

$B^+ = S_B + R_B'B$

$= AB' + AB'B$

$B^+ = AB'$

$y = x + AB'$

mealy model

* Design of sequential circuits

* Excitation Tables

- Excitation tables are derived from characteristic eqs

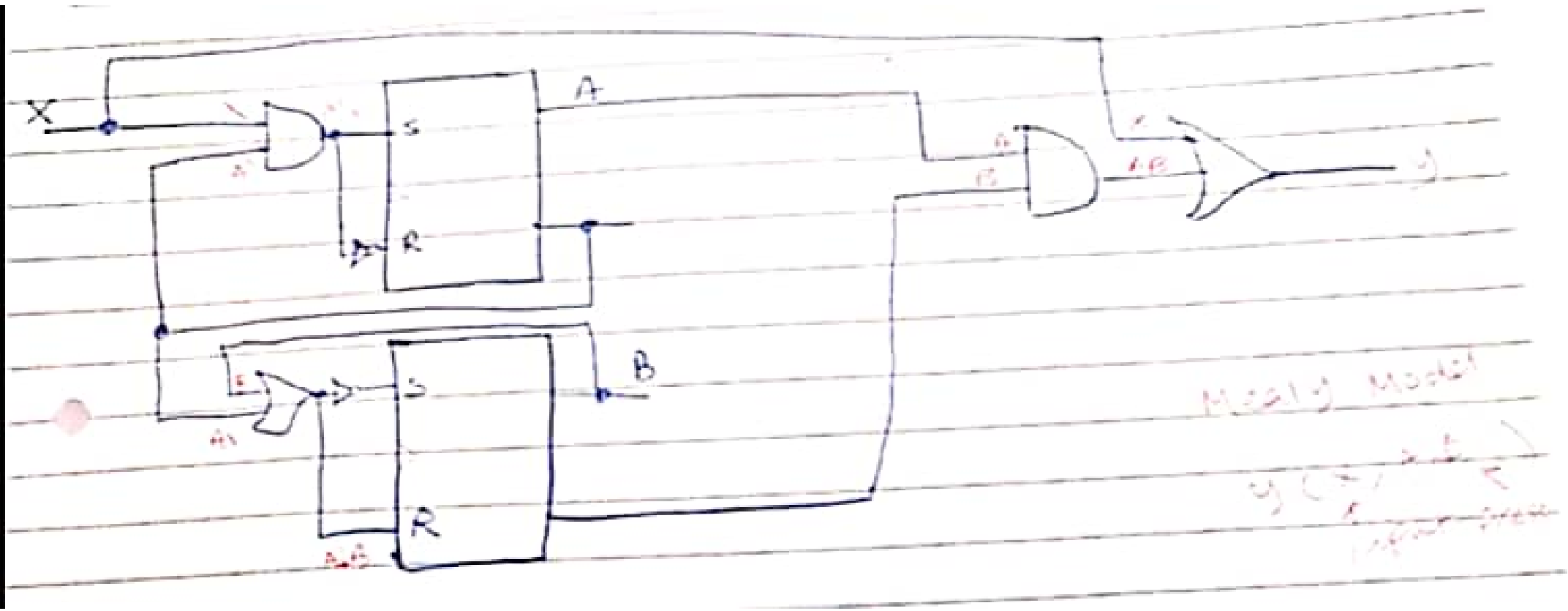
Q	Q^+	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Q	Q^+	D
0	0	0
0	1	1
1	0	0
1	1	1

Q	Q^+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Q	Q^+	T
0	0	0
0	1	1
1	0	1
1	1	0

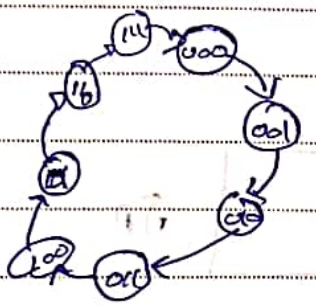
ALADIB


X
S
R
A
B
S
R

Ex Design a 3 bit up Counter using T-F-F 000 → 111

A	B	C	A^+	B^+	C^+	T_A	T_B	T_C
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1

T →	Q	Q^+	T
	0	0	0
	0	1	1
	1	0	1
	1	1	0

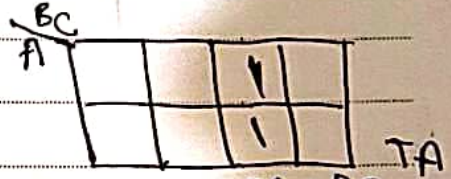


$T_C = 1$

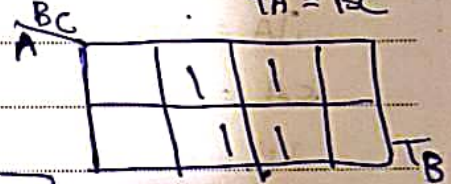
$T_A = \Sigma(3,7) \rightarrow BC$

$T_B = \Sigma(1,3,5,7) \rightarrow C$

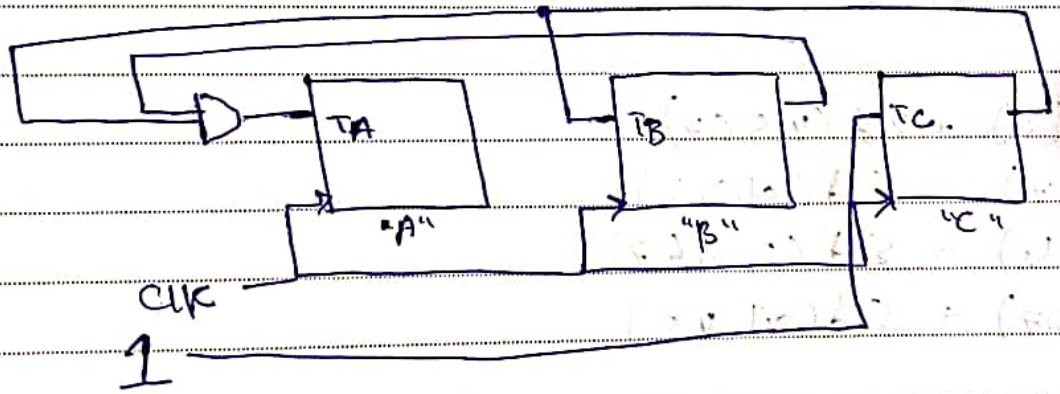
K-map



$T_A = BC$



$T_B = C$



EX Design a 3-bit up Counter using JK

A	B	C	A^+	B^+	C^+	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

Q	Q^+	JK
0	0	0 X
0	1	1 X
1	0	X 1
1	1	X 0

$J_A = BC$, $J_C, K_C = 1$

$K_A = BC$

$J_B = C$

$K_B = C$



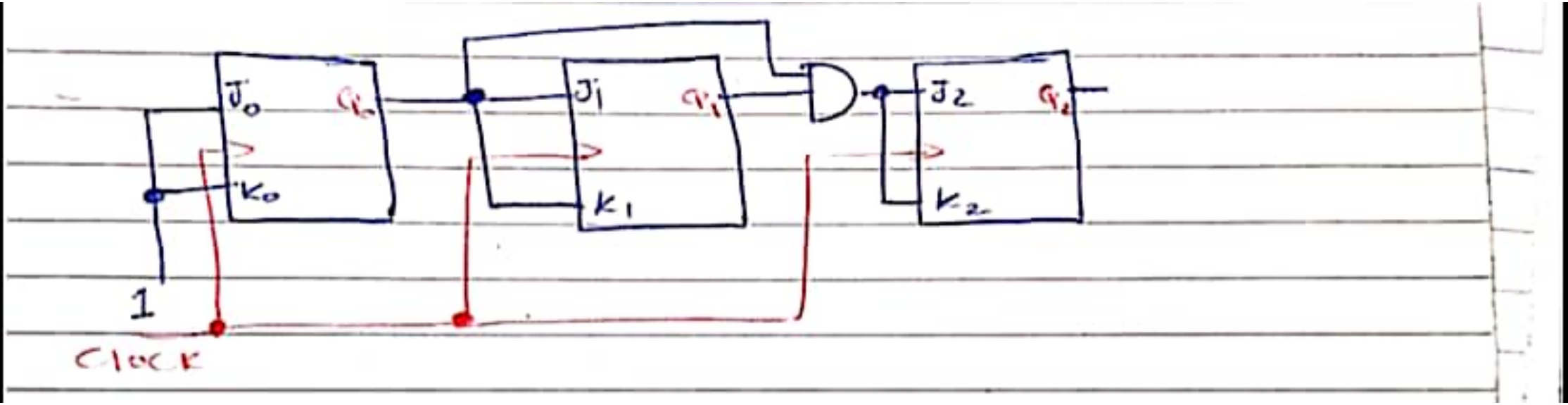
JA

$J_A = \Sigma(3) + \Sigma d(4,5,6,7)$

$K_A = \Sigma(7) + \Sigma d(0,1,2,3)$

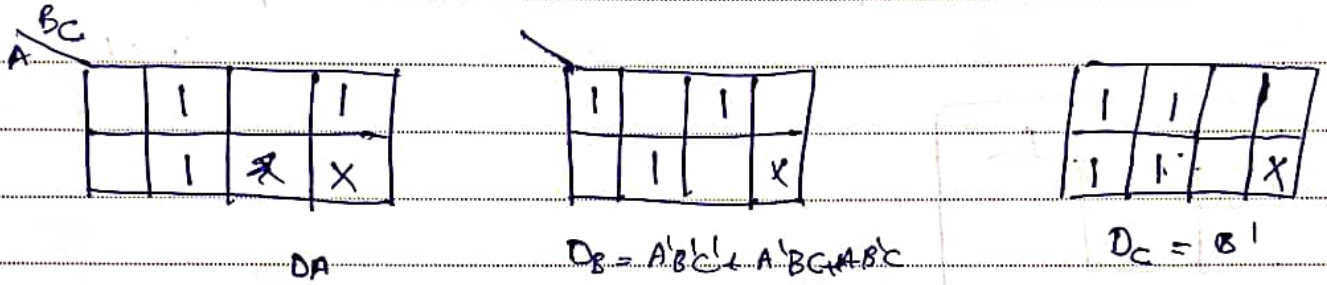
$J_B = \Sigma(1,5) + \Sigma d(2,3,6,7)$

$K_B = \Sigma(3,7) + \Sigma d(0,1,4,5)$


J_0
Q_0
K_0
J_1
Q_1
K_1
J_2
Q_2
K_2
1
Clock

EX Design a counter that goes through the sequence 0,3,2,4,1,5,7 & repeat (use D)

A	B	C	A^+	B^+	C^+	D_A	D_B	D_C
0	0	0	1	1	1	1	1	1
0	0	1	0	1	0	0	1	0
0	1	0	0	0	1	0	0	1
0	1	1	1	0	0	1	0	0
1	0	0	1	0	1	1	0	1
1	0	1	1	1	1	1	1	1
1	1	0	x	x	x	x	x	x
1	1	1	0	0	0	0	0	0

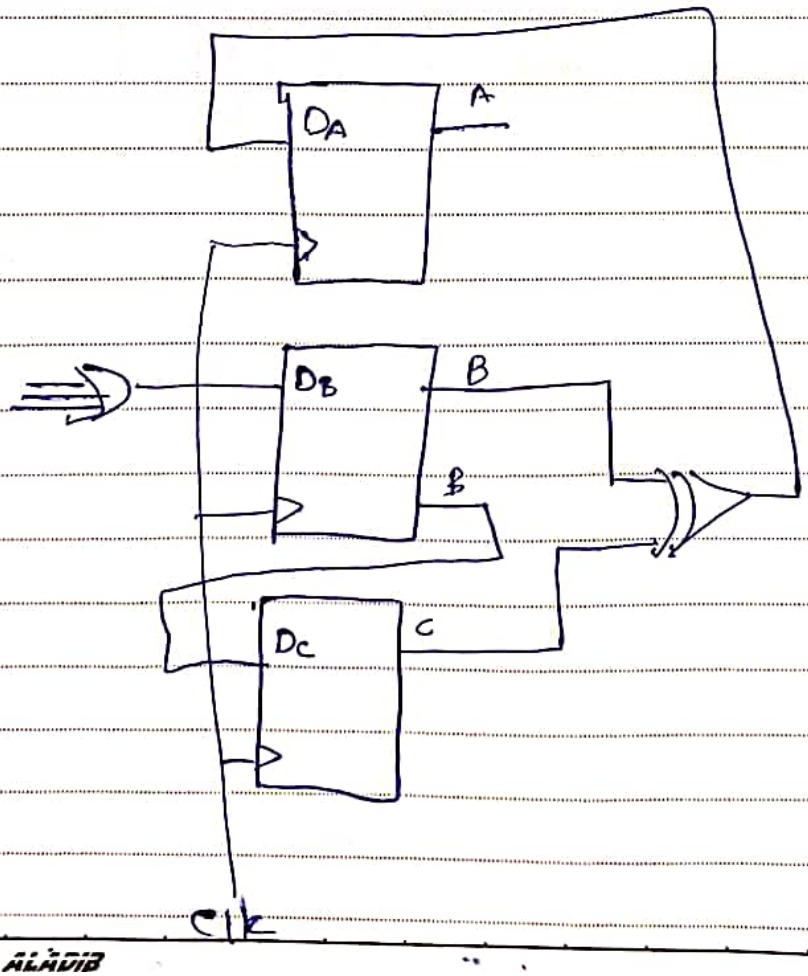


D_A

$D_B = A'B'C' + A'BC + AB'C$

$D_C = B'$

$D_A = B'C + C'B = C \oplus B$



Ex Design a 3 bit down odd counter use T-F-F

↑ 7, 5, 3, 1 → 7, x, 5, x, 3, x, 1, x

A	B	C	A^+	B^+	C^+	T_A	T_B	T_C
0	0	0	x	x	x	x	x	x
0	0	1	0	1	1	0	1	0
0	1	0	x	x	x	x	x	x
0	1	1	1	0	1	1	1	0
1	0	0	x	x	x	x	x	x
1	0	1	1	1	1	0	1	0
1	1	0	x	x	x	x	x	x
1	1	1	0	0	1	1	1	0

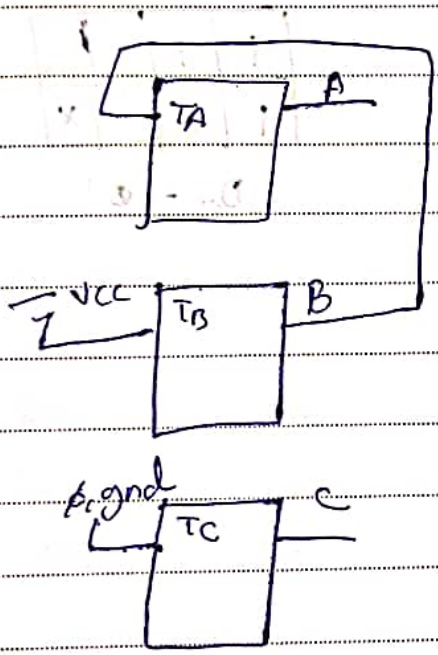
$T_C = 0$

$T_B = 1$

$T_A = \Sigma(3,7) + \Sigma d(0,2,4,6)$

x		1	x
x		1	x

$T_A = B$



EX Design a circuit for the following state table

use JK

Present A	Present B	input x	next A^+	next B^+	output y	J_A	K_A	J_B	K_B
0	0	0	0	0	0	0	X	0	X
0	0	1	0	1	1	0	X	1	X
0	1	0	0	1	0	0	X	X	0
0	1	1	1	0	1	1	X	X	1
1	0	0	1	0	1	X	0	0	X
1	0	1	1	1	1	X	0	1	X
1	1	0	1	1	1	X	0	X	0
1	1	1	0	0	1	X	1	X	1

QQ^+	JK
0 0	0 X
0 1	1 X
1 0	X 1
1 1	X 0

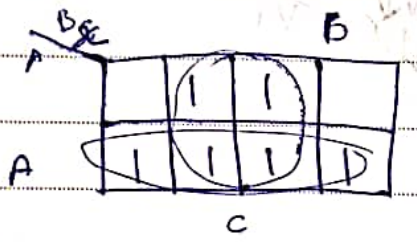
$y = \sum(1,3,4,5,6,7)$

$J_A = \sum(3) + \sum d(4,5,6,7)$

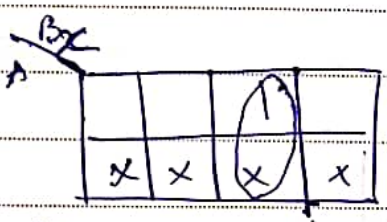
$K_A = \sum(2) + \sum d(0,4,2,3)$

$J_B = \sum(1,5) + \sum d(2,3,6,7)$

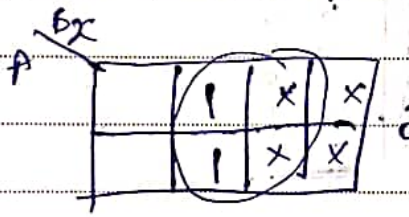
$K_B = \sum(3,7) + \sum d(0,1,4,5)$



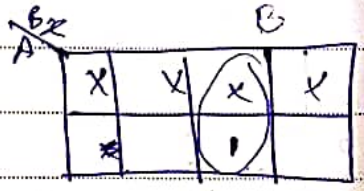
$y = A + x$



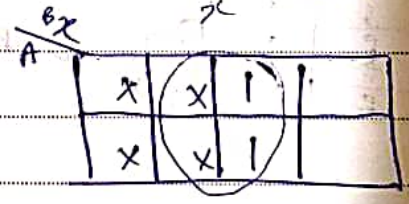
$J_A = Bx$



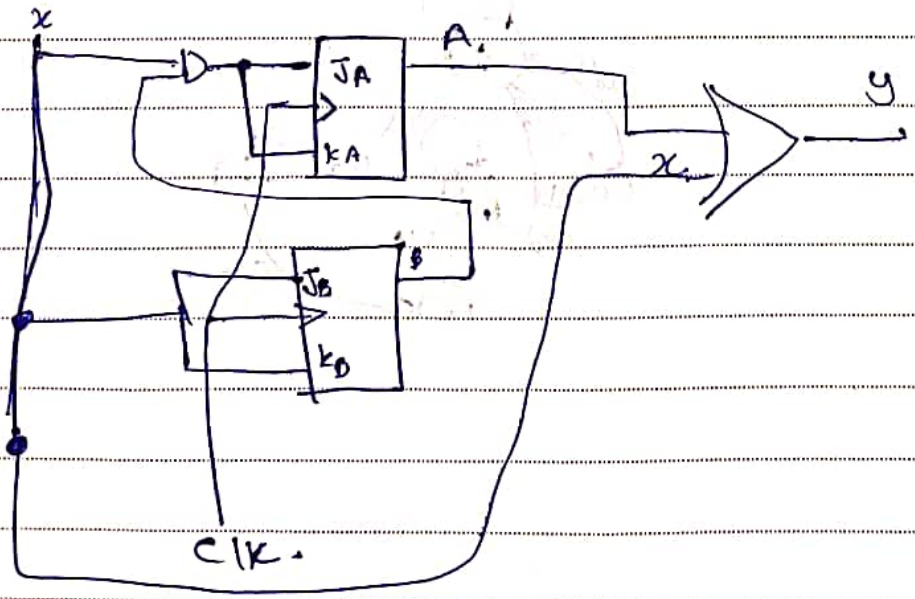
$J_B = x$



$K_A = Bx$



$K_B = x$



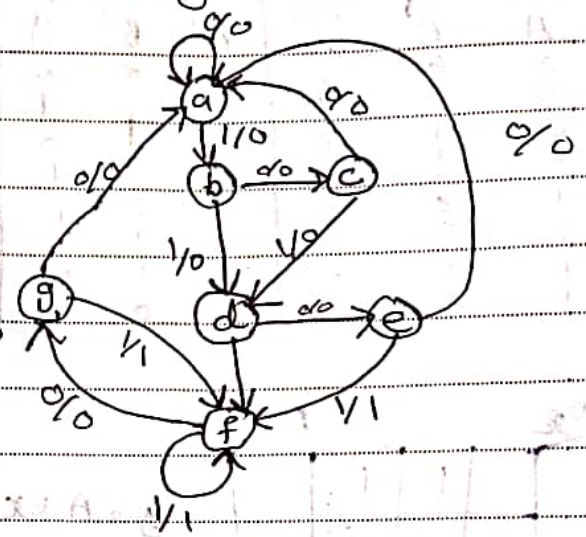
ALADDIN
صنع بالكهارة

* State Reduction

EX Reduce the Following State diagram

Present	next x=0	next x=1	output x=0	output x=1
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
→ g	a	f	0	1

Cancel



∴ g = e

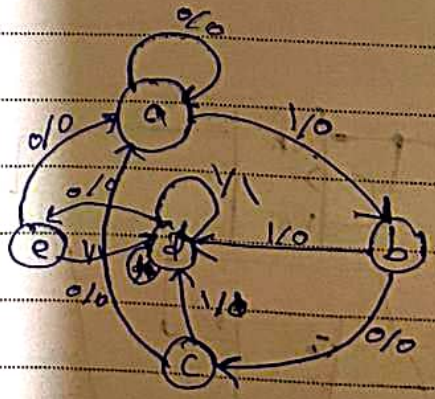
replace g → e

a	a	b	0 0
b	c	d	0 0
c	a	d	0 0
d	e	f	0 1
e	a	f	0 0
f	e	f	0 0

=

P → d

→ a	a	b	0 0
b	c	d	0 0
c	a	d	0 0
d	e	d	0 1
e	a	d	0 1



* Sequence ~~detector~~ detector

- A sequential circuit that takes an input string of bits and generates an output "1" when ever the target sequence has been detected.

EX Design a circuit to detect 110 from given input stream

x 0 1 1 0 1 1 0 1 0 1 1 0 .

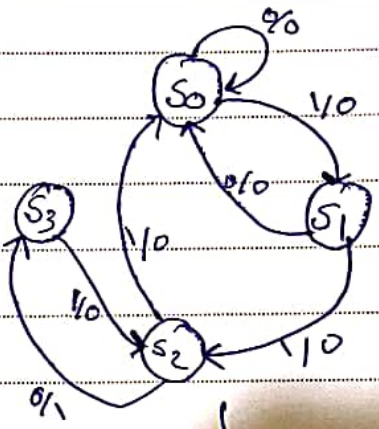
0 0 0 1 0 0 1 0 0 0 0 1

$S_0 = \phi$ (Reset)

$S_1 = 1$

$S_2 = 11$

$S_3 = 110$



↳ State table

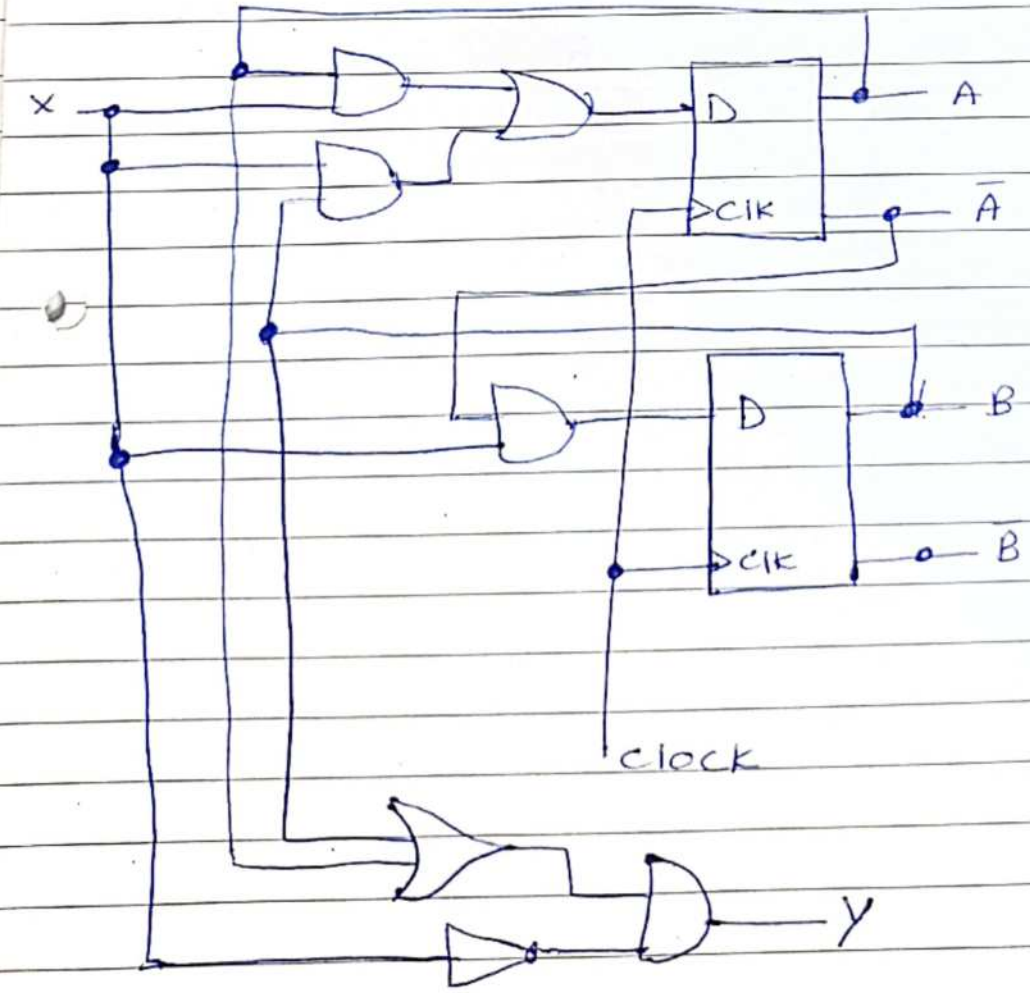
excitation table

* Analysis of clocked sequential Circuits:

- Obtaining a table or diagram for the time sequence of inputs, outputs & internal states.

- The Behaviour of the circuit can be described algebraically by means of state equations (transition equation) ⇒ specifies the next state as a function of the present state & inputs.

ex. Fig 5.15 P210



$A(t+1) = A(t)\,x(t) + B(t)\,x(t)$

$B(t+1) = A'(t)\,x(t)$

$y(t) = [A(t) + B(t)]\,x'(t)$

State Equations

$Q^+ = D$

Mealy model

1) output y is a function of both input x & the present state of A and B

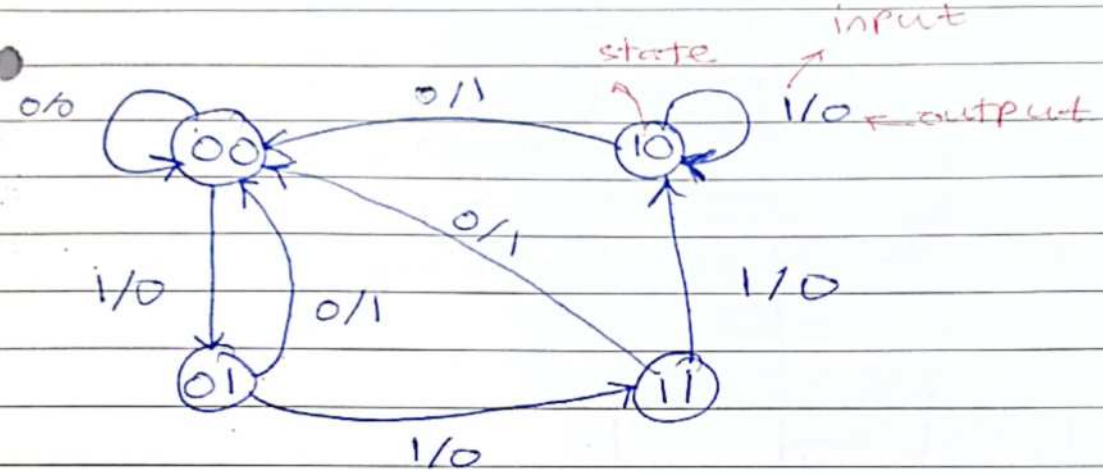
State table.

Present state		input	Next state		output
A	B	x	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

OR

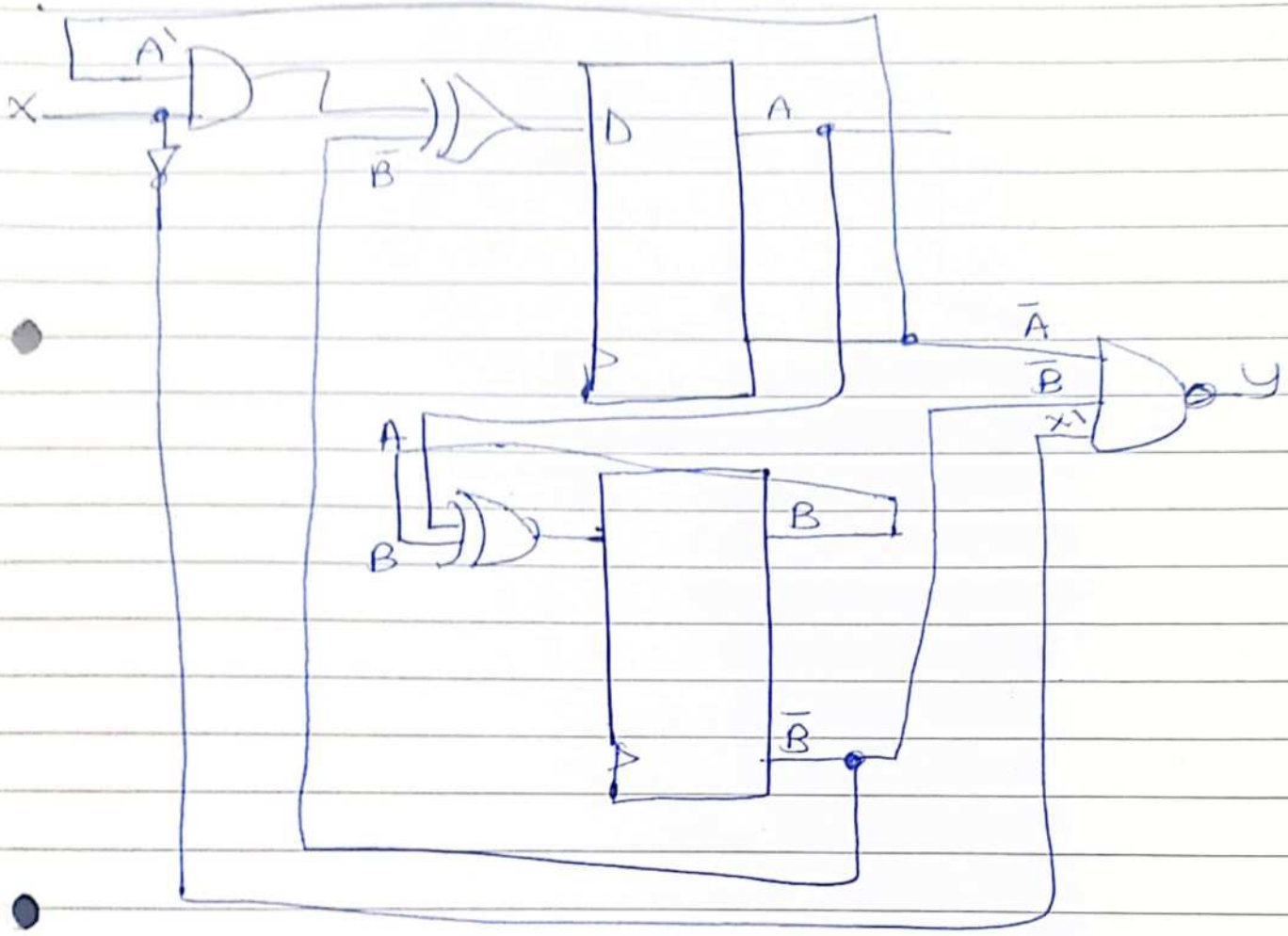
CamScanner

Present state A B	Next state x=0 A B	Next state x=1 A B	output x=0 Y	output x=1 Y
0 0	0 0	0 1	0	0
0 1	0 0	1 1	1	0
1 0	0 0	1 0	1	0
1 1	0 0	1 0	1	0



state diagram

Derive the state table & the State diagram

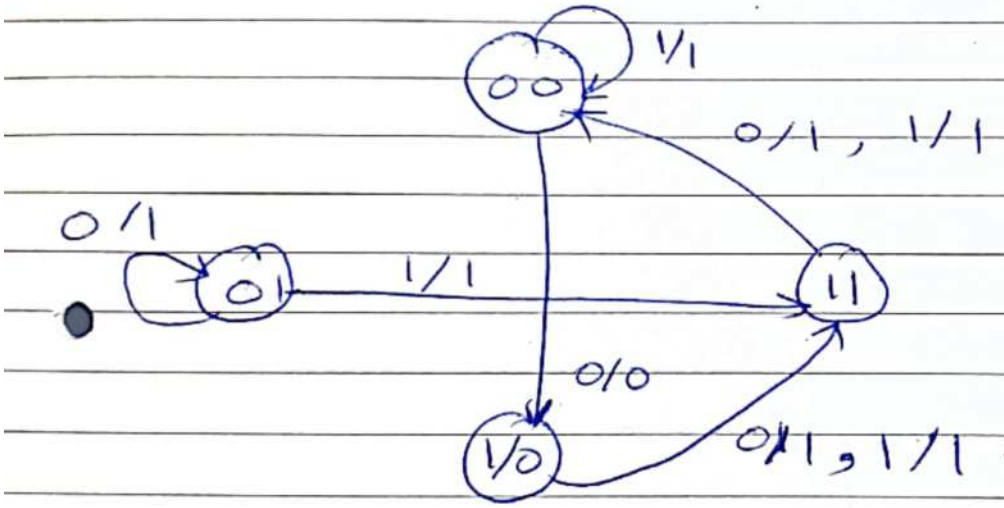


$A(t+1) = A'(t)\, x \oplus B'(t)$

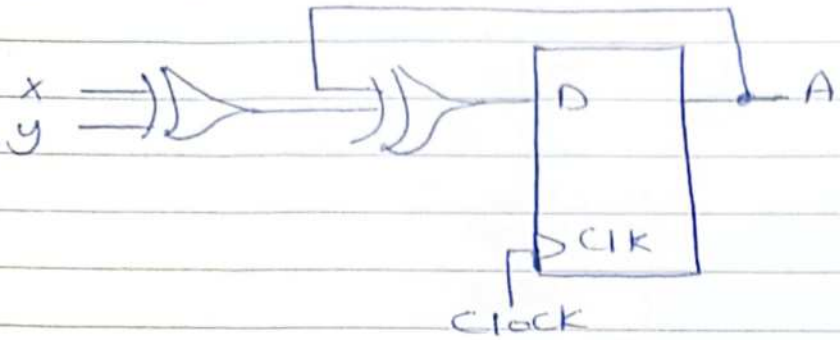
$B(t+1) = A(t) \oplus B(t)$

$y(t) = (A'(t) \cdot B'(t) \cdot x')' = A(t) + B(t) + x$

Present state		Input	Next state		o/p
A	B	x	A	B	y
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	1	0	0	1



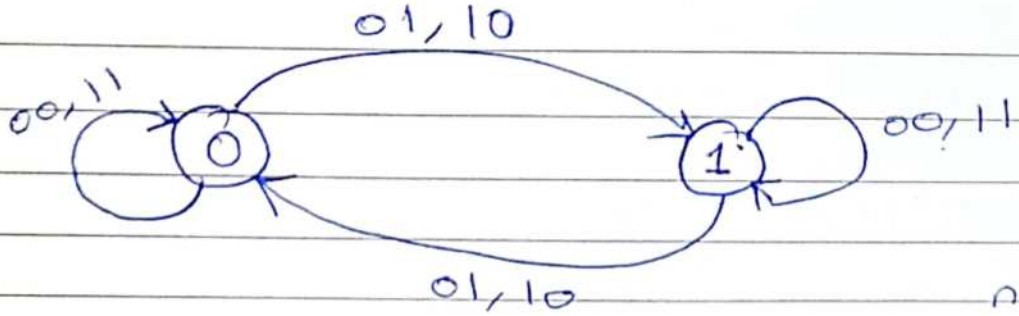
ex.



$A(t+1) = A(t) \oplus x \oplus y$

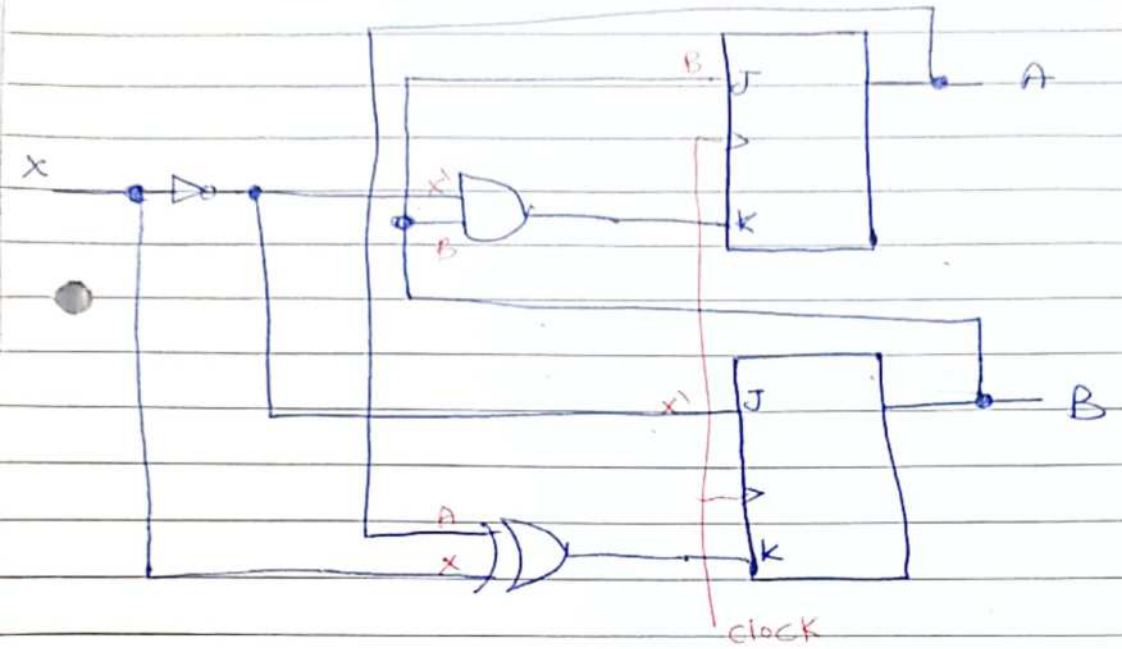
Moore Model output is a function of Present state only.

Present state	Inputs x	Inputs y	Next state
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



no need for slash because there is no output (y) from the comb. circuit.

Ex. Analysis with JK Flip Flops



$Q^+ = JQ' + K'Q$

$$A(t+1) = J_A A' + K_A' A$$
$$B(t+1) = J_B B' + K_B' B$$

$J_A = B$ $\quad K_A = Bx'$

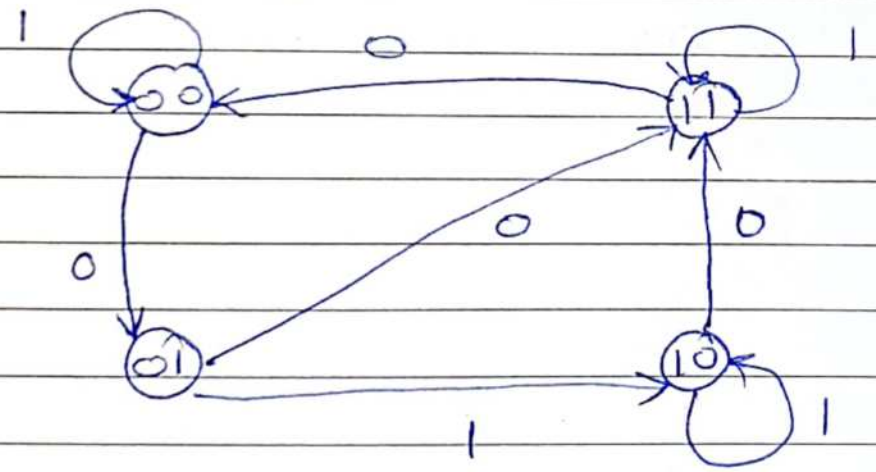
$J_B = x'$ $\quad K_B = A'x + Ax' = A \oplus x$

substitute J_A & K_A in $A(t+1)$

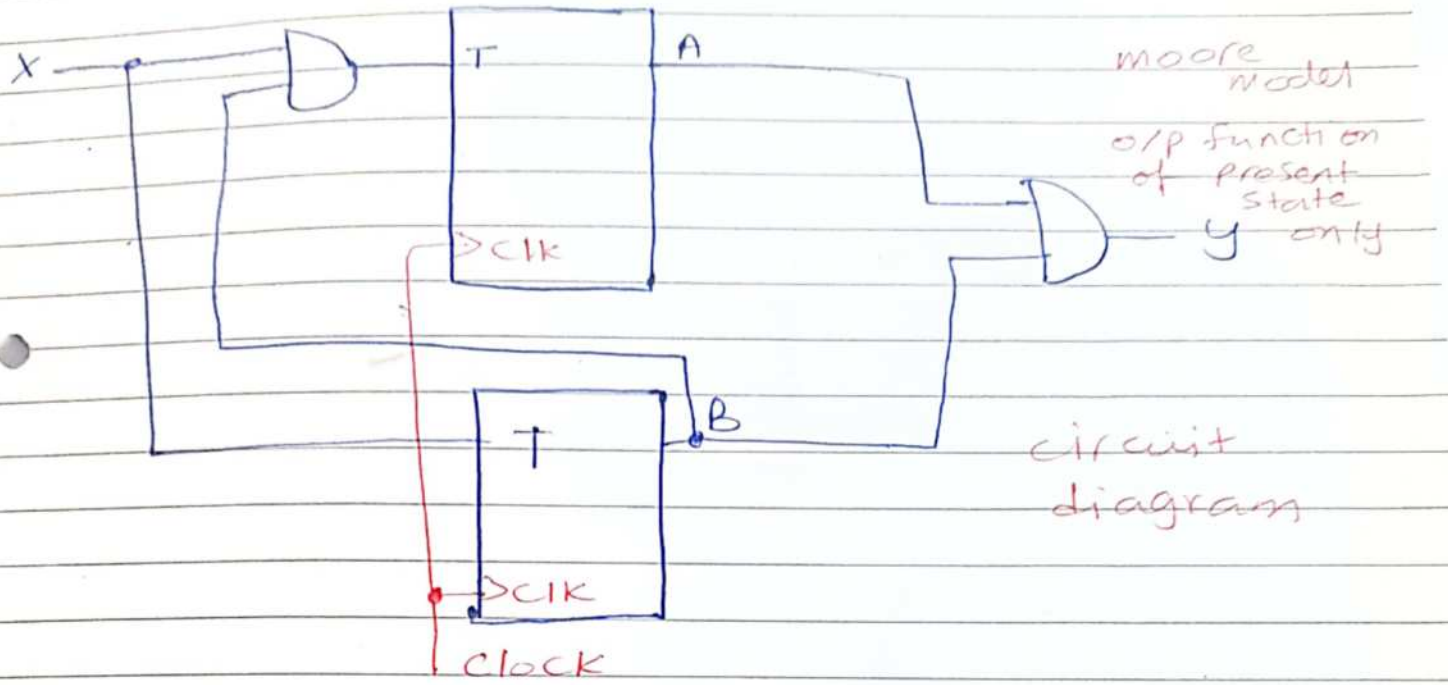
$$\Rightarrow A(t+1) = BA' + (Bx')'A$$
$$= BA' + (B' + x)A$$
$$= A'B + AB' + Ax$$

$$B(t+1) = x'B' + (A \oplus x)' B$$
$$= B'x' + ABx + A'Bx'$$

Present state A	B	Input X	Next state A	B
0	0	0	0	1
0	0	1	0	0
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	1	1



ex Analysis with T Flip-Flops



$Q(t+1) = T \oplus Q = T'Q + TQ'$

- $T_A = Bx$
  $T_B = x$
  $y = AB$

$$A(t+1) = T_A \oplus A$$
$$= T'A + TA'$$
$$= (Bx)'A + BxA'$$
$$= (B' + x')A + BxA'$$
$$= AB' + Ax' + A'Bx$$

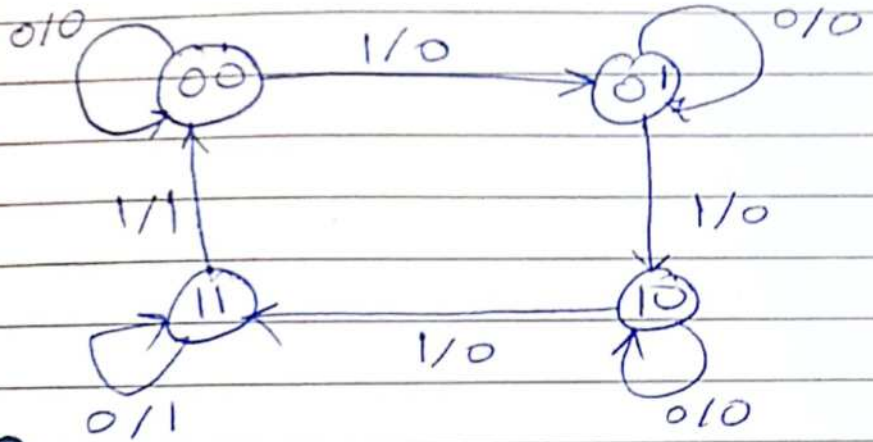
$$B(t+1) = x'B + xB'$$

state Equations

CamScanner

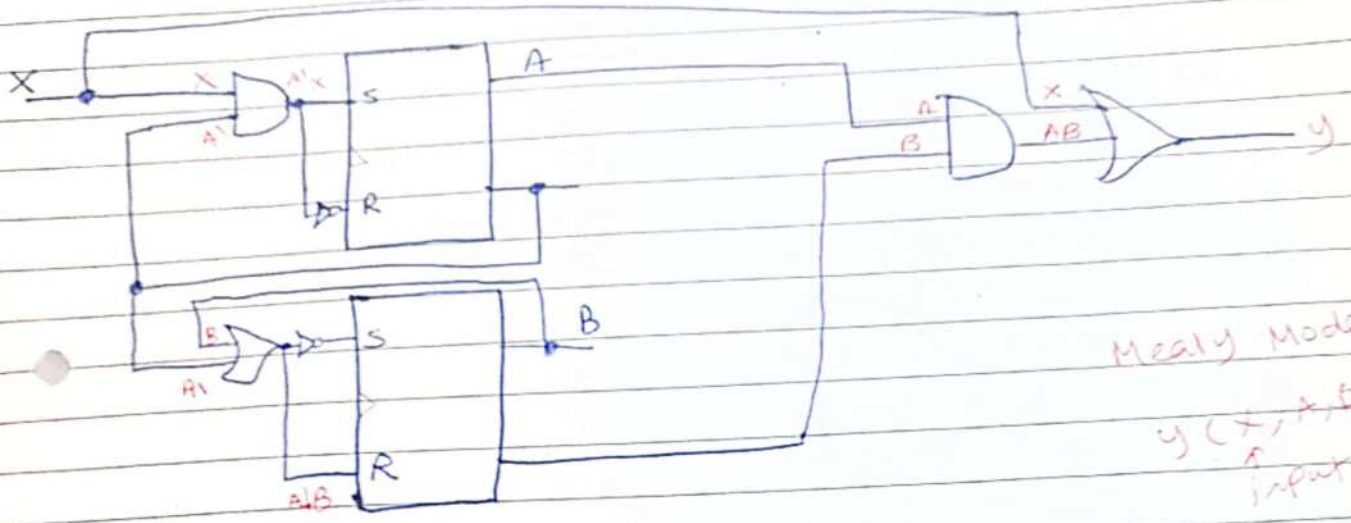
State table

Present state A B	Input x	Next state A B	output y
0 0	0	0 0	0
0 0	1	0 1	0
0 1	0	0 1	0
0 1	1	1 0	0
1 0	0	1 0	0
1 0	1	1 1	0
1 1	0	1 1	1
1 1	1	0 0	1



state diagram

ex. SR



Mealy Model

$y(x, A, B)$ ← input present

$S_A = A'x$, $R_A = (A'x)' = A + x'$

$S_B = (A' + B)' = AB'$, $R_B = A' + B$

$y(t) = x + AB'$

$Q+ = S + R'Q$

$A(t+1) = S_A + (R_A' \cdot A) = A'x + A'xA = A'x$

$B(t+1) = S_B + R_B' \cdot A = AB' + AB'A = AB'$

Present A B	input x	next A B	output y
0 0	0	0 0	0
0 0	1	1 0	1
0 1	0	0 0	0
0 1	1	1 0	1
1 0	0	0 1	1
1 0	1	0 1	
1 1	0	0 0	1
1 1	1	(11) 0 0	0

Flip Flops:-

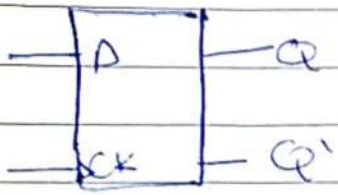
- The D-Flip Flop is the most economical & efficent Flip Flop.
- Other types of Flip Flops (JK, T, SR) can be constructed from the D-flip-flop.
- Three operations can be performed with a flip-flop

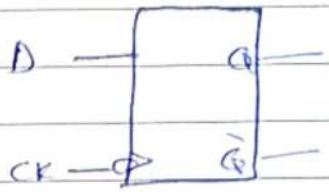
① set to "1".
② Reset to "0".
③ Complement Q

D-flip Flop

- Has two inputs ① clock (CLK) denoted by a small arrow head. ② Data (D).
- The output of the D Flip-Flop changes in response to the clock input only, not in response to a change in the D-input.
- The D Flip Flop is edge-triggered not level sensitive.
- positive (rising) edge triggered "0" → "1"
- negative (falling) edge triggered "1" → "0"

CamScanner





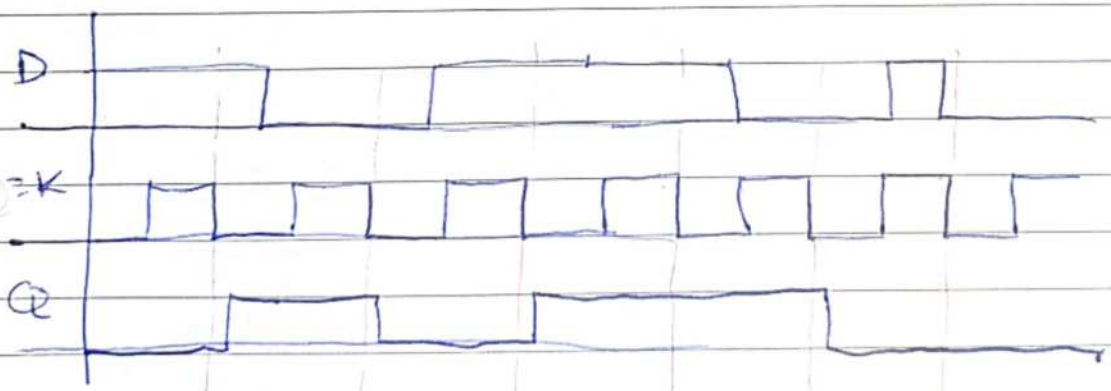
rising edge triggered

Falling edge trigger

D	Q	Q+
0	0	0
0	1	0
1	0	1
1	1	1

$Q^+ = D$ CC eqn.

Falling edge.



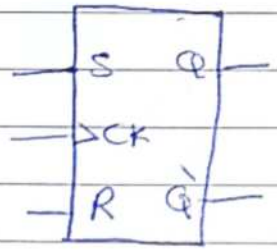
SR Flip-Flop:-

- Has three inputs
  1. Clock (CK)
  2. Set
  3. Reset.

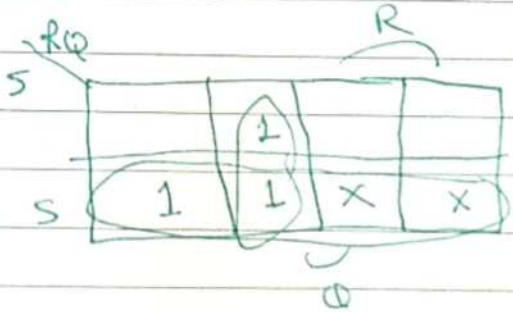
- Similar to SR Latch
  S = "1" sets the FF (Q^+ = "1")
  R = "1" Resets the FF (Q^+ = "0")

- Like the D-FF, the Q only changes in response to an active clock edge

- positive edge-triggered OR negative-edge triggered.



S	R	Q	Q^+	
0	0	0	0	
0	0	1	1	store $Q^+ = Q$
0	1	0	0	
0	1	1	0	Reset $Q^+ = 0$
1	0	0	1	
1	0	1	1	set $Q^+ = 1$
1	1	0	X	not allowed
1	1	1	X	don't care (3)



Char. eqn.

$Q^+ = S + R'Q$

$Q(t+1) = S + R'Q(t)$

JK - FlipFlop:

- Has 3 inputs J, K and clk.
  ↑ Set ↓ Reset

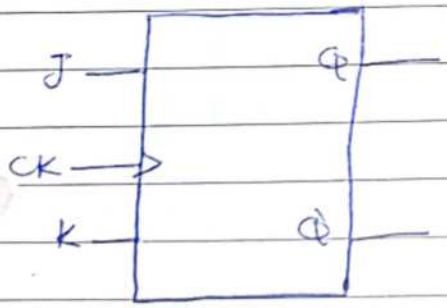
- performs three operations:

① set JK = 10

② Reset JK = 01

③ complement JK = 11

④ store JK = 00



J	K	Q	Q+	
0	0	0	0	store
0	0	1	1	Q+ = Q
0	1	0	0	Reset
0	1	1	0	Q+ = 0
1	0	0	1	Set
1	0	1	1	Q+ = 1
1	1	0	1	complement
1	1	1	0	toggle
				Q+ = Q'

char. equation

$Q^+ = JQ' + K'Q$

$Q(t+1) = JQ'(t) + K'Q(t)$

4) T- Flip Flop.

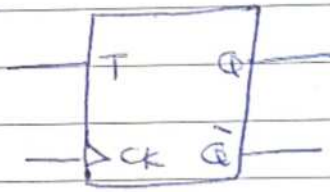
- Toggle (T) Flip-Flop has two inputs CK (Clock) & Toggle (T)

- The T-input controls the state change

T=0, State does not change $Q+ = Q$
T=1, state changes on active trigger
$Q+ = Q'$

- Often used in the design of counters.

T	Q	Q+
0	0	0
0	1	1
1	0	1
1	1	0



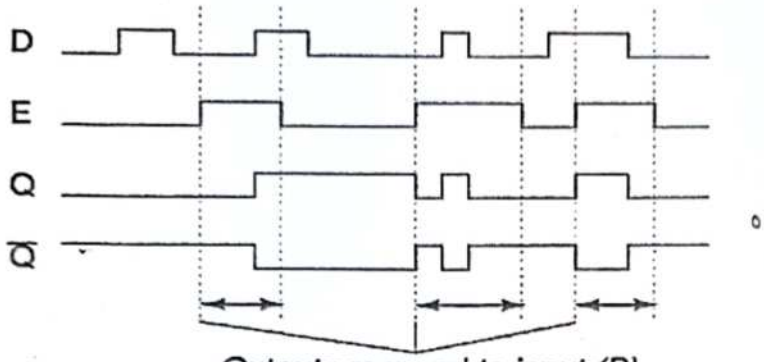
char. equation

$$Q+ = TQ' + T'Q$$

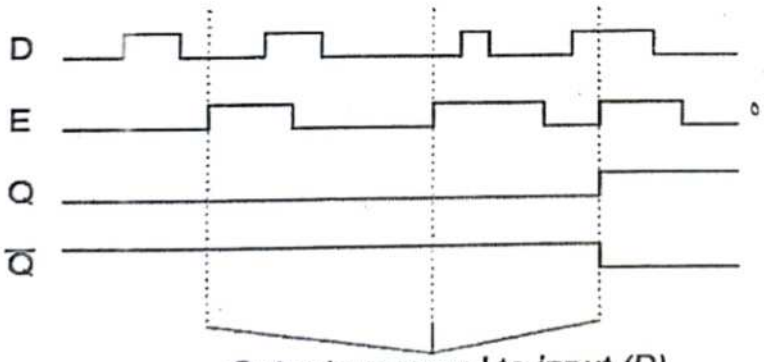
$$= T \oplus Q$$

CamScanner

Regular D-latch response

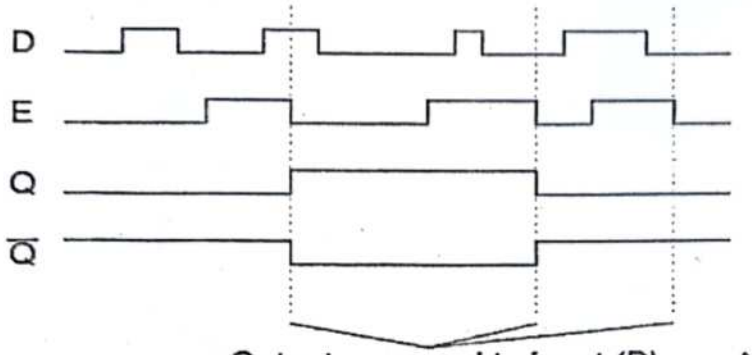


Positive edge-triggered D-latch response



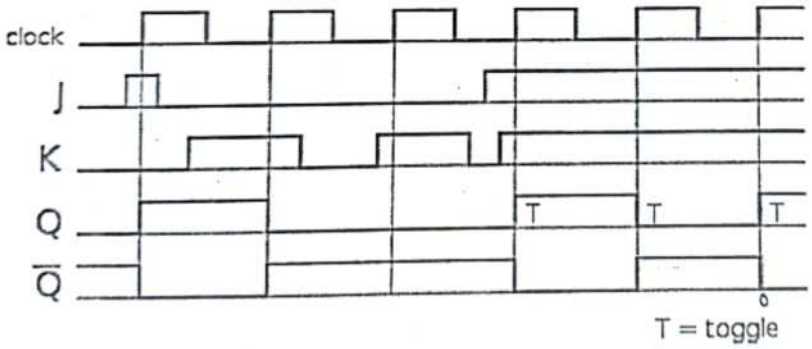
CamScanner

Negative edge-triggered D-latch response



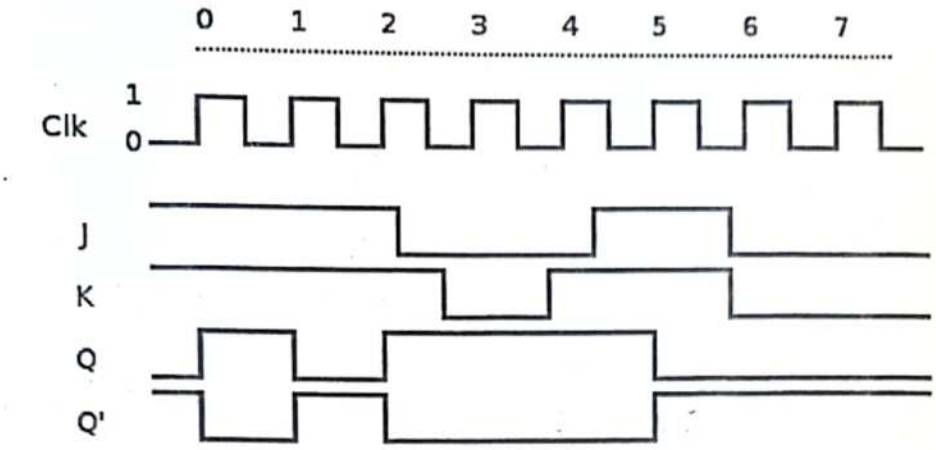
Outputs respond to input (D) only when enable signal transitions from high to low

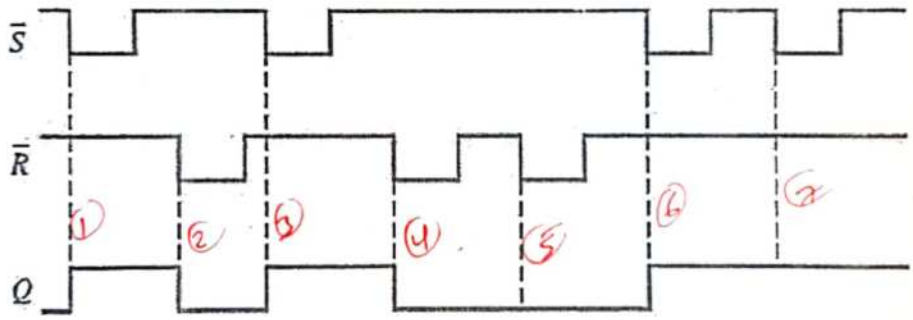
+ve edge

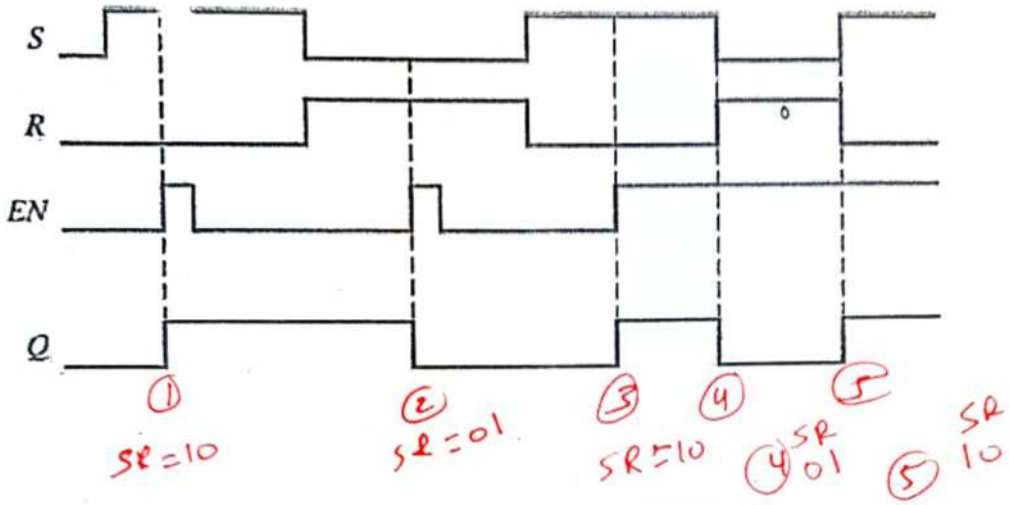


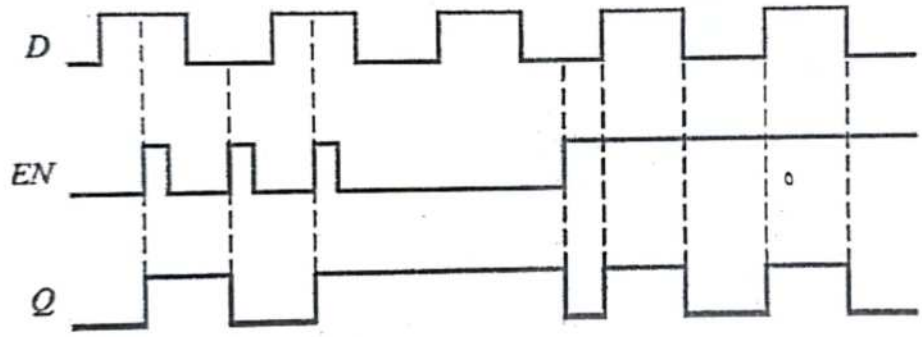
T = toggle

CamScanner


0 1 2 3 4 5 6 7
Clk
1
0
J
K
Q
Q'


$\overline{S}$
$\overline{R}$
Q
1
2
3
4
5
6
7

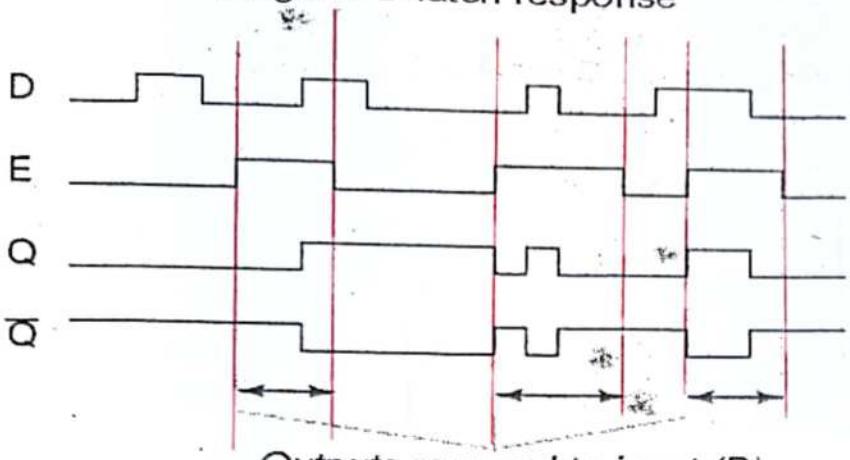




CamScanner

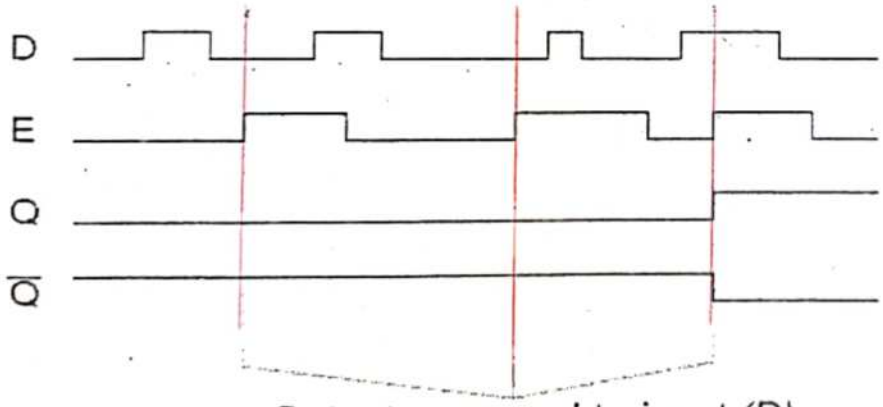
المهندسة
سماح عبد الهادي مساعده

Regular D-latch response



Outputs respond to input (D) during these time periods EN=1

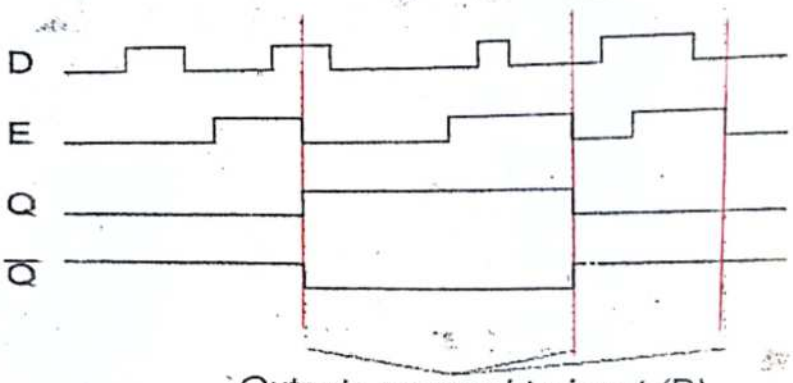
Positive edge-triggered D-latch response



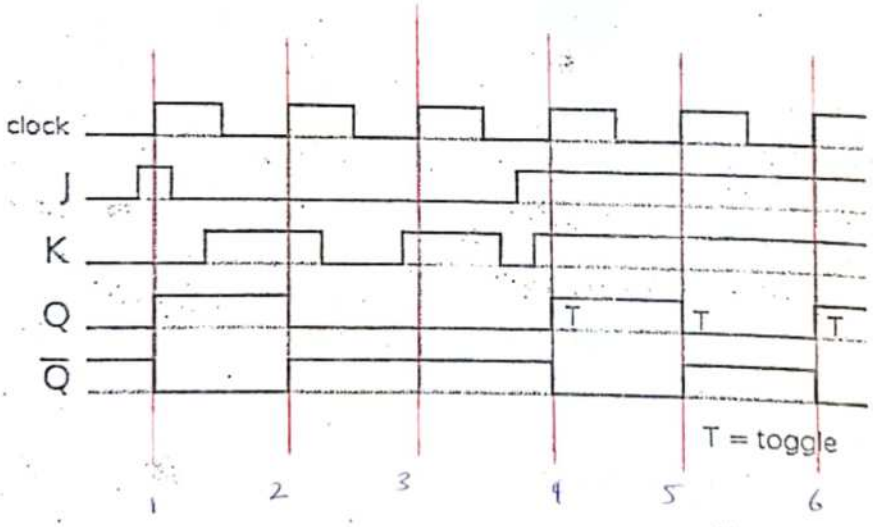
Outputs respond to input (D) only when enable signal transitions from low to high

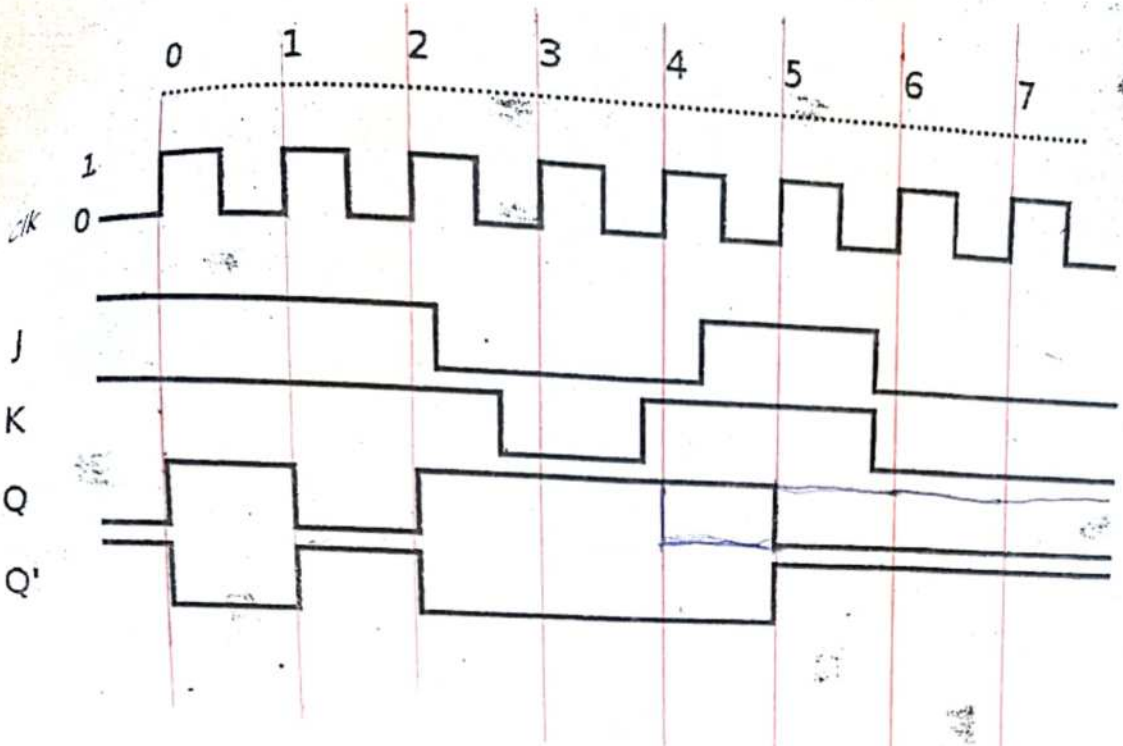
CamScanner

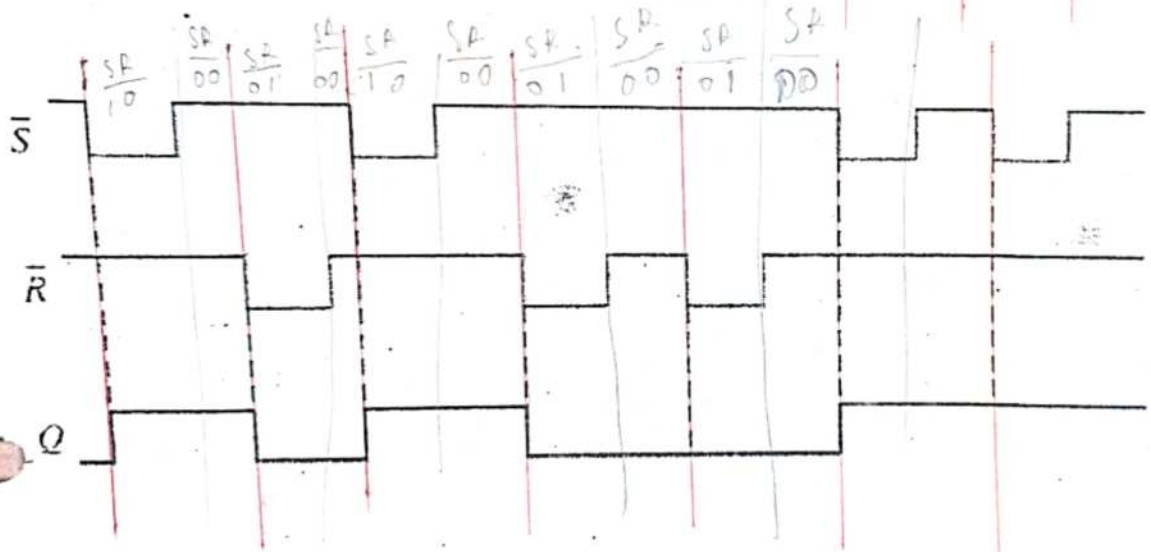
Negative edge-triggered D-latch response



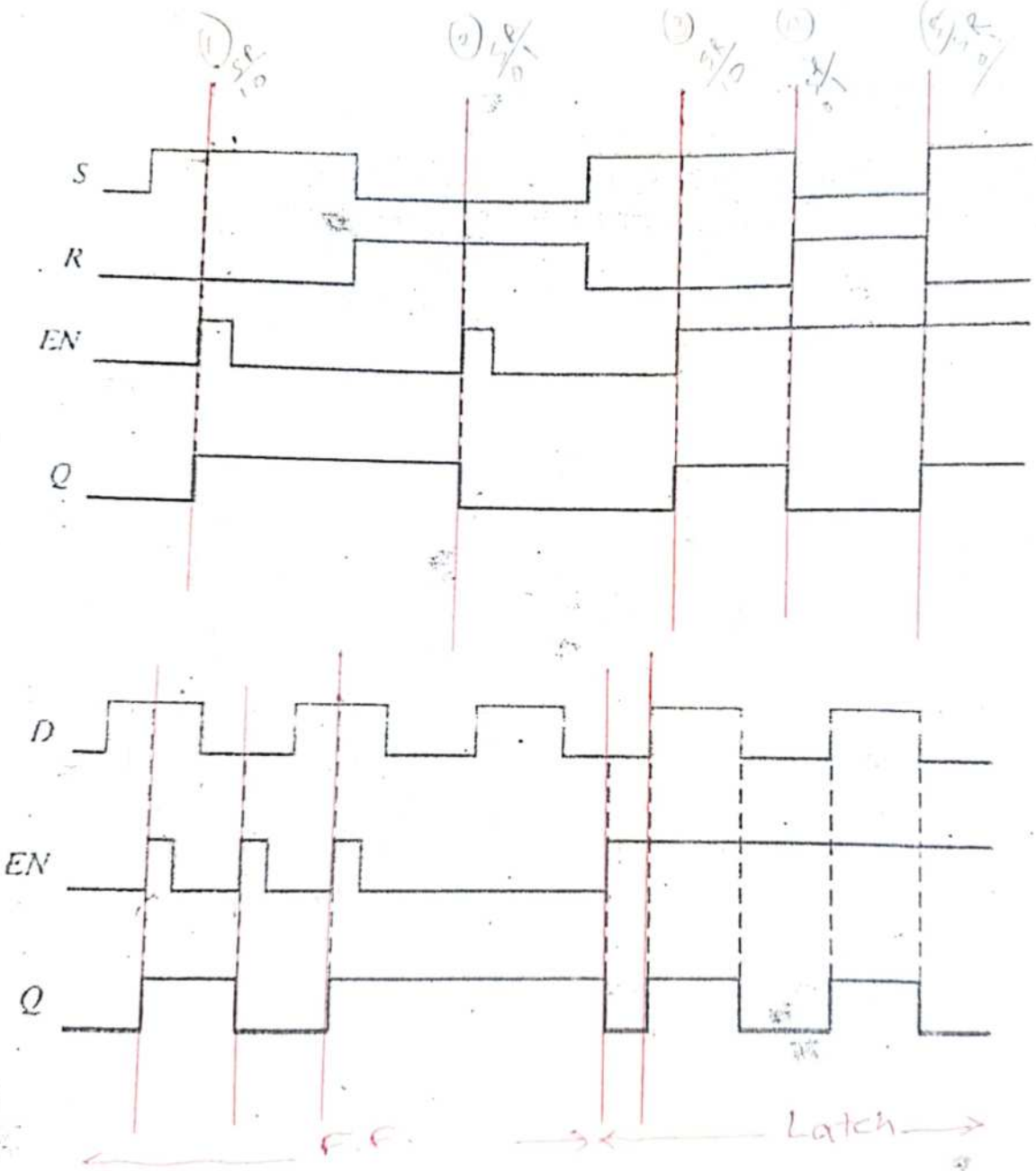
Outputs respond to input (D) only when enable signal transitions from high to low




0
1
2
3
4
5
6
7
1
0
CLK
J
K
Q
Q'


SR 10
SR 00
SR 01
SR 00
SR 10
SR 00
SR 01
SR 00
SR 01
SR 00
$\overline{S}$
$\overline{R}$
Q

CamScanner


S
R
EN
Q
D
EN
Q
F.F.
Latch

mealy y(A,B)
moore y(A,B)

* Design of sequential Circuits:-

*Excitation Tables.

Q	Q+	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

S	R	Q	Q+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	x
1	1	1	x

SR-FF

$Q^+ = S + R'Q$

Q	Q+	D
0	0	0
0	1	1
1	0	0
1	1	1

D	Q	Q+
0	0	0
0	1	0
1	0	1
1	1	1

D-FF

Q	Q+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

J	K	Q	Q+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

JK-FF

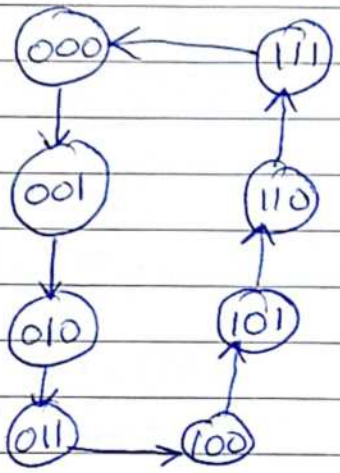
$Q^+ = JQ' + K'Q$

Q	Q+	T
0	0	0
0	1	1
1	0	1
1	1	0

T	Q	Q+
0	0	0
0	1	1
1	0	1
1	1	0

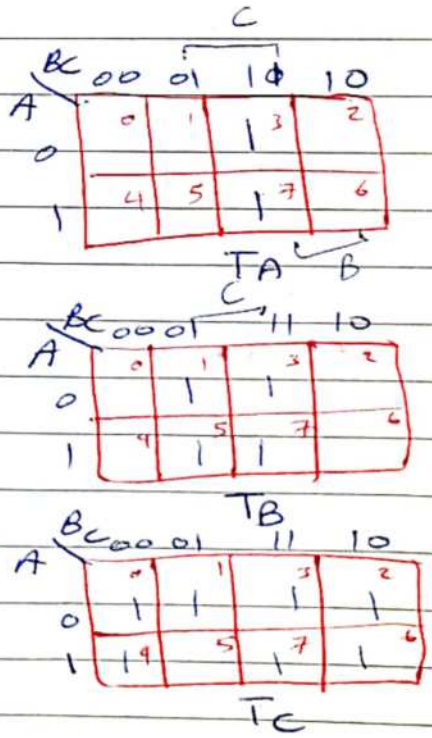
T-FF

ex. Design a 3 bit up counter using T-FF



Q	Q_{t+}	T
0	0	0
0	1	1
1	0	1
1	1	0

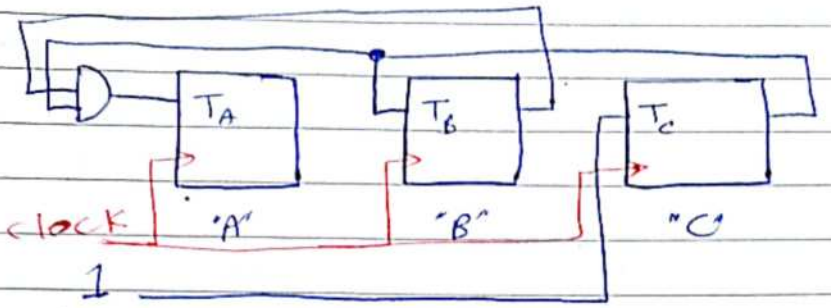
	Current state			next state			FF inputs		
	A	B	C	A	B	C	T_A	T_B	T_C
0	0	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	0	1	1
2	0	1	0	0	1	1	0	0	1
3	0	1	1	1	0	0	1	1	1
4	1	0	0	1	0	1	0	0	1
5	1	0	1	1	1	0	0	1	1
6	1	1	0	1	1	1	0	0	1
7	1	1	1	0	0	0	1	1	1



$T_A = BC$

$T_B = C$

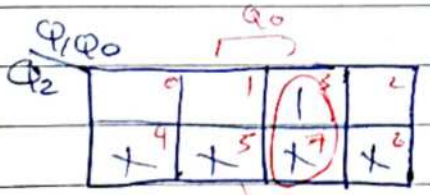
$T_C = 1$



2 Design a 3-bit up counter using JK FF.

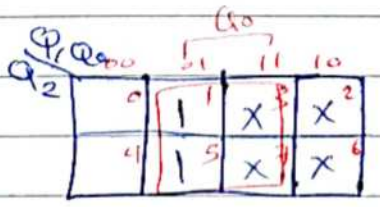
	Present $Q_2Q_1Q_0$	next $Q_2Q_1Q_0$	F.F. inputs J_2K_2	J_1K_1	J_0K_0
0	0 0 0	0 0 1	0 X	0 X	1 X
1	0 0 1	0 1 0	0 X	1 X	X 1
2	0 1 0	0 1 1	0 X	X 0	1 X
3	0 1 1	1 0 0	1 X	X 1	X 1
4	1 0 0	1 0 1	X 0	0 X	1 X
5	1 0 1	1 1 0	X 0	1 X	X 1
6	1 1 0	1 1 1	X 0	X 0	1 X
7	1 1 1	0 0 0	X 1	X 1	X 1

Q_nQ_{n+1}	JK
0 0	0 X
0 1	1 X
1 0	X 1
1 1	X 0



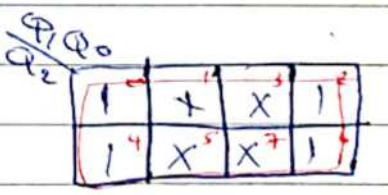
J_2

$J_2 = Q_0Q_1$



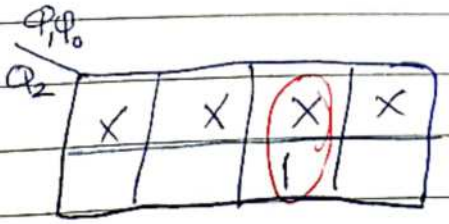
J_1

$J_1 = Q_0$



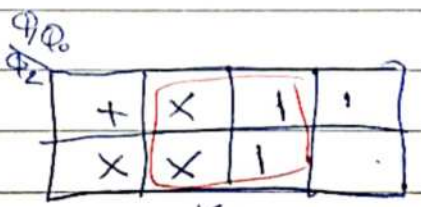
J_0

$J_0 = 1$



K_2

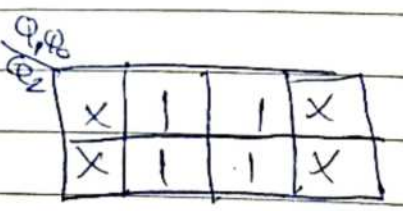
$K_2 = Q_0Q_1$



K_1

$K_1 = Q_0$

3



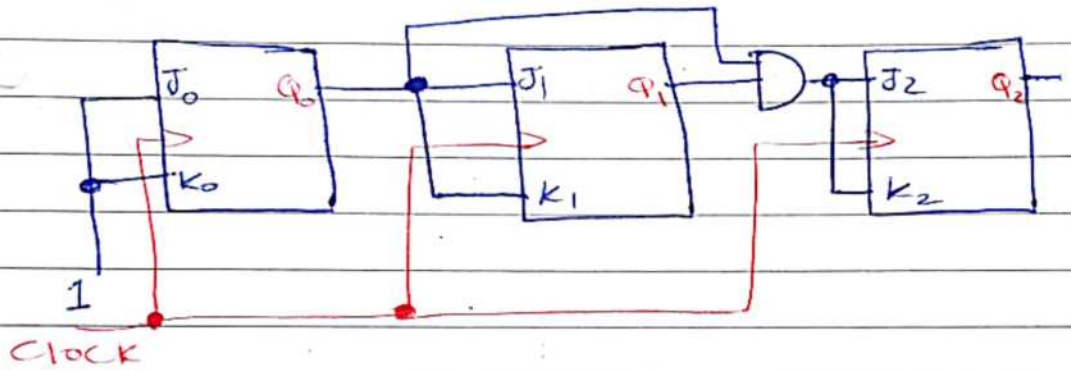
K_0

$K_0 = 1$

$J_0 = K_0 = 1$

$J_1 = K_1 = Q_0$

$J_2 = K_2 = Q_1 Q_0$

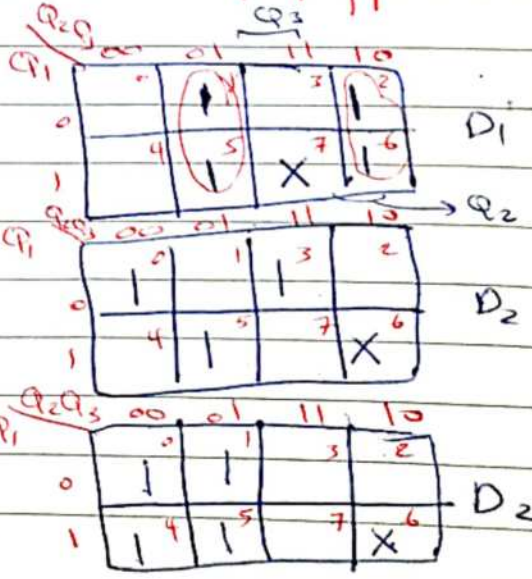


ex (3) Design a counter that goes through the sequence

0, 3, 2, 4, 1, 5, 7 and repeat.

Present $Q_1 Q_2 Q_3$	next $Q_1 Q_2 Q_3$	$D_1 D_2 D_3$
0 0 0 0	0 1 1	0 1 1
1 0 0 1	1 0 1	1 0 1
2 0 1 0	1 0 0	1 0 0
3 0 1 1	0 1 0	0 1 0
4 1 0 0	0 0 1	0 0 1
5 1 0 1	1 1 1	1 1 1
6 1 1 0	X X X	X X X
7 1 1 1	0 0 0	0 0 0

Q	Q+	D
0	0	0
0	1	1
1	0	0
1	1	1

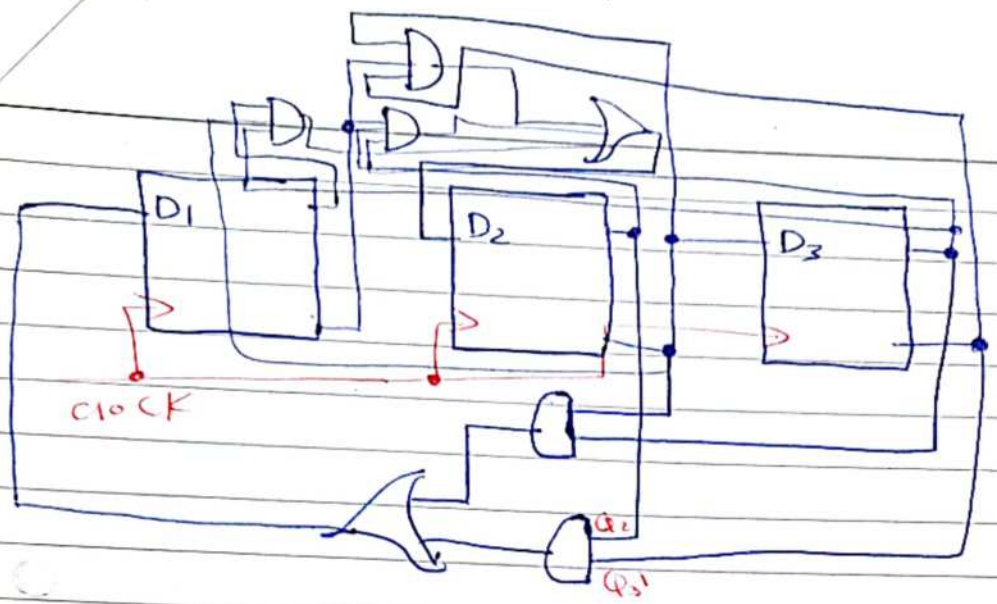


$D_1 = Q_2' Q_3 + Q_2 Q_3'$

$D_2 = Q_1' Q_2' Q_3' + Q_1' Q_2 Q_3 + Q_1 Q_2' Q_3$

$D_3 = Q_2'$

(4)



JK

$J_1 = Q_2' Q_3 + Q_2 Q_3'$

$J_2 = Q_1' Q_3' + Q_1 Q_3$

$J_3 = Q_2'$

$K_1 = Q_3' + Q_2$

$K_2 = Q_1 + Q_3'$

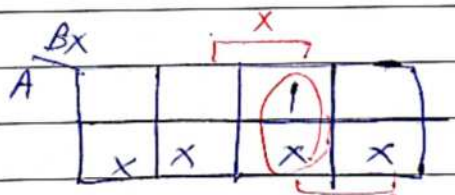
$K_3 = Q_2$

ex (4)

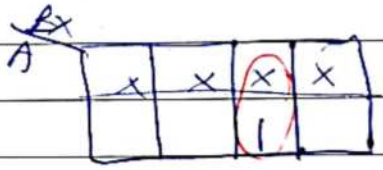
Design a sequential circuit that implement the following State table, use JK F.F.

Present	input	next	output	F.F
AB	x	AB	y	$J_A K_A$ $J_B K_B$
0 0	0	0 0	0	0 x 0 x
0 0	1	0 1	1	0 x 1 x
0 1	0	0 1	0	0 x x 0
0 1	1	1 0	1	1 x x 1
1 0	0	1 0	1	x 0 0 x
1 0	1	1 1	1	x 0 1 x
1 1	0	1 1	1	x 0 x 0
1 1	1	0 0	1	x 1 x 1

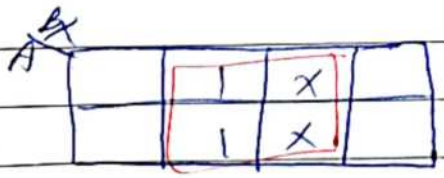
Q	Q+	J K
0	0	0 x
0	1	1 x
1	0	x 1
1	1	x 0



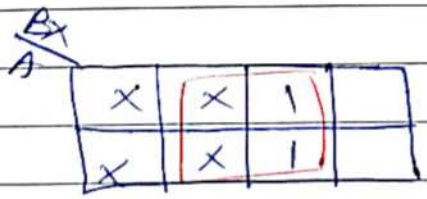
$J_A = XB$



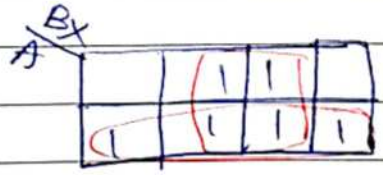
$K_A = XB$



$J_B = x$

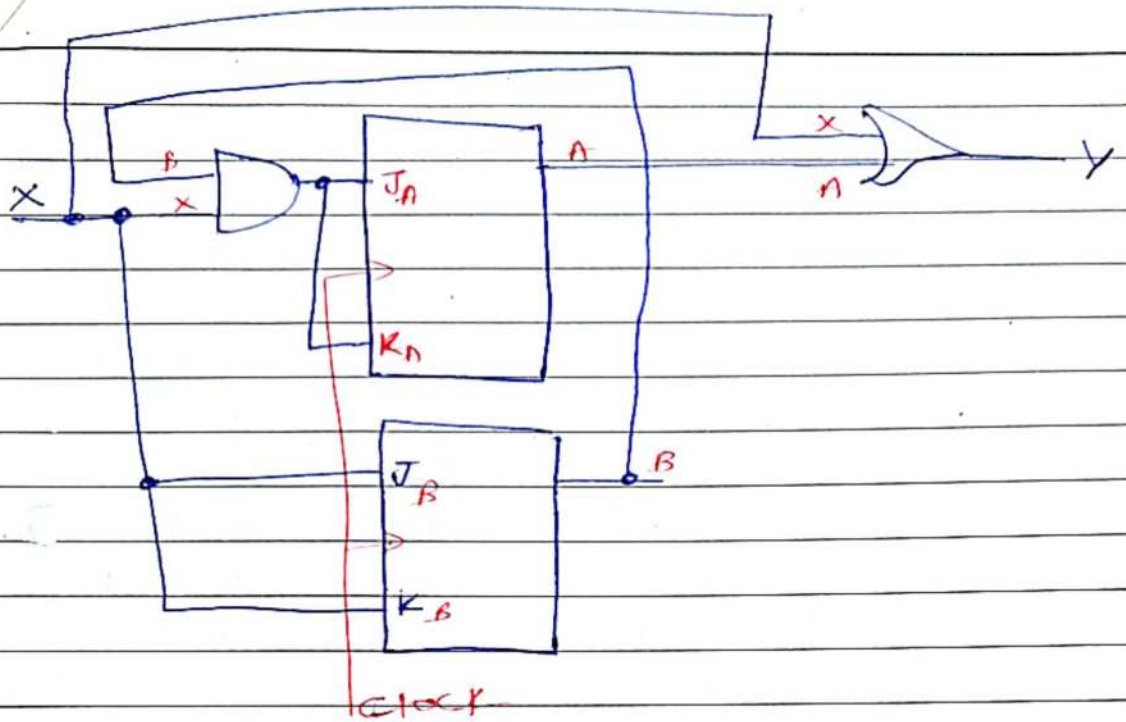


$K_B = x$



$y = X + A$

"mealy machine"


x
B
x
J_A
A
K_A
x
A
Y
J_B
B
K_B
Clock

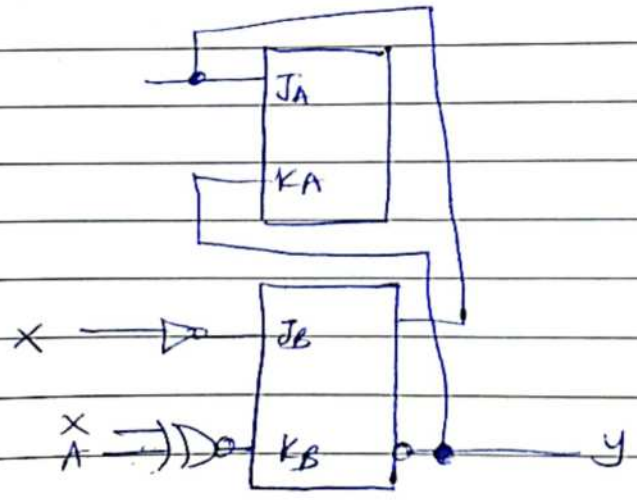
ex. Use JK FF to design

Present	input	next	output	FF.			
AB	x	AB	y	J_A	K_A	J_B	K_B
0 0	0	0 1	1	0	x	1	x
0 0	1	0 0	1	0	x	0	x
0 1	0	1 0	0	1	x	x	1
0 1	1	1 1	0	1	x	x	0
1 0	0	0 1	1	x	1	1	x
1 0	1	0 0	1	x	1	0	x
1 1	0	1 1	0	x	0	x	0
1 1	1	1 0	0	x	0	x	1

$J_A = B$, $K_A = \bar{B}$

$J_B = x'$, $K_B = A \oplus x$

$y = B'$ (Moore Machine)



ex 6

Design an even counter

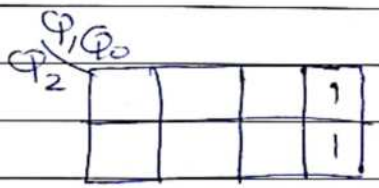
0 → 2 → 4 → 6 ↺

Using T-FF

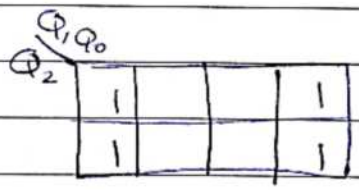
TFF excitation

Q	Q+	T
0	0	0
0	1	1
1	0	1
1	1	0

Present	Next	FF Inputs
$Q_2 Q_1 Q_0$	$Q_2^+ Q_1^+ Q_0^+$	$T_2 T_1 T_0$
0 0 0	0 1 0	0 1 0
0 1 0	1 0 0	1 1 0
1 0 0	1 1 0	0 1 0
1 1 0	0 0 0	1 1 0



$T_2 = Q_1 Q_0'$



$T_1 = Q_0'$

$T_0 = \phi$

ex (7)

Design the following counter (with unused states) using SR FF.

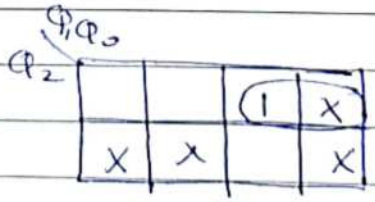
$0 \to 1 \to 3 \to 4 \to 6 \to 7$

(2, 5) unused states (x for simplification)

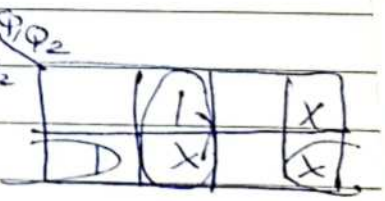
Present	next	FF-inputs		
$Q_2Q_1Q_0$	$Q_2^+Q_1^+Q_0^+$	S_2R_2	S_1R_1	S_0R_0
000	001	0x	0x	10
001	011	0x	10	x0
010	xxx	xx	xx	xx
011	100	10	01	01
100	110	x0	10	0x
101	xxx	xx	xx	xx
110	111	x0	x0	10
111	000	01	01	01

SR excitation table

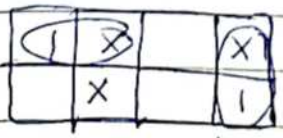
QQ^+	SR
00	0x
01	10
10	01
11	x0



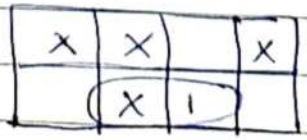
$S_2 = \bar{Q}_2 Q_1$



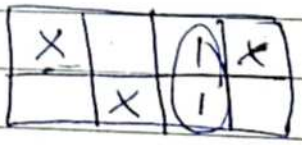
$S_1 = Q_1' Q_0 + Q_2 Q_0'$



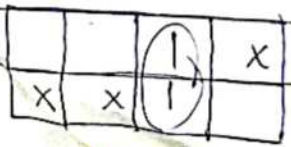
$S_0 = Q_2' Q_1' + Q_1 Q_0$



$R_2 = Q_2 Q_0$



$R_1 = Q_1 Q_0$



$R_0 = Q_1 Q_0$